Published by Red Hat Developer

The Grumpy Developer's Guide to OpenShift



Ian "Uther" Lawson

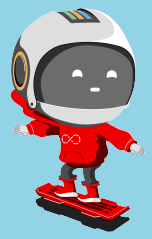
Red Hat Developer

The pace of building and delivering innovative applications has never been higher, creating complex challenges for developers. Red Hat Developer provides the technical tools and expert knowledge to help you succeed.

As a member, you unlock the following benefits:

- **Access to latest betas:** Experiment with the latest Red Hat technologies and new features to transform your application development.
- **Curated learning resources:** Advance your technical knowledge and practice the skills you need to stay ahead with tailored resources that propel your learning journey.
- **Events and webinars:** Connect, collaborate, and learn directly from Red Hat experts through virtual and in-person events.
- **No-cost access to the Developer Sandbox:** Instantly try products like Red Hat OpenShift and Red Hat Ansible Automation Platform, developer tools, and Red Hat cloud services—no cost, no setup.
- **1-year subscription to Red Hat products:** Get access to a full portfolio of products and tools for development use to create enterprise software built on microservices, containers, and the cloud.
- **Red Hat Customer Portal access:** Find immediate answers and expert guidance through Red Hat's extensive knowledge base and community forums.

Ready to get started? Register and learn more at developers.redhat.com.



Contents

Foreword....5
Introduction: The grumpy developer....7
0. The elegance of OpenShift and Kubernetes....13
Part 1: Fun-damentals....33
1. Code to container....35
2. Deploying your creation....53
3. Changing stuff without changing the image....63
Part 2: Infrastructure: The stuff around the application....79
4. Wiring it up: The SDN....81
5. What (and how) is my app doing?....101
6. State of state, persisting in an ephemeral world....109
7. Pass the tray, Jenkins....119
8. GitOps and the state watchman, Argo CD....153
Part 3: The fun stuff....161
9. Serverless with Knative....163
10. A developer's guide to OpenShift security (and little robots)....177
11. Those pesky legacy VMs....183
12. Hello, operator? I'd like a new object....187
Postscript: Developers are artists....189
About the author....191

Foreword

It is always a pleasure to see new content from Ian "Uther" Lawson, whether a blog post, video, nonfiction, or fiction[1]). This book is no exception.

Aimed squarely at developers, *The Grumpy Developer's Guide to OpenShift* helps you get the most out of using Red Hat OpenShift to build and deploy your applications. Uth provides the practical information you need to apply concepts like Knative for serverless applications, security, and virtual machines to your daily work.

You can use the Red Hat Developer Sandbox at no cost to build and test the examples provided in these chapters. This hands-on aspect helps reinforce the concepts and practical hints found throughout this guide.

With over a decade of experience helping customers adopt OpenShift, Uth is uniquely qualified to guide you through this topic. He has worked with the platform since the OpenShift V2 days, before it was even using Kubernetes. At Red Hat, he collaborates with customers and colleagues to ensure they get the most from the technology. I have worked alongside Uth to support customers and co-deliver guest lectures at University College London. Whatever the circumstances or audience, he takes complex topics and explains them clearly in a way that is easy to remember.

1 The mention of "fiction" might intrigue you, as it did me a few years ago. I was delighted to find out that Uth also wrote books of fiction. As his unofficial (and self-appointed) literary agent, I encourage you to search for Ian "Uther" Lawson on your favorite online book site.

The book starts with the basics of containers, pods, and OpenShift to help you get your applications running quickly and efficiently. Uth explains topics like deployment options, using environment variables, and how to use secrets to run your applications.

In Part 2, he discusses the infrastructure around your application on OpenShift, including network access, application health checks, and storing state and data. This section concludes with a look at automating your application builds using Tekton and getting your applications to production using Argo CD as part of Red Hat OpenShift GitOps.

Finally, the book covers the fun stuff. These chapters are helpful for modern developers using serverless technologies like Knative in their applications. Uth explains how to run and manage virtual machines just like pods and containers. This section also covers connecting virtual machines to your pods to form even better applications and services. The book wraps up with an introduction to Operators, which are an important part of OpenShift (even if they were originally all written in Go!).

As a developer, you should understand how to use Kubernetes to build applications for the hybrid cloud. OpenShift is a platform built on Kubernetes and other open source technologies from the Cloud Native Computing Foundation (CNCF). This guide helps you learn the basics of OpenShift and build your skills through hands-on experience. It's a fantastic way to get a solid grounding in the concepts and become productive quickly.

Enjoy the book, learn lots and prosper, and go build fantastic applications on OpenShift.

—Anthony Kesterton
Principal Solution Architect
Red Hat UK Ltd.

The grumpy developer

You know those books where the author starts by listing what they have done, where they have worked, and their relevance to the subject matter? This isn't one of those. I'm what you might call a grumpy developer.

When I first got my grubby hands on a computer (back in 1981—yeah, I'm that old), I sat with the BASIC manual (it was a ZX81) and was just amazed; I could write something that made the TV change.

Looking back, people don't realize how huge that was. Now that devices are so prevalent, we often overlook their capabilities and, by extension, the work of developers. Developers make magic. We type language into a machine and it does incredible things.

We are creators, and creators need the right tools and space to work. Developers are artists. We don't work with paint, we don't work with clay, but we sculpt and we create nonetheless.

I've spent a lot of my career feeling frustrated with non-developer types who undervalue software creation as an art form. Somehow, over the last 30 years or so, we have normalized the magic of development.

So that's why I'm grumpy. Well, that and the fact that creating software has evolved from sitting in front of a keyboard with an idea to—in my opinion—an unnecessarily complex way of working.

I want to create. I don't want to spend hours debugging a POM file (damn you, Maven, and your dependency model). I don't want to lose valuable brain CPU cycles figuring out how to install a framework that, while essential to the way my software runs, adds absolutely nothing to my creations' functionality.

The 70/30 split

For a long time, I have talked about something I call the 70/30 split. When I was a full-time developer, my colleagues and I would spend, on average, 30% of our work time actually writing code that was unique and functionally important. The other 70% was wasted writing the same boilerplate chunks of code (often again and again). Now, in 2025, I can literally go to bed at night and still subconsciously build Oracle connection strings in my head.

When Kubernetes and then OpenShift came along, it instantly clicked how these tools, when used properly, made it simple to create code that did what I wanted it to. OpenShift, in particular, instantaneously removed a huge number of extraneous tasks that took up my time but added nothing to my software.

In fact, this made me less of a grumpy developer—although I will always be one. Call it scars from working on too many (fr)Agile projects when Agile was new, or spending months on waterfall projects to deliver something that was already months out of date. Grumpy is good. It's an acceptable way of being an informed cynic.

OpenShift makes it harder to be grumpy.

So, what is this book about?

Early on in my career, I loved buying technical books. Occasionally I'd even read them, and the most useful ones told me how to do things without splurging tons of unnecessary technical details. As a developer, I'd have a problem and want to get right to the solution, so those cookbooks were a godsend.

My goal with this book is to do just that: provide simple solutions to classic problems that we, as developers, face in our daily work.

The book starts off with the absolute basics (which I frequently have to remind myself of) and works up to some of the coolest and newest functionality available with OpenShift: Pipelines, GitOps, serverless, and a little bit of artificial intelligence (AI) as well.

Each subsection provides a simple overview of the technology that underpins the problems, along with a number of atomic recipes (which can be combined) that solve distinct problems.

About the code examples

These recipes were written using the no-cost Red Hat Developer Sandbox. All of the solutions you can try in the Sandbox will have detailed instructions on how to do them.

Versions used in this book

All examples in the book were tested on the Developer Sandbox against Red Hat OpenShift 4.19. In OpenShift 4.20, the Software Catalog is located in the Ecosystem navigation area. All other instructions remain the same.

User interface elements might change in future releases. For the most current instructions, refer to the official OpenShift documentation.

What's inside

This is not a book that you have to read from beginning to end, although you certainly can if you want a good primer for building modern cloud-native applications. Each chapter can be considered a standalone reference for accomplishing common tasks on OpenShift.

Chapter 0: The elegance of OpenShift and Kubernetes

> This chapter serves as your bootstrap phase. It introduces Kubernetes as a "desired state engine" that continuously ensures an object's desired state is realized and maintained. It covers the fundamental tools developers use to interact with the platform, such as the `oc` command-line utility and declarative YAML definitions.

Chapter 1: Code to container

> This chapter explains how containers, images, and pods form the foundation of OpenShift's architecture. It details how containers provide isolated, process-bound file systems, overcoming the inefficiencies of one application per physical machine or VM.

Chapter 2: Deploying your creation

> This section focuses on using deployment objects to simplify and automate application deployment. It explains how deployments act as a layer above pods, managing their creation and ensuring they match a defined specification.

Chapter 3: Changing stuff without changing the image

This chapter covers methods for modifying an application's behavior without rebuilding the image, a process known as dependency injection. It introduces ConfigMaps and Secrets as objects for injecting configuration and sensitive information, respectively, into containers as either environment variables or files.

Chapter 4: Wiring it up: The SDN

This chapter explains how OpenShift's software-defined network (SDN) handles application communication. It introduces Services as internal "doorbells" that provide a single, consistent endpoint for an application, and routes as the objects that expose these services to external network traffic.

Chapter 5: What (and how) is my application doing?

This chapter describes the three-tiered approach to observability in OpenShift: events, metrics, and logs. It explains that events track state changes in objects, metrics provide real-time resource consumption data, and logs capture output from individual containers.

Chapter 6: State of state, persisting in an ephemeral world

This chapter addresses the ephemeral nature of containers by introducing persistent volumes (PVs) and persistent volume claims (PVCs) for durable storage. It explains how these objects mount a file system into a container, allowing data to persist even when the container is recreated.

Chapter 7: Pass the tray, Jenkins

This chapter introduces Tekton, a native Kubernetes framework for building and automating CI/CD pipelines within the cluster. It defines tasks as atomic, repeatable steps and Pipelines for orchestrating these tasks, with unique `TaskRun` and `PipelineRun` objects serving as auditable records of each execution.

Chapter 8: GitOps and the state watchman, Argo CD

This chapter explores the GitOps workflow and the role of Argo CD in maintaining application state consistency. It describes Argo CD as a "watchman" that continuously monitors a Git repository for changes and automatically reconciles the cluster's live state to match it, which prevents configuration drift.

Chapter 9: Serverless with Knative

This chapter delves into Knative, a framework enabling serverless functionality on OpenShift, which allows applications to scale down to zero replicas when not in use. It differentiates between Knative Serving, which manages traffic and scaling, and Knative Eventing, which builds event-driven architectures using a standardized eventing model.

Chapter 10: A developer's guide to OpenShift security (and little robots)

This chapter provides an overview of OpenShift's robust security model, which is based on a "deny by default" philosophy and role-based access control (RBAC). It introduces Service Accounts as "little robots" that execute actions on behalf of users with specific, fine-grained permissions.

Chapter 11: Those pesky legacy VMs

This chapter discusses Red Hat OpenShift Virtualization, a feature that allows traditional VMs to run natively on OpenShift's worker nodes. It explains that each VM is managed by a dedicated pod, allowing it to be controlled and migrated using the same cloud-native principles as containers.

Chapter 12: Hello, Operator? I'd like a new object

In the final chapter, I'll explain the concept of Operators, which extend OpenShift's object model by introducing custom resources and their corresponding controllers. An Operator acts as a "custom controller," allowing it to manage the lifecycle of complex applications in a way that is consistent with native Kubernetes objects.

Acknowledgments

Before we start, a couple of heartfelt acknowledgements. Shout out to Johana Limka for technically reviewing the book (on weekends and PTO, no less); Anthony Kesterton for being utterly fun to work with; and Colleen Lobner for having the patience to put up with my edits.

CHAPTER 0

The elegance of OpenShift and Kubernetes

Kubernetes is hard. A lot of people say this, and they are absolutely right, but it's also simple. I realize this sounds a little nuts, but bear with me.

The concepts behind Kubernetes are wonderfully elegant. It is, by design, a "desired state engine," which separates the desired state from the reconciled physical state. Put simply, you ask Kubernetes to maintain an object of a given state, and it works to ensure that object's desired state is constantly realized if it needs to be, physically.

This distinction between what you want and how it is realized is the key strength of the design. When you interact with OpenShift, all you are doing is either creating objects or altering (including deleting) existing objects. Once that clicks, understanding what OpenShift is doing becomes much simpler.

It's an elegant and clever design that does have a couple of small flaws. The first, which we saw a lot of in the early days of Kubernetes and OpenShift 3, is that while the system can have an extensible object model (that is, the ability to add new currency to the system), it was often a fiddle to add it.

This is because of the core design of OpenShift and Kubernetes. As a user, you interact with OpenShift and Kubernetes by basically throwing declarative definitions—essentially, the API is a desired state engine. You throw a piece of YAML that defines what the object is (the type) and its desired state (the specification). These requests are processed by pushing an event onto an internal stack or list that is keyed by type.

In core Kubernetes, the control plane, which handles the interpretation of these requests, has a set of `while-true` loops that own and maintain individual object types by creating, updating, and deleting required state objects in the "brain" (more on this later).

These are called *controllers*, and their sole job is to listen for events of the object type they own, take the required action, and watch for "reconciliation," which refers to object state changes based on physicalization of the object.

This reconciliation is the cool part; OpenShift and Kubernetes have what are called *worker nodes*, where the system's workloads are realized. A little robot called a `kubelet` (a cute name) lives on each of these nodes. This robot responds to requests from the control plane to create components on the node and reports back to the control plane with the current state of those components.

The control plane is happy when the components' current state exactly matches the required state of the object in the "brain." This isn't a one-off process; the controller constantly monitors the state, the `kubelet` reports state changes, and the reconciliation cycle is used again if a state change is detected.

The point is that the lexicon of the object model is fixed by the objects offered by the controllers in the control plane. If I throw an object's required state into OpenShift asking for, say, a ginger cat, OpenShift will respond politely, telling me it has absolutely no clue what that is; there is no registered controller for `type:GingerCat`.

Back in the early days when Kubernetes was starting off, additional objects were added every five minutes. OpenShift 3 itself had a number of additional objects created to handle the more "enterprise-y" side of container orchestration. Every one of these new objects needed a fully written controller that had to be added directly into the control plane's codebase.

As you can imagine, this became very hard to maintain. With Kubernetes' fast cadence and OpenShift's additional objects requiring a full rebuild and retest of the entire control plane every time something new was added, it not only became more difficult to extend Kubernetes, the control plane itself became bloated.

Put it this way: your organization might want objects to represent the state of its own end products. It might not be interested in ginger cats at all. But it would have had to install a bloated control plane, with all of those 24/7 infinite-loop controllers consuming valuable system resources, just to use Kubernetes or OpenShift.

Fortunately, there is a solution: the even more elegant Operator Framework. This allows you to install custom controllers. In English, you add an application to the cluster and register it with the event engine. This application defines which objects it owns and then works as a controller. It sits in a `while-true` loop, waiting for declarative definitions of the objects it

monitors to land on the event stack. It then processes them, adds the objects to the "brain," and watches the object state for changes, either driven by the cluster itself or from declarative requests.

This approach makes OpenShift and Kubernetes infinitely extensible while still maintaining a solid, small core. You can install a basic OpenShift and then tailor exactly what functionality, through Operators, you can use.

If I only had a brain...

So far we have talked about OpenShift being a "desired state engine." But where is this state stored?

The answer is a clustered data store called etcd. It's simple, scalable, and fast, which makes it perfect as the centralized point of truth for the whole system.

OpenShift works by storing the desired state of objects in etcd, which is clustered to avoid data bottlenecks. The etcd datastore acts as the final statement of state. It's updated by state changes via the API (the declarative approach we talked about before) and state change reports from the `kubelet` on the worker nodes. The job of OpenShift as a whole is to ensure that the state definitions in etcd match the state required in the system at all times. The two-way approach of control plane updates and `kubelet` reconciliation updates provide the events to work with.

As a developer, you don't really need to care about etcd; it is always completely abstracted from the way you interact with the system. But it's worth being aware of it, along with the object model. Once this clicks, any interaction you have with OpenShift should make sense.

The developer's viewpoint

I'm a developer. I have been developing since I can remember, and I love it. Developing is an art form, a way for us tech-heads with a creative streak to create.

But as time and progress have moved on, the landscape of development has changed radically. Gone are the days that you could fire up a text editor, write a piece of code, compile it at the command line, and *et voilá,* a finished application. With an explosion of interest in development and some very smart minds out there, we developers face a plethora of new languages, frameworks, libraries, APIs, and even a complete rethink in coding philosophy.

It's easy to get mired in the cognitive overload of modern day development. This is where your tools should help you, rather than hinder you (looking at you, Maven and POM files).

One of the best side effects of how Kubernetes and OpenShift work is that OpenShift, once you understand the nature of the object model, makes sense from a developer perspective. The fact that OpenShift can act as the entire platform—from build, deployment, agile configuration, framework control, the works—means that we can focus on our code, on writing something to do what we want it to do, and not on bending frameworks, wasting brain power on understanding overly complex tooling, and all the other modern-day pitfalls of working at the cutting edge.

This book was written to fast-track developers into an OpenShift way of thinking. Its goal is not to rewrite how you create your gems, but to show you the tips, tricks, shortcuts, and vast functionality you get out of the box with an OpenShift instance—purely from the developer's viewpoint.

Introducing the tooling

OpenShift has a functionally rich web interface out of the box, which combines two distinct, opinionated viewpoints.

The first is what is called the **Administrator** viewpoint. This provides a categorized drill-down interface for interacting with the entire object model (or at least the objects your user has access to). OpenShift provides a fine-grained role-based access control (RBAC) model that can control exactly what actions can be applied to which objects by which user.

The **Administrator** viewpoint is a little badly named. It isn't designed specifically for administrators, but rather as an interface for administering the objects that you have access to. In this book's examples, we will cover the developer's use of this viewpoint.

OpenShift 4 originally had a separate **Developer** viewpoint designed to let developers interact with their workloads without needing deep knowledge of the underlying OpenShift and Kubernetes object models. As of the latest version (the one currently running as the Developer Sandbox), the core developer functionality is merged into the **Administrator** interface. This approach is simpler; while the former **Developer** viewpoint included great features like the Topology and Software Catalog, working with the system properly meant constantly switching between viewpoints.

In addition to the user interface, I will introduce and explain how to use the command-line client, helpfully called `oc` (OpenShift client). It runs at a terminal and is an expanded version of Kubernetes' `kubectl`. You can use both `kubectl` and `oc` to interact with OpenShift, but `oc` adds some key features that make it essential for the developer.

These command-line utilities offer a direct connection to the object model. They have specific commands for specific object types, but you can also use them to push the declarative definitions to the cluster to which you are logged in. The use of `oc create` and `oc apply` will form a crucial part of many tips and exercises in the book.

You can download the `oc` command-line utility and a number of other specific command-line tools directly from an installed OpenShift cluster. (Because of the nature of the Operators, some of the more complex enhancements to the system, such as Tekton, Argo CD, and Knative, have their own opinionated command-line utilities.) The "help" icon at the top right of the UX provides a link that displays downloadable versions of all the command lines for the appropriate version of the cluster.

It is good practice to download the appropriate version from the cluster itself. The command-line utilities are pretty much backward compatible, but the nature of object versioning in Kubernetes and OpenShift means that you can sometimes see odd behavior between different versions of the CLI and the cluster.

Incidentally, OpenShift has an Operator (more later) that lets you add a terminal into the web console. This is incredibly useful because the terminal runs on the cluster, eliminating the need for you to install and maintain a local copy. The Developer Sandbox includes this terminal, and we'll use it later on (it's the **>_** icon at the top of the web console if you want to use it right away).

So, enough introduction. Let's get up to speed with the basic tools you'll need to interact with an OpenShift cluster.

A box of tools

Like Kubernetes, OpenShift has a command-line utility that lets you interact directly with the object model for a cluster you're logged into. Because OpenShift is Kubernetes under the covers (with some cool object extensions and hardening), `oc` uses a kube config file. This file contains a list of user, cluster, and tokens for active (and inactive) clusters. (My kube config file has over 5,600 lines in it because I log into a lot of clusters).

But first, let's take a quick step back. Both `oc` and its Kubernetes counterpart, `kubectl`, are wrappers around the RESTful APIs that Kubernetes and the systems built upon it expose. This very comprehensive RESTful API lets you interact with objects inside the target cluster. We talked briefly about objects, controllers, and Operators in the introduction, but everything we do with OpenShift or Kubernetes involves creating, changing, or deleting objects in the control plane.

The RESTful API exposes these actions, but to be honest, it's a fiddle to use. Hence the command-line utilities, which wrap the low-level process of calling the RESTful API in a much easier way.

Old faithful: oc

If you are used to `kubectl`, then `oc` won't be a surprise to you. The main difference is that `oc` has a number of opinionated additional features and applies a syntax/content check to the objects that you are trying to update (more on that later).

To follow along with the examples in this book, you'll need, at the very least, two things: an OpenShift cluster to play on and `oc` to interact with it (without using the UX—we will extensively use the UX, but more on that later in this chapter).

If you don't have access to an OpenShift cluster, you have two easy options. I highly recommend using the Red Hat Developer Sandbox. This is a free service that gives you 30 days of access to the latest and greatest version of OpenShift. It has a couple of caveats: you can't create additional projects (you are bound to one) and it resets every 30 days. This isn't too much of a hassle, though, as you can export the entire contents of your project (as declarative YAML) and then import it when you've recreated your account.

The second option is Red Hat OpenShift Local. I don't normally recommend this, as OpenShift was specifically designed to be an enterprise, distributed system, and squeezing the whole thing onto a single virtual machine (VM) leads to some unexpected performance issues. I like people to experience OpenShift at its best, the way it was designed. But if you have a stonking huge laptop, go for it. It turns my poor little MacBook Pro into a hair dryer.

Getting `oc` is simple. If you have direct access to a cluster (which you will need), log in to the UX/console. At the top right is a question mark icon. Click that and select **Command Line Tools**. You will see a screen like the one shown in Figure 0-1. Depending on what additional functionality is installed (using Operators), you might see other CLIs for download, such as

`tkn` for Tekton/Pipelines and `kn` for Knative/serverless (we will talk about this at length later). Select the appropriate one for your OS, download, and follow the instructions.

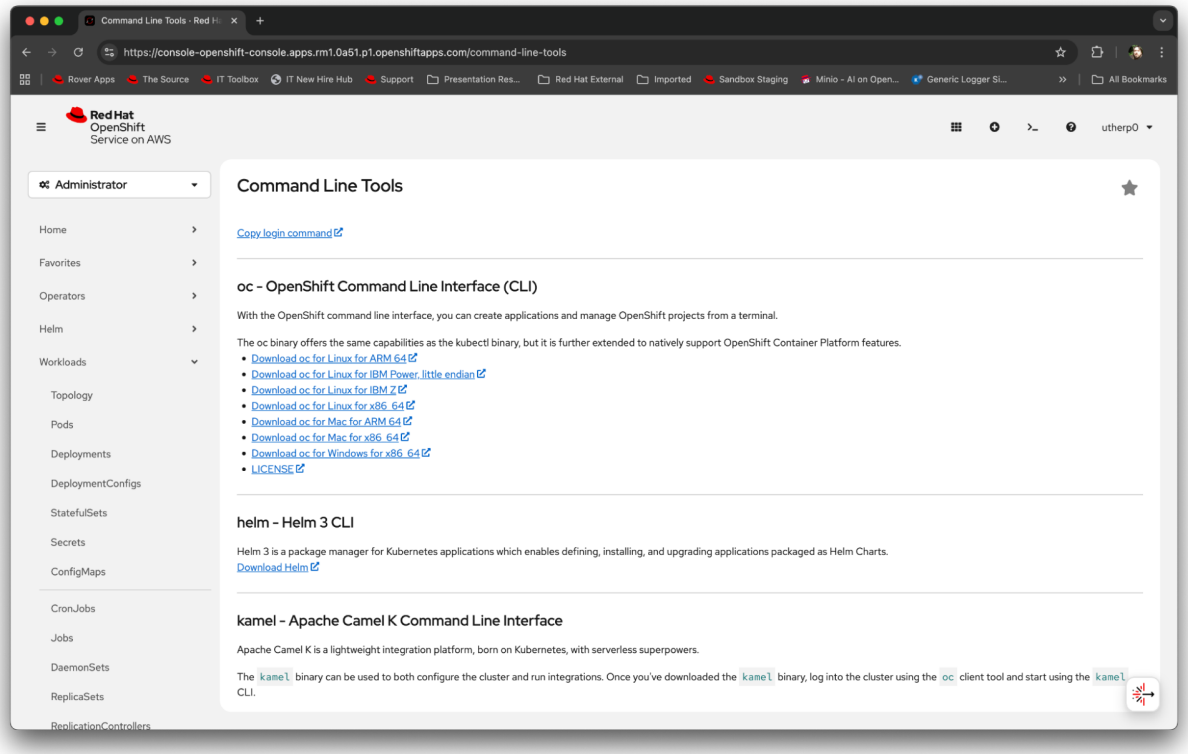


Figure 0-1: The oc download page, accessed via the question mark icon in the OpenShift user interface.

Once you have the `oc` tool installed, check that it is available by typing `oc version`.

Unless you have previously used `kubectl`, you will get the version of `oc`, the supported version of Kustomize (more later), and an error indicating you are not logged into a cluster.

Now we can start the fun stuff.

Recipe 0:1: Logging in and checking your credentials

For this recipe, you will need access to an OpenShift cluster (if you don't have one, head to Red Hat Developer Sandbox to set one up).

With `oc`, there are many ways to connect to a target cluster. If you enter `oc login --help`, you will see a help page like Figure 0-2 that lists all the login options, including interactive (just give it your username), username/password, `cert`, and `token`.

```
uther@ilawson-mac ~ % oc login --help
Log in to your server and save login for subsequent use.

 First-time users of the client should run this command to connect to a server, establish an authenticated session, and
save connection to the configuration file. The default configuration will be saved to your home directory under
".kube/config".

 The information required to login -- like username and password, a session token, or the server details -- can be
provided through flags. If not provided, the command will prompt for user input as needed.

Usage:
  oc login [URL] [flags]

Examples:
  # Log in interactively
  oc login --username=myuser

  # Log in to the given server with the given certificate authority file
  oc login localhost:8443 --certificate-authority=/path/to/cert.crt

  # Log in to the given server with the given credentials (will not prompt interactively)
  oc login localhost:8443 --username=myuser --password=mypass

Options:
  -p, --password='': Password for server
  -u, --username='': Username for server
      --certificate-authority='': Path to a cert file for the certificate authority
      --insecure-skip-tls-verify=false: If true, the server's certificate will not be checked for validity. This will
make your HTTPS connections insecure
      --token='': Bearer token for authentication to the API server

Use "oc options" for a list of global command-line options (applies to all commands).
```

Figure 0-2: oc login –help output in the terminal.

Normally, to save typing, I log in to the OpenShift Container Platform cluster console and then click the user at the top right. The drop-down has a **Copy login command** entry that gives you the tokenized version of the login command.

Copy and paste this command into your command line and execute it. The command will look something like this:

```
oc login --token=notdaftenoughtoshowone
--server=https://notdaftenoughtogiveserveraddresseither:6443
```

Notice it gives the token and a URL that differs from your console address. This URL is the control plane API endpoint.

Recipe 0:2: Who am I and where am I?

If you use a lot of clusters like I do, you occasionally forget which one you are on (and, in my case, inevitably delete things you shouldn't delete). I now make it a point to check where I am (again, often after deleting things, because, well, I'm an idiot).

A quick way to check who you are is:

```
oc whoami
```

You can also get your token for the valid login (which is very useful when playing with the RESTful API) as follows:

```
oc whoami -t
```

You can also check which cluster you are connected to, which is good when you want to make sure you haven't accidentally deleted something (again, idiot):

```
oc whoami --show-server=true
```

Recipe 0:3: Describing objects and introducing projects

So, this is where OpenShift starts to be a little different from Kubernetes. Kubernetes has the concept of *namespaces*, which are effectively a bucket where you group the objects you're working on. OpenShift is Kubernetes (I might have said this before) and supports namespaces, but it works with a super-object called a *project*.

These projects are an extended version of a namespace. You can extend objects in OpenShift and Kubernetes using labels and annotations. You use *labels* to identify object groupings, while *annotations* are specific data added to an object and not searchable or used for identification.

For projects, there are a number of immutable annotations added to the object that are used specifically by OpenShift for control and security. These include the originator, the SELinux mandatory access control indicators, and the unique identifier (UID) and globally unique identifier (GUID) ranges for containers generated within this project.

You can use the `oc` command to describe, get, and edit objects within OpenShift. The `describe` command provides a user-readable version of the object with all pertinent information.

To see which project you are currently working in (again, one of the main mistakes I normally make are not checking which project I have context to before creating things), use:

```
oc project
```

To list all the projects you have access to, use:

```
oc projects
```

Note: If you're using the Red Hat Developer Sandbox, you only have one project and it is auto-created for you. If you're using another cluster and have no projects, you can create them directly from `oc` or from the console. To create one using `oc`, use:

```
oc new-project project_name
```

The `new-project` command allows you to add a display name and a description as well, as shown in Figure 0-3. Also note that on Sandbox, if you try this, you will get an error. This is not a bug, but a feature: the Sandbox security constraints on Sandbox prohibit developer users from creating their own projects. We'll talk about this in the Security section later.

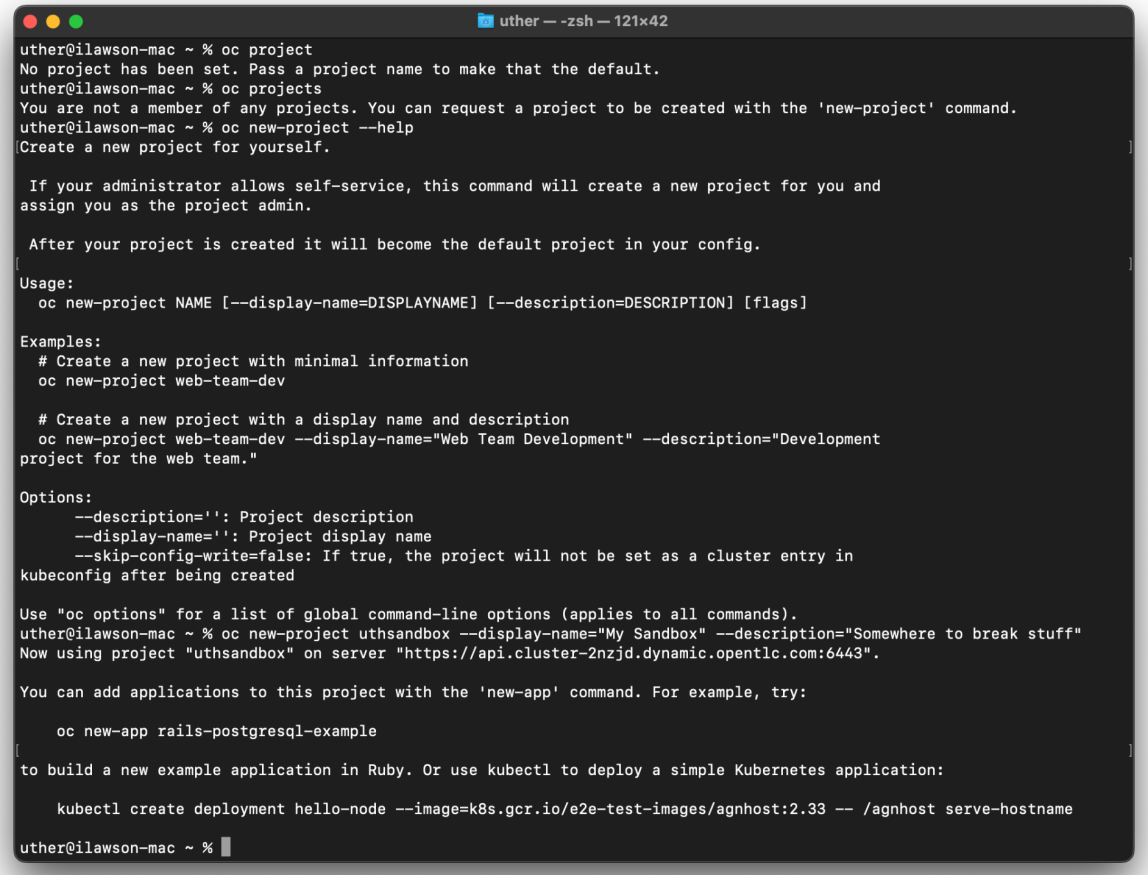


Figure 0-3: oc new-project help in the terminal.

Recipe 0:4: Create or apply?

Many of the examples in this book will use the `oc create` command to declaratively push an object definition into OpenShift. As with `kubectl`, `oc` has both `create` and `apply` commands, which take a file (via `-f`). So what's the difference?

The `create` command, like replace, is imperative. It will create (or replace) an object with the exact definition stated in the source file. The apply command lets you modify an object. For example, if you apply the same object definition twice, the second `apply` returns "unchanged."

You can use `oc create` with the `save-config` option, which allows `apply` to work from that configuration. Also, the `dry-run` option executes the create command without sending the object to the cluster, which sounds mad, but it allows you to check the syntax of your object definition without bothering the cluster.

To create an object using a predefined YAML file, use:

```
oc create -f (targetFile)
```

To apply an object using a predefined YAML file, use:

```
oc apply -f (targetFile)
```

If you want to do a dry run, use:

```
oc create -f (targetFile) --dry-run=client
```

You should see output similar to the following:

```
uther@ilawson-mac app % oc create -f service.yaml --dry-run=client
service/devexterminal created (dry run)
uther@ilawson-mac app % oc get svc
No resources found in uthsandbox namespace.
```

Recipe 0:5: Editing an object, or fixing "someone else's" (cough) mistakes

The `oc` command line also allows you to edit an existing object (assuming you have access to it—more on that later).

The functionality offered by `oc edit` is quite comprehensive. You can edit the object with the editor of your choice (by setting the `KUBE_EDITOR` environment variable to an editor executable), although on Linux and macOS, it defaults to `vi`.

It's worth explaining the object name syntax at this point as well. When using the `oc` commands, you refer to an object using its type as well as its name. For example, `oc edit svc/myservice` will edit the service object `myservice` in your current project.

You can override the project/namespace using `oc edit svc/myservice -n myotherproject`, where `-n` allows you to target another namespace/project. (Note that even though projects are different from namespaces, the referencing seems interchangeable, with `-n` referring to a project in this case. This is because `oc` is an extension of `kubectl`).

You can also use the fully qualified, version-inclusive naming convention for object type, such as `job.v1.batch/job`.

The editor is useful in that it will not allow you to save (and therefore commit) objects with invalid syntax. This also includes incorrect values as stated in the API definition, as well as incorrect field names.

Also note that the OpenShift console—both the Administrator and Developer viewpoints—has a built-in YAML editor that performs the same function as `oc edit`, albeit in a prettier way (no offense, `vi`).

To show this in action, we are going to create a service (more on these later) and then edit it. The source is available for this example in this book's GitHub repository.

```
apiVersion: v1
kind: Service
metadata:
  labels:
    app: devexterminal
  name: devexterminal
spec:
  ports:
  - name: 8080-tcp
    port: 8080
    protocol: TCP
    targetPort: 8084
  selector:
    app: devexterminal
  sessionAffinity: None
  type: ClusterIP
```

Either download this file or create one of your own. Then, assuming you are logged in to a cluster, use:

```
oc create -f (filename)
```

Check that the file has been created:

```
oc get svc
```

The service is called `devexterminal` and should be listed as a service. Now enter:

```
oc edit svc/devexterminal
```

If you haven't changed the default editor, you should see something like Figure 0-4.

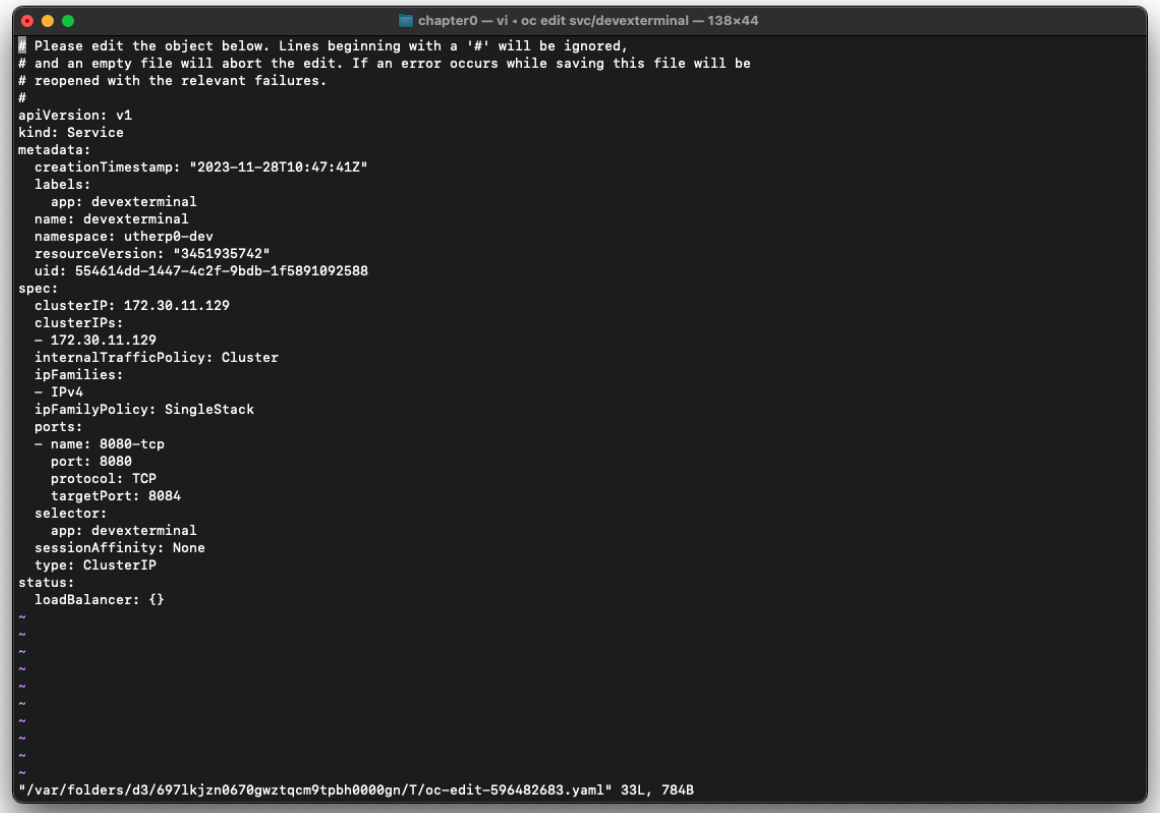


Figure 0-4: Example vi in the terminal.

So, this is a local copy of the definition on your machine. It reflects what the cluster has, hence the additional information within it. What you are doing is editing the state of the object you just created.

What we are going to do now is something daft: try changing `kind: Service` to `kind: NotAService`, and then save (if you are using the default editor, `vi`, it is `[Esc] :wq`).

The editor will reappear with the following message embedded as a comment:

```
# The edited file had a syntax error: resource mapping not found for name:
"devexterminal" namespace: "utherp0-dev" from "edited-file": no matches for kind
"NotAService" in version "v1"
```

Cancel out of the edit (following keystrokes in wonderful vi).

```
[Esc] :q!
```

Now start the edit again (using the preceding command). This time, we will edit something sensible: change the target port from `8084` to `8080` (look for the `targetPort:` variable). Save it (`[Esc] wq!`).

This time, the edit command will complete. You have changed the target port of the `Service` object to `8080`. You can check this by typing:

```
oc get svc/devexterminal -o yaml
```

This is a very powerful command. Effectively, you can (with correct syntax and appropriate RBAC) edit any object you have access to.

Recipe 0:6: Fun with JSONPath

So, `oc` has an extension that allows you to manipulate the returned objects using the `JSONPath` syntax. This is incredibly powerful. A lot of the automation I use around `oc` is via this extension.

In essence, what it allows you to do is pick apart the returned values from an `oc` command using the JSON format. If you know the format of the object, you can extract individual components and then use them in other commands.

I won't go into a full overview of JSONPath. The Kubernetes documentation does a great job of explaining the syntax. But I will share a little script I used for finding `BuildConfig` and `DeploymentConfig` objects (both types of objects developers will be interested in, as we'll discuss later) that used a defined image name:

```
# Parameter check
if [ -z "$1" ]; then
  echo "Usage: ./(script) TARGET_IMAGE_SUBSTRING";
  exit 0
fi
# Setup counts
bcCount=0;
dcCount=0;
```

```
for project in $(oc get projects -o jsonpath='{.items[*].metadata.name}');
do
  echo "PROCESSING BC for "$project;
  for bc in $(oc get bc -n $project -o jsonpath='{.items[*].metadata.name}');
  do
    imageName=$(oc get bc $bc -n $project -o
jsonpath='{$.spec.strategy..from.name}');
    #echo "  "$bc" "$imageName;
    # Check the bc for target image name
    if grep -q $1 <<<$imageName; then
      dc_count=$(oc get dc $bc -n $project -o jsonpath='{.status.replicas}');
      echo "[Found] "$dc_count" replicas of "$imageName;
      bcCount=$(($bcCount+$dc_count));
    fi
  done
  echo "PROCESSING DC for "$project;
  for dc in $(oc get dc -n $project -o jsonpath='{.items[*].metadata.name}');
  do
    dcImageName=$(oc get dc $dc -n $project -o
jsonpath='{.spec.template.spec..image}');
    #echo "DEBUG "$dcImageName;
    if grep -q $1 <<<$dcImageName; then
      dc_dc_count=$(oc get dc $dc -n $project -o jsonpath='{.status.replicas}');
      echo "[Found] Direct DC deployment replicas "$dc_dc_count" of
"$dcImageName;
      dcCount=$((dcCount+$dc_dc_count));
    fi
  done
done
echo "Total of Pods via BC match: "$bcCount
echo "Total of Pods via direct DC match: "$dcCount
```

It's a reasonably complex little script, but the important bits are where it references the JSON model directly using JSONPath. If you input the following, you can reverse-engineer the target JSON components:

```
oc get (objectType)/(objectName) -o json
```

For instance, to get the name of an object, which is part of the metadata component, you use:

```
-o jsonpath='{.metadata.name}'
```

Again, this is incredibly powerful, and I use it all over the place to automate object interactions via oc.

Recipe 0:7: Doing the CrashLoopBackOff shuffle

Jumping ahead slightly, we will cover containers and pods in the next chapter. But it's worth covering a nice little command within `oc` that can be a godsend when you find yourself doing the `CrashLoopBackOff` shuffle.

The `CrashLoopBackOff` shuffle is a side effect of the brilliant desired-state engine that is Kubernetes and OpenShift. You ask the cluster to deploy an application, but there's something wrong with it that causes it to crash when started. (I know, we developers never write bad code, so it's probably something config-related, *cough*).

So the cluster deploys the application, which immediately crashes. The nature of the desired state system is that the cluster says, "Ooh, the application has failed," removes the offending pod (more in the next chapter), and instantly recreates it to be reconciled with the desired state. But the application hasn't changed, and it will instantly crash again.

The problem with this—and again, we'll talk about observability later—is that when a pod is removed, the logs are lost. So it's very hard to debug what is going on, as the application is appearing and disappearing continuously (hence the `CrashLoopBackOff`).

There's a nice little `oc` feature that allows you to stage the application *without executing the final statement that starts it*. This means that the pod is created, the file system is set up, and by using `oc debug`, you can shell into the pod and check the files.

The syntax of this command is:

```
oc debug (resource)/(name)
```

The nice thing about this is that it is not limited to just pods. You can debug a node itself (if you are an administrator) or create a resource using an image stream (more on that later).

This is actually one of the most useful features, at least to me, because I have a tendency to, well, break things. Once you have an application running (which we'll cover in recipe 0:8), remember this command: it's a lifesaver.

Alias docker=podman

For some of the recipes and examples in this book, we will need to build images. In the next chapter, I'll cover images, containers, and pods in detail—the basic currency used in OpenShift and Kubernetes—and explain how to create, use, and move them. But before we get there, I'm going to provide a short introduction to Podman.

Podman (which stands for *pod manager*, a fact I went blissfully not knowing for a year) is a utility that allows you to build images, run containers, and construct and run pods. The next chapter will explain these in detail and how to get started with them as a developer, but in the meantime, it is worth downloading and installing the Podman tools.

Podman is available on Linux (natively) and also on macOS and Windows boxes. On Mac and Windows, the system executes a Fedora-based VM that does all the image stuff. The commands you issue from Podman are forwarded into the VM. This is incredibly useful. In the old days, I used to have to run my own VM on my Mac box and SSH in to do my builds.

You can get the `podman` command-line interface (CLI) from podman.io. There's also a brilliant UX tool that sits on top called Podman Desktop, which offers all the functionality of Podman through wizards and visual components.

If you have Docker or Docker Desktop, you can use these, but Podman gives you the ability to create and test pods, including generating the Kubernetes and OpenShift YAML for importing and executing on the clusters. In fact, Podman Desktop has plug-ins for both Kubernetes and OpenShift, which make it even easier.

Recipe 0:8: An example application

Before we get going, I'm going to introduce and show you how to create a quick application. We'll be using this one for many of the recipes to show various features of OpenShift, so rather than repeat the instructions throughout the book, this recipe will walk you through creating it. Don't worry about understanding the ins and outs; we will dive into more details in later chapters.

You will need a Developer Sandbox account for this one. The Red Hat Developer Sandbox is a completely no-cost facility that gives you access to the latest OpenShift release as a hosted service. It offers 30 days of access, a single project, and constrained resources, but it is perfect for playing and teaching yourself the developer-friendly components of the system.

Once you log in to the OpenShift system, you will have access to the user interface. This will default to the **Administrator** viewpoint; the other one is **Virtualization**, which we will talk about later.

In the latest web console, the developer tooling—specifically the **Topology** and the **Software Catalog**—has been merged into the **Administrator** viewpoint. Select **Home** in the left-hand navigation panel, and then choose **Software Catalog**. The screen will look something like Figure 0-5.

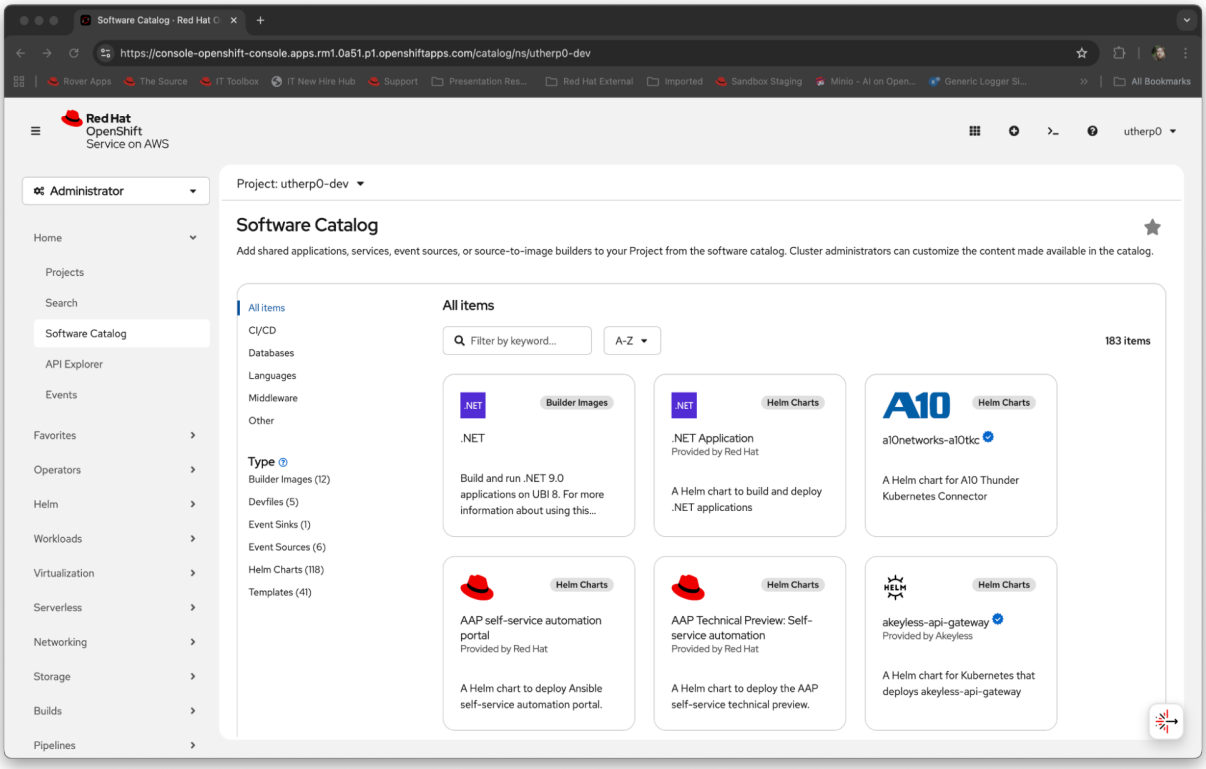


Figure 0-5: Example Add dialog from the OpenShift user interface.

In the catalog, in the **Filter by keyword** box, type `node.js`. You will see many tiles offering different components you can add. Select the one with the label **Builder Images** and then click **Create**. Don't worry about the components it describes just yet; we will cover all of them in the forthcoming chapters.

In the **Create Source-to-Image application** panel, leave all the defaults except for the following:

1. In the **Git repo URL** text field, enter: `https://github.com/utherp0/ocpnode`
2. In the **Application Name** field, enter: `ocpnode-app`
3. Make sure the **Name** field is automatically set to `ocpnode`.
4. Make sure the **Build Option** is automatically set to **BuildConfig**.
5. Change the **Resource type** in Deploy to **Deployment**.

Now click **Create**. Wait for the application build. Eventually, the roundel (the circle around the application name and icon) will switch to blue (light blue means the build is in progress, dark

blue means it is shutting down). Once the roundel changes to blue, you have an active application.

If you click the roundel, a panel will appear on the right showing the state of the various components that make up this application. If you click the icon at the top right of the roundel, the application's webpage will be served in a new tab.

At this point, you have an example application. If you delete it at any point and then want to recreate it, be aware that the image stream (more on that in Chapter 1) remains if you delete the deployment. That object retains a project-wide history of the image and isn't removed by default when you delete the deployment.

However, the **Topology** page uses a cosmetic grouping called an application. (I know, massive overloading of terms, my apologies.) If you remove the application by clicking the application label and choosing **Delete application** from the drop-down on the right (shown in Figure 0-6), OpenShift will remove all the associated objects correctly. If, like me, you are bothered by cluttered projects, try deleting it now using this method.

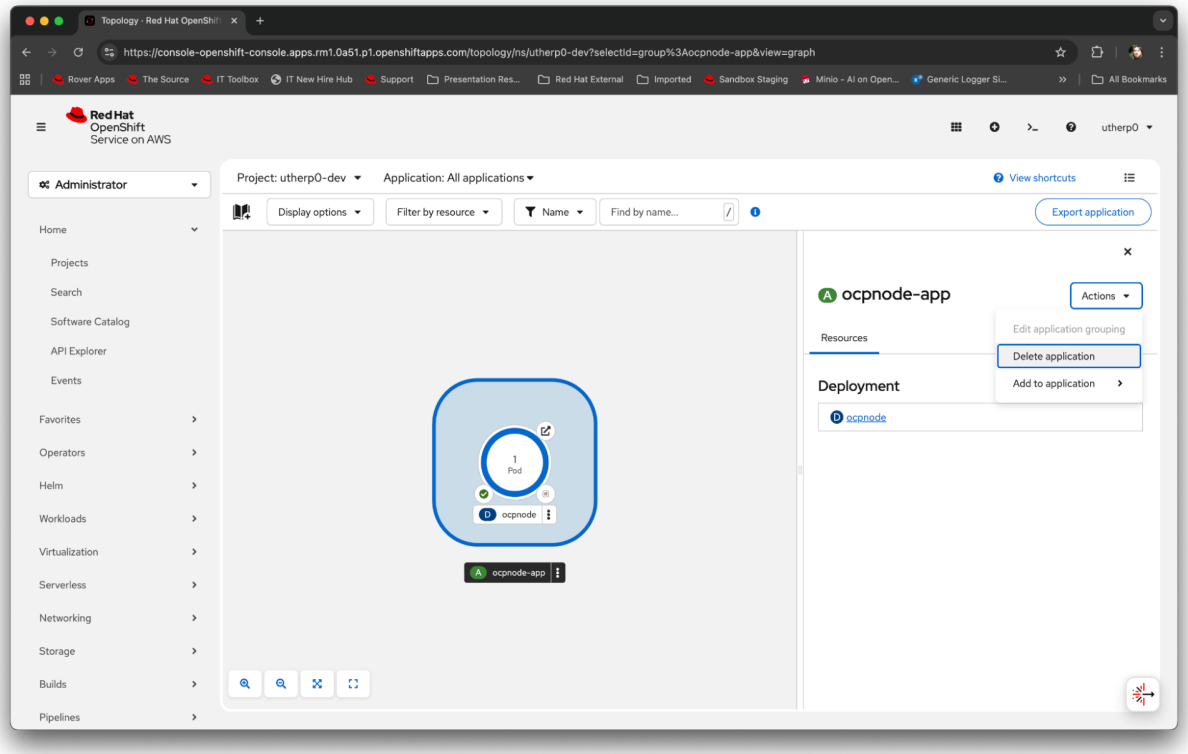


Figure 0-6: Application dialog box in the OpenShift user interface.

What's next?

So, you now have the tools to start playing with OpenShift. As a longtime developer on the platform, I find myself using a combination of the tools (specifically `oc` and `jsonpath`) with the user interface, basically because I like dynamic webpages that give me a lot of information.

In Part 1, we'll cover the basics every developer needs to know to get started with the platform. In Part 2, we'll dive into key infrastructure components to be aware of. Then, if you can wait that long, Part 3 talks about the real fun stuff.

PART 1

Fun-damentals

CHAPTER 1

Code to container

By now, you hopefully understand the nature of objects in OpenShift and how you can interact with them as a developer. That's all well and good, but it's just the basic structure that underpins everything.

So, what's a container?

I say this in all my pitches: *containers are file systems with delusions of grandeur*. They are process-bound file systems on a shared kernel that think they are standalone operating systems. To understand containers, images, and pods, we need to step back in time for a moment and talk about the great Dev versus Ops wars of the 1990s.

See, back then, when a developer wrote an application, they had a tendency to take over the machine on which it was executed. Here's a good example: I worked on some huge Java applications back in the late 1990s, and the machine on which they ran was completely locked into my application's needs. It had to have my JDK, my system libraries, and my versions of Tomcat and Lucene installed.

This was a complete anathema to any other applications on that box, especially other Java applications that needed different versions of the JDK and system libraries. So my application always seemed to "taint" the boxes on which they were deployed, and this was before the advent of virtual machines. We had a situation of one application per physical machine, which was terribly inefficient.

From VMs to containers

Then came the time of the virtual machine, which was a great technology. Now you could emulate many machines on one piece of physical hardware. But a virtual machine contained an entire OS and my application, which tainted it, so I needed one VM per application. This

was more efficient than tainting an entire machine, but it was still an Ops nightmare to maintain and provision, partly because they needed an entire "gold image" for all the things my application needed to run.

Then along came containers. Put simply, containers are stacks of file differences applied in layers, with an overlay writable file system that contains process, network, and file bindings. Unlike a virtual machine, the container runs as a process on the kernel of the operating system but is insulated from it using its own `glibc`. In English, this means the container can effectively pretend to be another version of Linux.

Also, the layers that I mentioned are shared between containers, which means if you are running, say, 10 containers that are Red Hat Enterprise Linux-based and don't change the underlying files, the shared layers that don't change are referenced by symbolic links. This is a very efficient way of running multiple containers.

Going back to my original example, all the "taint" I need on a box to run my application now travels as part of the container. The underlying OS doesn't need to have, say, a singular version of the JDK installed, as I can install the JDK within my container, and it has no effect on the OS or other containers. The clue is in the name: containing.

Understanding container images

Let's talk terminology before we start having fun with the containers themselves.

A *container* is created on a *container runtime host* using an *image* as a template. The thing is, images don't actually exist (bear with me here). In reality, an image is a map of layers at a given temporal time.

A *registry* is a store of images, but in reality, a registry is a collection of file layers and a mapping of an image (identified by a tag) that defines the set of layers used to create the container.

This is very efficient. When storing the layers, they exist singularly. If an image shares layers with another image, rather than having two distinct sets of layers, the image is a map to a single shared copy of the layer.

You can test this easily by using Podman to build an image with a name and then repeating the build with a different name (I'll give a quick overview later in this chapter). You will get separate "images," but if you look at the unique ID that Podman creates for the images, they will have the same ID because they are the same set of identical layers.

As a dev, I like this a lot, it feels elegant. Also, as a dev, you don't have to care about this, but it's always good to know what is going on under the covers.

A couple more things of interest: the container runtime host in OpenShift is, by default, Red Hat Enterprise Linux CoreOS. This is a specialized spin of Red Hat Enterprise Linux (RHEL). In fact, it's a massively cut-down version of RHEL designed only for running containers, which gives it a much smaller footprint than "full-fat" RHEL.

It is also designed to be an "immutable" OS, although as of OpenShift 4.13, there is a version of Red Hat Enterprise Linux CoreOS that allows the addition of file layers for user workloads, but this isn't the way it should be used.

The advantages of having an OS designed only for containers are many, but the best ones include a much smaller attack vector for incursions (because the OS is immutable, you can't add functionality to it, and the only functionality it has is container-specific) and a smaller footprint. When you are setting up worker nodes (more on those in a second), you want to get the most out of your resources. A low-profile, small footprint OS is perfect for giving you more room to run your actual workloads.

A developer's perspective on containers

I fell foul of the first error you make when you start to play with containers. I followed all the instructions and built an image based around Tomcat, with my little Jakarta Server Pages (JSP) application copied into the application serving directory. However, I forgot to add security changes. You needed to add a file in the Tomcat root with user information, or at least you used to (this is going back a while). So when I created my container, it would run but wouldn't serve the pages.

So, like all newbies, I shelled into the container (which you can do because containers think they are active operating systems), found the flaw, edited the file, and added my user details, and *et voilá*, the container worked. But fast forward a day, and I'm demoing it to a customer, including creating the container from scratch. And bang, it failed to serve the pages.

See, I'd done what most developers do: I solved the problem the way I knew how to, but I forgot that the container's file state does not match the original image's file state. This means any changes I make to the file system within the container are not reflected back on the original image. The image is immutable once created and can never change (unless you rebuild it). So, when I shut down my container and then recreated it for the demo, I recreated it from the image, not from the container file system that I had changed.

Once this clicked, I never made that mistake again. Later, we'll talk about "state of state" and how you can use OpenShift to retain and store persisted file systems between create, destroy, and recreate actions. But for now, just remember that a container file system is ephemeral: if the container is recreated, the file system state comes from the image.

The main question I always get asked is: What should I, as a developer, put in a container? To be honest, the only real restriction is your imagination (although petabyte-sized data stores would be difficult). The real advantage of the container revolution is that it inherently lets you do what I call *atomic decomposition* of your applications in a way that standalone applications couldn't really do before.

Now you can write a complex application using many individually hosted (and kept-healthy) containers. This allows you to simplify the applications, as the individual components or microservices have singular tasks. It also lets you fix or enhance the entire application by updating individual components. In the old days, you'd have to rebuild the whole thing if you wanted to change a small part of it.

Also, you can reuse microservices in different complex applications, negating the old problem of having the same code components in multiple applications. I have lost count of how many times I've worked on an application, updated or enhanced one of its components, only to find another application doing the same thing the old way. By containerizing the complex application components and using microservices, you can change or update one and all the complex applications that consume it.

There are many other advantages to using OpenShift as the orchestrator, and we will talk about this in depth in Chapter 4. For a quick overview, all containers exist in pods, and all pods live on a single (by default) software-defined network within OpenShift. Traffic ingresses through an OpenShift/Kubernetes service object, which effectively acts as a doorbell into the application. You can reference this doorbell by name within a project (that is, instead of a fully qualified domain name, or FQDN, you can simply connect to `myservice:8080`), but it is also expressed as two distinct FQDNs that are available to components (depending on RBAC).

This means you get free service discovery. For instance, if I had a microservice called `myservice` in my project `myproject`, I can reference `service:8080` from within that project or `myservice-myproject.svc.cluster.local` from outside the project but within the SDN. These references are completely internal, meaning you can stand up a complex application with 10 microservices that are completely internalized (no external access at all) and one service that is externally facing. It's a nice feature.

All about pods

Don't worry, we're getting close to the point of actually doing something now. Before we start working with OpenShift, we need to understand the concept of the pod.

Put simply, the pod is the smallest atomic deployable component in Kubernetes/OpenShift. The job of Kubernetes/OpenShift is to orchestrate pods, along with a number of other objects that define behavior, connectivity, and the like. But at its root, it is all about the currency of pods.

But what is a pod?

Think of a pod as an isolated wrapper around one or more containers. Interestingly, as Kubernetes (and OpenShift) have developed, people have tended to deploy a single container per pod, and this has become a standard. But we are missing some cool stuff by doing that. A pod has its own network namespace, so if you run two containers as part of a single pod, both containers can refer to each other as `localhost`.

While this doesn't sound exciting (I think it is), it provides a nice way of, for example, co-locating session storage with the application (say, one container has a dynamic web content engine, another has an in-memory data grid). OpenShift guarantees that containers in the same pod are (a) orchestrated to the same worker node together and (b) share the same network ID (that is, `localhost` resolves to both).

Confession time: Before I wrote this book, even after many years of playing with OpenShift, I had never created a pod. I expect you to be going "what on earth" (or something ruder), but using OpenShift (and Kubernetes) means I don't need to create just a pod. I have used the `Deployment` and now obsolete OpenShift `DeploymentConfig` objects that wrap the creation of pods for you. As a developer, you should never need to create just a pod.

That said, Podman is a wonderful way to get started with pods, and it will hand-hold you through creating and populating pods. But when you are on OpenShift, you don't need to be that "atomic." For the sake of fun, we will create a pod to see what it looks like.

Recipe 1:1: Your first pod

To do this, you need to log in to a cluster. If you haven't done it already, go run Recipe 0:1. If you are on the Developer Sandbox, you have the option of using the built-in terminal (click the command-line terminal icon at the top of the user interface). This is fine for these kinds of

examples, but be aware that copy and paste from your system clipboard can be fiddly on some operating systems (looking at you, Windows).

Create a file containing the following YAML (call it `pod1.yaml`):

```
apiVersion: v1
kind: Pod
metadata:
  name: myfirstpod
spec:
  containers:
  - name: ocpnode
    image: quay.io/ilawson/ocpnode
    ports:
    - containerPort: 80
```

What we are doing is using the standard declarative approach for creating an object. In this case, we are sending a chunk of YAML to the cluster control plane saying that we want a pod, please, with the name `myfirstpod` and a specification that it contains a container with the name `ocpnode`, created from the hosted image at `quay.io/ilawson/ocpnode`, with a single port (80) exposed from the container. We won't be using the port (we will look at networking in a later chapter), but it's worth having a minimal specification handy.

When you have done this, use `oc` in the same directory you created the file:

```
oc create -f pod1.yaml
```

If all goes well, you should see something like Figure 1-1 in the Topology.

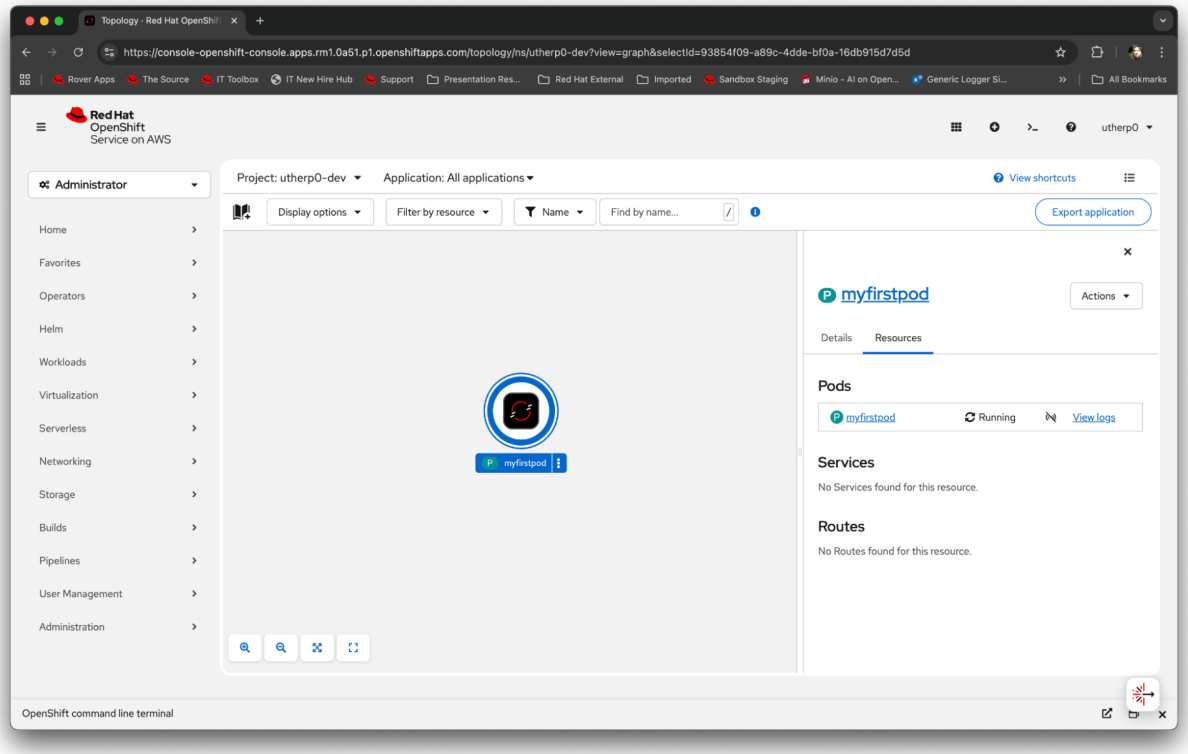


Figure 1-1: Topology page showing a pod running.

The lonely little pod is running by itself (note it has no attached network objects, hence the "No services" and "No routes").

You can actually check if the container is running: click the pod name in the list on the right-hand content panel, and when the pod overview appears, click the **Terminal** tab.

When the terminal appears with a prompt, enter:

```
curl http://localhost:8080
```

You will see the HTML output from the application running in the container.

Recipe 1:2: Checking the status of your pod

Having done Recipe 1:1, you will have an active pod in your project (for more info on projects, check out Recipe 0:3). We're now going to have a look at the running pod using the command line.

If you are using the OpenShift user interface (which is highly advisable, as it's such a good UX for interacting with the object model), this is easy; just use **Workloads/Pods** on the

Administrator viewpoint. But it's also nice to have a down-and-dirty way of looking at the pods.

Make sure you are logged in to the cluster appropriately (hint: Recipe 0:2), check that you are in the correct project (if you are on the Developer Sandbox, it will be `username-dev`), and then enter:

```
oc get pods | grep first
```

You should see output similar to the following:

```
[lawson@ilawson1-mac ocpbook % oc get pods
NAME         READY   STATUS    RESTARTS   AGE
myfirstpod   1/1     Running   0          19m
```

The `READY` column lists the containers running in the pod (number active/number required). The other columns (`NAME`, `STATUS`, `RESTARTS`, `AGE`) should be self-explanatory.

Now for a bit more information. We are going to use the `oc describe` command, which gives all the information for an object. We need to provide the object type and the name, so enter this:

```
oc describe pod/myfirstpod
```

You will get a stream of information about the pod. The information is provided in a user-friendly way and is consistent for all objects it returns (the style, not the content). You can now expand on the information specific to the pod using:

```
oc get pod/myfirstpod -o yaml
```

This will give you the full definition of the pod using the declarative YAML (you can also choose JSON as an output format). Now for a quick test, enter this:

```
oc get pod/myfirstpod -o yaml > myfirstpod.yaml
```

Note this assumes you are on a Linux box. If you aren't, use the Developer Sandbox and choose the terminal prompt icon to start a web-based terminal, or if you have your own cluster, install the Terminal Operator (or ask the platform engineers very nicely to do it for you).

Have a look at the file generated and compare it to the one you provided to create the pod. You will see the object has a large amount of cluster-specific content in it. This was

autogenerated by the cluster and represents the object created to resolve your desired state (which is a small piece of YAML).

Swimming in the OpenShift image streams

Image streams are confusing. There, I said it. Even as someone who has spent a lot of time elbow deep in OpenShift, I've always had a bit of an issue understanding them. But once they click, they make complete sense.

To put it simply, containers are instantiated from images. Images are static (and immutable). Images can be originally sourced from external registries, built on a cluster, and served from the integrated registry within OpenShift.

Think of image streams as an abstracted metadata object around the sourcing of images; Images exist at the cluster level, and then usage of the images within a project is via an image stream.

So what can you do with image streams themselves, given they provide abstracted information about the actual images?

Let's try a little test. We won't call this a recipe as it will fail. For the sake of this test, we will use the Developer Sandbox. Log in using Recipe 0:1 and set yourself in a valid project (the project name will be `username-dev`) using `oc project`. Now in `oc`, enter:

```
oc get images
```

Unless you're a Red Hat Developer Sandbox admin (and if so, what are you doing reading this, just for the humor?), you will get bounced with a `User cannot list resource "images" in API group "image.openshift.io" at the cluster scope` error:

```
ilawson@ilawson1-mac ocpbook % oc get images
Error from server (Forbidden): images.image.openshift.io is forbidden: User
"utherp0" cannot list resource "images" in API group "image.openshift.io" at the
cluster scope
```

Which seems odd. How can you create containers if you can't access the images?

This is where image streams come in. They provide a project-based handle for controlling access to the images themselves. This basically stops people from accidentally deleting images from the cluster. (Remember, there is a single integrated, shared registry for the cluster.)

In the words of the official docs, "An image stream and its associated tags provide an abstraction for referencing container images from within OpenShift Container Platform." Essentially, the OpenShift cluster talks image streams when doing deployments or builds.

The image stream contains the name and tags of the relevant image. In fact, when you start to look at them, the image streams contain the image ID within the registry rather than the abstracted name/tag. This allows the image stream to detect if the image has changed. Even with the same name and tag, the image will have a different unique ID (UID) when rebuilt. The registry maintains the `image name:tag` to `image ID` mapping, and the image stream contains the name, tag, and the ID it is currently pointing at.

Interestingly, this allows for rollback if something goes wrong. In addition, the tagging schema on image streams allows for two mechanisms:

- **Permanent tagging:** Used to point to a permanent version of an image, say a release of node or the like.
- **Tracking tags:** Work like symlinks and point at *another* image stream.

The tags in an `ImageStream` that can contain historical information, which can be used to roll back an image to a previous *generation* of the tag. This looks like the following:

```
kind: ImageStream
apiVersion: image.openshift.io/v1
metadata:
  name: my-image-stream
# ...
  tags:
  - items:
    - created: 2017-09-02T10:15:09Z
      dockerImageReference: 172.30.56.218:5000/test/origin-ruby-
sample@sha256:47463d94eb5c049b2d23b03a9530bf944f8f967a0fe79147dd6b9135bf7dd13d
      generation: 2
      image:
sha256:909de62d1f609a717ec433cc25ca5cf00941545c83a01fb31527771e1fab3fc5
    - created: 2017-09-01T13:40:11Z
      dockerImageReference: 172.30.56.218:5000/test/origin-ruby-
sample@sha256:909de62d1f609a717ec433cc25ca5cf00941545c83a01fb31527771e1fab3fc5
      generation: 1
      image:
sha256:47463d94eb5c049b2d23b03a9530bf944f8f967a0fe79147dd6b9135bf7dd13d
    tag: latest
# ...
```

Note the `items` array in the `tags` component. There are two generations of the image stream reference.

As I said, they are a bit complex. Just be aware of them. The majority of problems you will see when deploying an image will probably be due to the image stream tagging.

Builds

Now for the fun part. OpenShift has always been a joy to use if you're a coder like me. Having spent too many years messing up builds, the idea of having one where I just rack up with my code and let it get on with it is just bliss.

Granted, it's not that simple, but it is much easier than the old way. And it makes complete sense from a container platform perspective. In the old days with Kubernetes and Docker, you'd build your image (externally, in the case of Kubernetes) and then orchestrate your creation with either Docker (as a standalone container) or Kubernetes (as a pod on a Kubernetes cluster). Kubernetes was all about orchestration.

The evolution of OpenShift builds

The original Kubernetes release of OpenShift, good old 3.x, introduced a technology that was both a lifesaver and a pain for me when I was talking about it: Source-to-Image.

As a technology, it was brilliant. But talking to potential users and customers about injecting "STIs" onto their clusters was, well, problematic at best. I actually raised a RFE (Request for Enhancement) pleading with them to change it, and just like that, it became S2I. But I digress.

The S2I build process

The concept of S2I was that you could provide OpenShift with a code repository (that is, a Git repo) and a base image (which contained the compilation and runtime framework for your application). OpenShift would then execute what was called a Source-to-Image build process.

In actuality, it was three scripts: `Compile`, `Assemble`, and `Run`.

- The `Compile` script takes the source code and compiles it to produce the runtime (for example, binaries or class files in Java).
- The `Assemble` script combines these binary artifacts into the correct positions within the framework image, creating a new composite image.
- The `Run` script is then used to start the application within the image (for example, `java -jar "theexecutablejarlocation"`).

That was great, but the way in which it executed it on the cluster used the original Docker processes. This means a user would kick off a S2I build (which wasn't done in the user space as it needed raised privileges), which would run a Docker build process as root. Even with OpenShift's brilliant security (SELinux for the win), this wasn't ideal.

A new container runtime

So, as part of the OpenShift 4 design, a new container runtime was created: CRI-O. All of the components within Kubernetes use an abstracted API to talk to the underlying components. Networking uses CNI (Container Network Interface), storage uses CSI (Container Storage Interface), and CRI-O is the Container Runtime Interface (Open Container Initiative).

Basically, it's a Kubernetes-focused replacement for Docker and its tools. One of the tools it provides is Buildah, which is a userspace-bound container construction tool. It performs image builds using Containerfiles (Docker uses Dockerfiles), which produce an OCI-standardized image. This image, because it adheres to the OCI standards, can be executed by any runtime that adheres to the same OCI standards.

The good thing is that Buildah can execute in a constrained namespace. This means it doesn't need to run as root. Without going off on a rant, running as root is not good.

Also, in OpenShift 4, before the new Buildv2 days, the build processes were all there but scattered in different places. There was a `BuildConfig` object that described exactly how to build an image (that is, which S2I framework image to use, which image stream to use, etc.). You could use the S2I technology to build from a Dockerfile or Containerfile as well. In fact, the language-based S2I does just that: it executes the compilation and then uses a Containerfile to construct the target image.

Introducing Shipwright

However, in the years since that was invented, the industry moved on a little bit. Rather than extending the S2I concept (which people liked but didn't really adopt outside of OpenShift), an open source project kicked off to rationalize the creation of images to a single custom resource: a `Build`.

This project was (and is) called Shipwright, and it provides a simple way to declaratively create an image without having to understand the underlying tools. It uses a basic object type called `Build`, which handles all the mechanisms on the cluster side to build the image. It supports multiple build types, such as Kaniko, Buildah, S2I, BuildKit, and Cloud Native Buildpacks, and is

extensible. This means if you have or use an esoteric mechanism for building images, you can wire it into the Shipwright way of doing things.

OpenShift now supports this technology natively, using the builds for Red Hat OpenShift framework.

How S2I simplifies development

Before we try out some of this `Build`-goodness, I want to go over the concepts of Source-to-Image. For me, this is one of the best things about OpenShift. It was pretty much the first Kubernetes distribution to try and address the ability of a developer with no Kubernetes experience to natively take their source code (which is their application with their expertise in it) and produce an image that can be used to execute the application using Kubernetes.

It was a manual process at first, and some people still use that approach. Back then, it was a Dockerfile, and using Docker, you would assemble an image using the components needed for the application. In the simplest way, you would build your application framework-specific (for example, you would use Maven to compile and package your Java JARs), and then use Docker with a Dockerfile to add the binary components to the framework (that is, copying the EARs to a JBoss deployment directory).

The job of the image builder was to assemble—you could compile and build as part of the Dockerfile but that always felt a bit clunky.

What S2I did, and does, is provide a mechanism for automating all those clunky bits. A developer doesn't need to know how to build an image or write a Dockerfile (now called a Containerfile and standardized). The S2I approach in the earlier versions of OpenShift (before builds came along) provided a standardized, non-language-specific way of building these images.

In essence, it broke the process into three virtual parts: compilation, assembly, and execution. In English:

- How do I build my application components (language, framework, and compilers)?
- How do I add the compiled application components to a framework that can host them (framework/base images, that is, RHEL with a JVM installed)?
- How do I execute my application to start it running correctly (basically, what is the start command for my application, that is, `java -jar` or starting Tomcat when the application is added to the appropriate directory)?

What the S2I framework provided was a way to register the three steps as an opinionated object. For example, if I wanted to do a Node.js Source-to-Image, I would use the `node.js` builder image, which I would provide with a source location (that is, a Git repo) and a configuration for the naming of objects created on OpenShift and which native objects to create (that is, a `Deployment`, a `Service` with port config, and a `Route` with a name).

Why the build process changed

Prior to the builds initiative, you would use a `BuildConfig` object, which defined the S2I stages to use, the framework image (because you can configure a framework image, you can use the same S2I approach for different versions of language frameworks—more in a moment in a recipe), and the object specifications to create. You would then instantiate a `Build` object, which would execute the S2I scripts. The `execute` script simply added the `RUN` or `CMD` to the composite Containerfile used to generate the combination of the base framework image and the application components. Upon a successful build, a composite image (and associated image stream) would be created.

So why do we now have builds? That comes down to where the builds themselves were executed. The S2I process creates images, and prior to a full version of CRI-O (the pod-based OCF-compliant container runtime that replaced Docker inside of OpenShift), the build process (a) had to run as root because of its use of the Docker daemon and (b) had to run as a cluster admin because of creating images. If you remember from the earlier chapter, images exist at the cluster level, not the user/project level. OpenShift uses image streams for the use of images at the user/project level.

Let's run a build to see it in action.

Recipe 1:3: Letting OpenShift do a build for you

For this recipe, we are going to use the world's worst Containerfile (I will admit, I wrote it as a cluster-breaking test because, you know, idiot). What we are going to do is provide OpenShift with a Git repo and let it decide what to build. OpenShift scans the target repo for certain build files or configuration details and makes a very educated guess on the contents.

For reference, the Git repo we will be using is at https://github.com/utherp0/openshiftbook, and we will be using the Developer Sandbox. If you have your own cluster, it will work identically.

Log in to the Developer Sandbox. In the top right of the web console, you will see several icons, including Terminal (**>_**) and a plus sign (**+**). This shortcut is for quickly adding content to the project.

Click the **Import from Git** drop-down item.

Enter the repo URL:

```
https://github.com/utherp0/openshiftbook
```

When the panel has validated the link, click **Show advanced Git options**. Set the **Context dir** to `/chapter01`. You will notice that the OpenShift UX detects multiple import strategies, but suggests **Dockerfile at Containerfile**. Leave everything as default, but scroll down to the **Resource Type** drop-down in **Deploy** and change the type from **Serverless Deployment** to **Deployment** (we'll talk about serverless at length in Chapter 9). Now click **Create**.

When the UX switches back to the Topology view, click the roundel for the application (labeled **(D) openshiftbook**). If you were quick enough, you can watch the build happen. In the **Builds** component of the content tab that appears on the right, click the build name. If you then click the **Logs** tab at the top of the content pane, you can see the build as it happens. Or, if you are doing it after the build has completed, you will see the logs for the successful build, as shown in Figure 1-2.

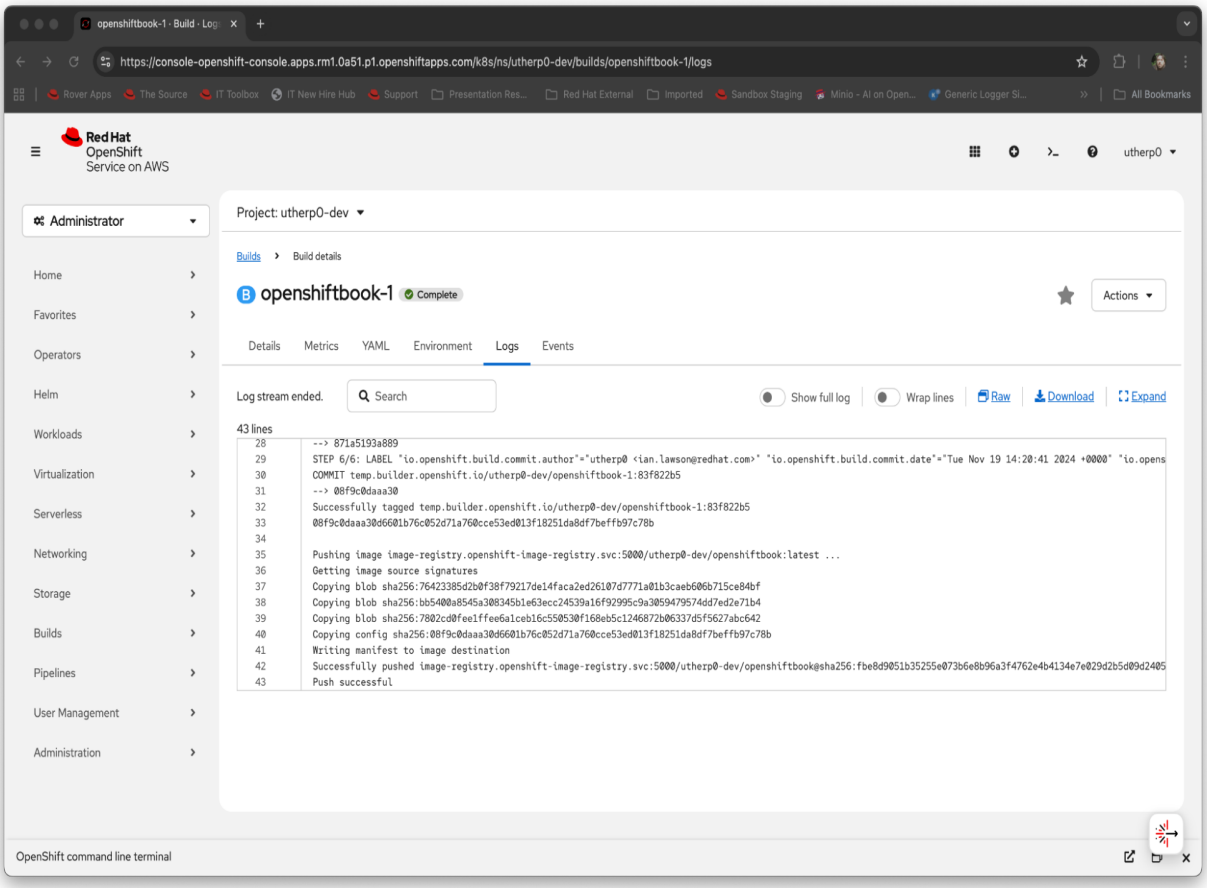


Figure 1-2: Build logs from a successful build from Containerfile.

Note that it has executed an image build. It detects the presence of a Containerfile in the context directory and uses that, with the context directory as a base for the build process.

Once the build process has completed, OpenShift will automatically create the objects needed to host the compiled image as an application. It will create a `Deployment` object with the created image from the integrated registry as the source, one replica, and a `Service` and a router for exposing traffic.

Switch to the **Topology** view by selecting **Workloads** / **Topology**. This is the biggest change in the web console, since the **Topology** page was previously part of the **Developer** viewpoint. Make sure the content pane for the deployment is shown (clicking the roundel for the `openshiftbook` application will toggle it on and off). In the **Pods** component, it will list a single pod (preceded by **(P)**). Next to this pod will be a state (in this case, "Running") and a link to the logs. Click the link to the logs.

As I said, world's worst Containerfile. It's in an infinite loop (shudder) and outputting the server time constant. Who says development is hard?

Finish up by removing the application. Switch back to the **Topology** view and click the application group (the label beneath the application starting with `(A)`). Then, in the **Application Group** content pane that appears on the right-hand side, click the **Action** drop-down menu and select **Delete application**. It's clever enough to force you to enter the name of the application group before it wipes it, and when you do, it removes all of the objects associated with the deployment(s) with the same application group.

In actuality, the application group isn't an object, but rather a label on the objects grouped within it. Figure 1-3 shows the YAML for the deployment with the application group label highlights. Small tip: You can manually change this, and the Topology will adjust appropriately. This is good if you are fussy like me and want everything on the Topology page to be tidy.

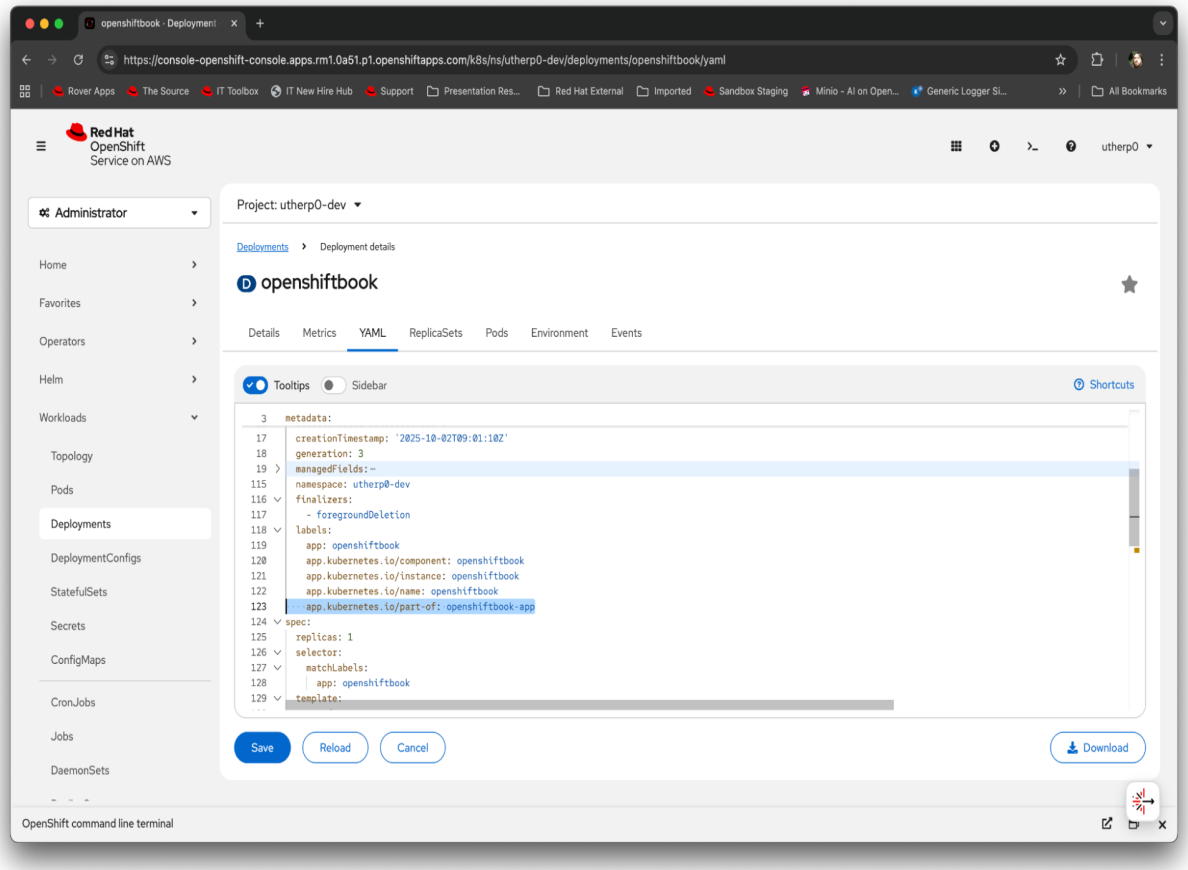


Figure 1-3: YAML editor with a label for application grouping highlighted.

What's next?

The next chapter dives into the ways to deploy an application. I have always visualized the process as two sides of a coin: *builds* provide a repeatable process for creating the currency (in this case, an image), and *deployments*, which we will look at next, works from the currency, appropriately running and managing it.

CHAPTER 2

Deploying your creation

Teasing the desired-state-engine

In Chapter 1, we looked at how to create images and pods—the container side of the fence, so to speak. You might remember I mentioned that, until I wrote this book, I had never actually created a pod myself. The joy of OpenShift and Kubernetes is that there are abstract, simple objects that do all the wiring for you when deploying your applications. The most basic one is, surprise, called a `Deployment`. But before we get to that, let's go over a tiny bit of history.

A key goal of OpenShift is to make a developer's life simpler. The developer components in the web console—notably the **Topology** and the **Software Catalog**—provide an opinionated view designed to ease the impact of "Kubernetes-speak" on developers. As part of this goal, the initial design of OpenShift objects (which extend and add functionality to the core of Kubernetes) was to create a couple of `xxxConfig` objects to help with the build and deploy process.

These were, again no surprise, called `BuildConfig` and `DeploymentConfig`, where `DeploymentConfig` was an extended version of the Kubernetes `Deployment` that provided rollback and other advanced features. As with all the extensions done for OpenShift, these were submitted back to the Kubernetes codebase, but neither was adopted. For a long time, we had this odd situation of being able to use both `Deployment` and `DeploymentConfig`, which was a little confusing, to say the least.

With the advent of `Build` (which removes the need for `BuildConfig`) and a rationalization (and increase in capability of the native `Deployment`), both `BuildConfig` and `DeploymentConfig` are gradually being deprecated in favor of the native `Deployment` and, in the case of builds (as we discussed in the last chapter), the `Shipwright`/`Build` object approach.

I mention this in passing because if you start to dig around examples for Deployments in OpenShift, you will see a lot of `DeploymentConfig`. Don't be confused: going forward, Deployments will be the standard way of doing a **static** deployment (more on that with serverless later in the book).

The point of this chapter is to give you a quick overview of the options for doing a deployment. There are a couple, and they are defined for significantly different use cases, as we will see.

Deployments

Let's start with the workhorse of workload deployments: deployments themselves.

Remember how OpenShift and Kubernetes work as a desired state engine. With a pod, you have a declarative statement (of state intent) that consists of a chunk of YAML containing metadata specific to the instance of the object and a direct specification that defines the pod itself to be created. With deployments, think of them as a level higher than pods. Again, you have a declarative statement saying you want a `Deployment`, with appropriate metadata defining characteristics of the object, and then a defined specification of what makes up the deployment.

So, with deployments, their base currency is effectively pods, with a lot of extras that define the context and additional objects that make up your core application. We'll get into a lot of these later in the book, notably storage, config maps, and secrets, and a dive into the witchcraft that is networking. For now, let's start with a basic deployment and go from there.

This example will also show you how easy it is to get started with OpenShift. We're not going to write any code at all. We're going to use an example application that is built from source. In fact, to make it fun, we're going to do a .NET build. (In my defense, I was a Microsoft developer for a little while back when C# had annotations and Java didn't.)

Recipe 2:1: A basic deployment

Log in to your Developer Sandbox account using the web interface. We will do this recipe entirely with the OpenShift UX.

Once you log in, you should find yourself in the Administrator viewpoint. A tiny bit of history: OpenShift 3 was exclusively a developer interface. When Red Hat designed the next version (the current one), the focus shifted toward simplifying the administrator's experience. As such, the first release of OpenShift 4 shipped without a dedicated developer perspective.

This was quickly addressed, resulting in a two-viewpoint system. The Developer viewpoint then offered components more focused on developers and less on deep Kubernetes concepts.

Over time, however, the Administrator viewpoint expanded to include most developer functionality. For developers, this often meant constantly switching back and forth between the views. As of version 4.19, the two key developer components—the Software Catalog for adding new items to your project and the versatile developer Topology—are consolidated into a single viewpoint.

Now, ensure you are in the **Administrator** viewpoint. (Note: There is a **Virtualization** viewpoint for managing virtual machines, but we will not use it.) Select **Home / Software Catalog** (Figure 2-1).

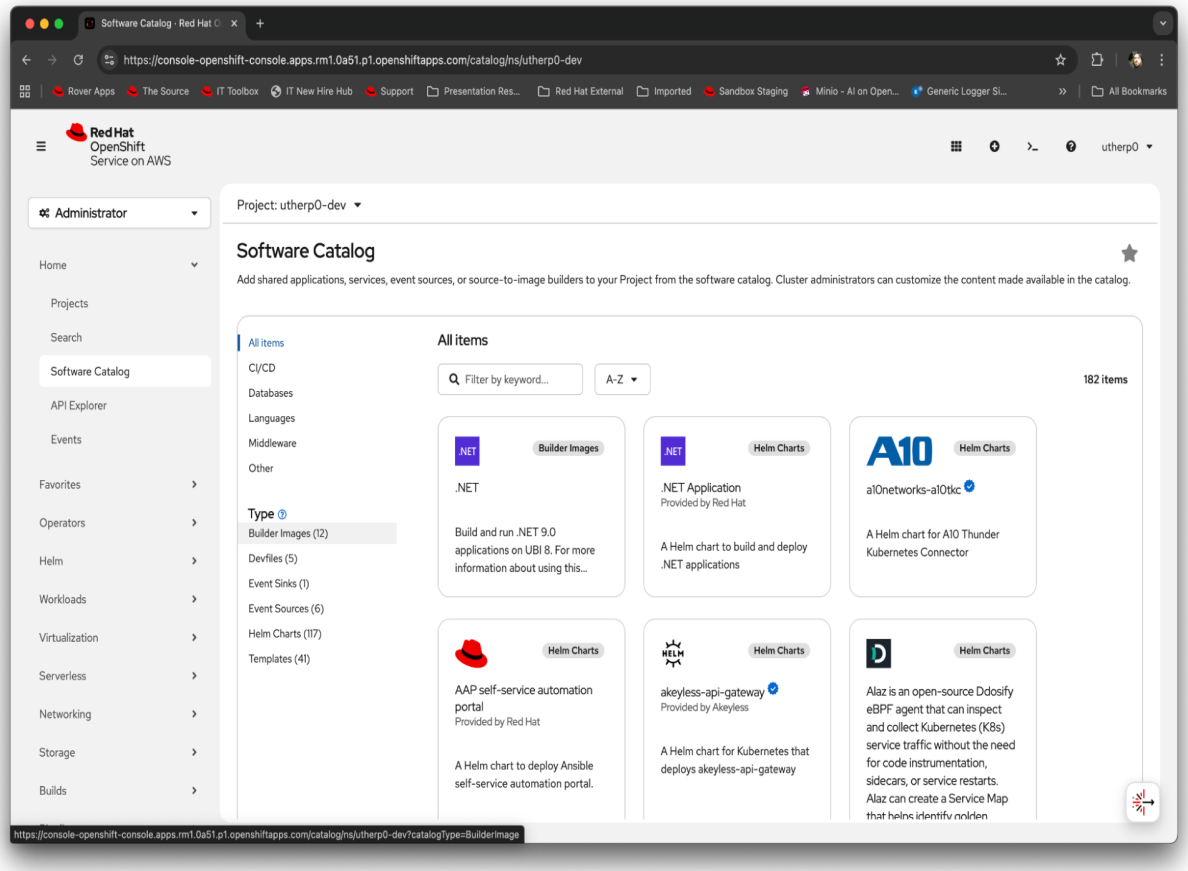


Figure 2-1: Catalog entries for the **Add** option in the Developer viewpoint.

Select **Builder Images** in the Type submenu.

.NET should be the first selectable tile on the right. If not, type `.NET` in the filter by keyword box. Click the **.NET** tile and you will see an overview of what OpenShift is going to create for you (Figure 2-2).

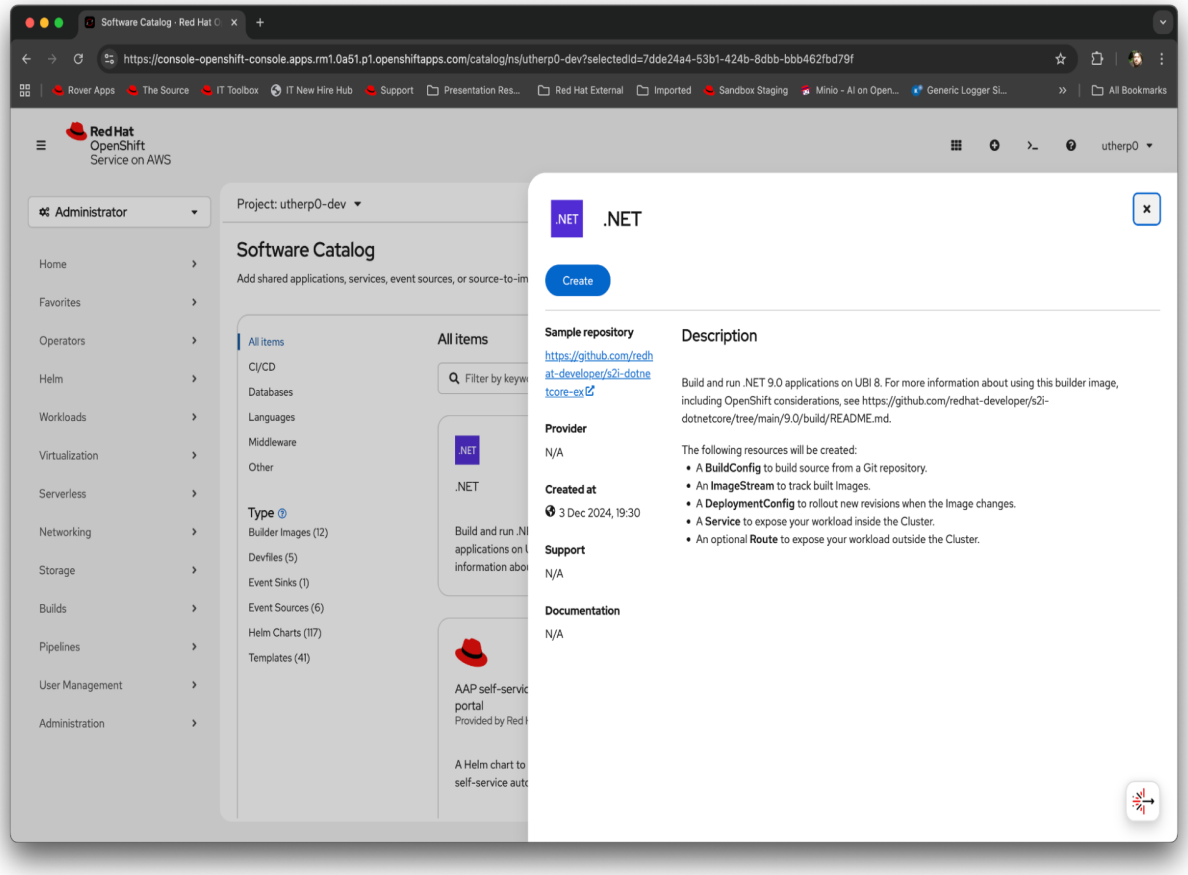


Figure 2-2: Object creation details on the .NET overview content pane.

Now, it's not strictly true. When we instantiate this template, we get to choose what kind of deployment we do. But you'll see that the system will create all the objects it needs for a full workload: the build, the deployment, the networking, and one of those pesky image streams we talked about in the previous chapter.

Click **Create** in this template.

You will now see a basic wizard that prompts you for all the info it needs to realize the workload. We're going to change a couple of things, but for now, click the **Try Sample** link beneath the Git repo URL to pre-fill the field with the Git link for an example .NET app.

Leave the **Application** and **Name** as is. Again, we have some overload of terminology, but in this case, *application* refers to a cosmetic grouping on the Developer viewpoint Topology

page. *Name* is important because all the objects that relate to this workload will share that name, which sounds mad at first but makes a huge amount of sense. `Build`, `Deployment`, and all the other base objects will have the same name but be different object types, which is nice.

Scroll down to the **Deploy** section. By default, the Developer Sandbox sets the deployment mechanism to serverless, which is one of my favorite things (but I will hold back on enthusing until we get to that chapter). For now, click the drop-down, take note of the definitions it provides (again, `DeploymentConfig` is going to be deprecated eventually), and click **Deployment**.

We don't need to change anything else. Click **Create** and watch the action happen.

A quick overview of what is happening: the system is doing an S2I build of a .NET application whose source is in a Git repo. Once it has completed the binary build, it produces a composite image based on a framework containing the runtime for the .NET and the binary artifacts. This is stored in the integrated registry and referenced by an `ImageStream` (a local object to the project in which the workload lives). It then creates a `Deployment` object that does a simple, one-replica instantiation of the workload using the composite image.

This is the strength of the deployment. Once it has finished and you have a workload running (there will be a roundel in the Topology with a blue ring if everything is healthy), click the roundel. On the right-hand side of the UX, a content panel will pop up that represents the deployment's desired and actual state.

Click directly on the name of the deployment. OpenShift always renders an indication of the object before the name, which will be a letter or an abbreviation of the object type. In this case, it is a **(D)** indicating a `Deployment`, followed by the text **s2i-dotnetcore-ex**.

Once you click it, the UX will render an overview of the deployment. This details page is actually a précis of the deployment. For the sake of this recipe, we will be looking directly at the YAML to see what a `Deployment` object looks like. Click the **YAML** tab on the deployment overview page, and you will see something similar to Figure 2-3.

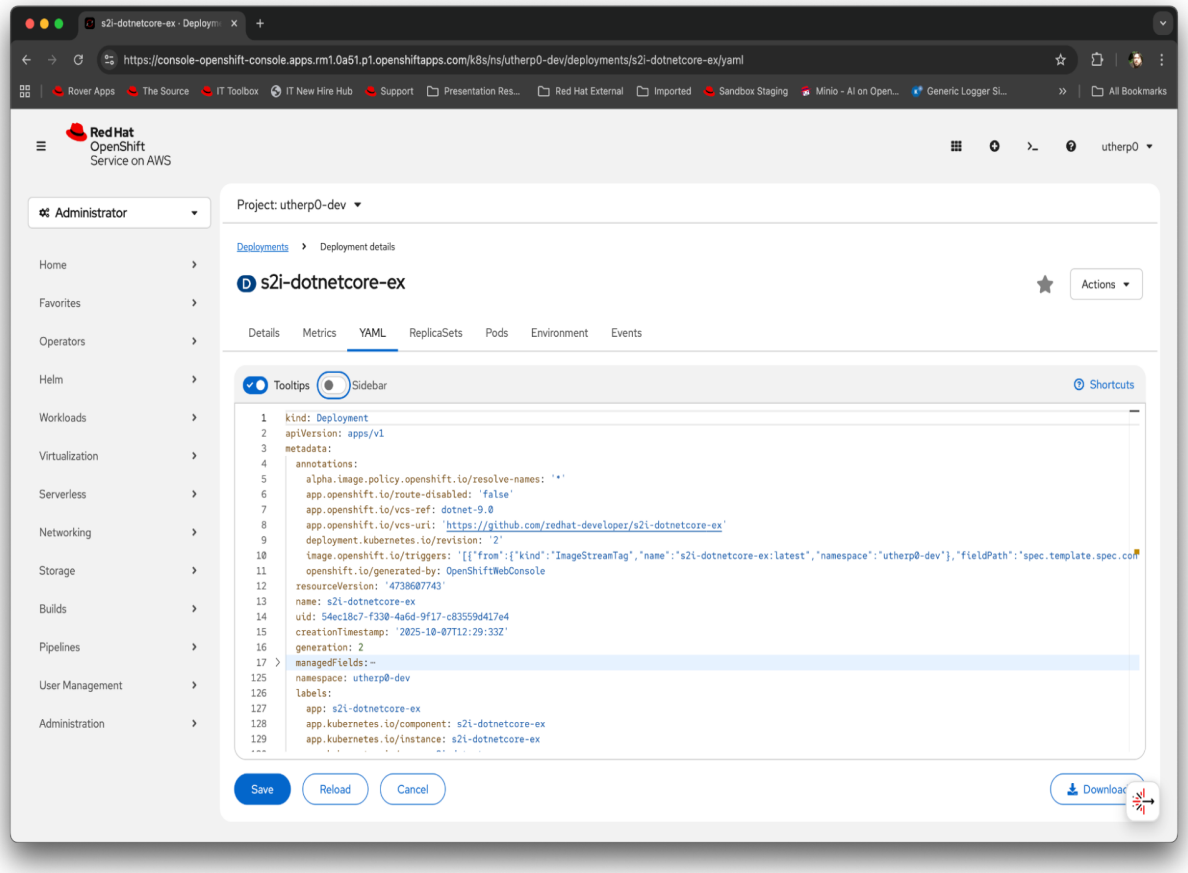


Figure 2-3: YAML editor view for the .NET example deployment.

If you scroll down the YAML, you'll see that the object, like every object in Kubernetes/OpenShift, has a `kind` and a `version`. This directs the declarative engine to process it accordingly.

Then you have two main chunks: `metadata`, which has definitions of *this* instance of the object, and `spec`, which defines the specification of the deployment.

This is where it gets a bit fun. If you look carefully at the `spec`(ification) for the deployment, you will see it defines a `template`, which in itself defines a `spec`(ification). The `template` and its specification define the containers themselves. In effect, the deployment has a desired state (its specification) that within itself has one or more desired states for the containers.

This is by design. The deployment wraps one or more container definitions that have to be realized. When the OpenShift cluster receives the declarative YAML for the deployment, the

nested specification will cause the creation of sub-objects that are directly linked, by state, to the health of the overall `Deployment`.

Recipe 2:2: Changing the deployment

Select **Workloads/Topology**, then click the **(D) s2i-etc** tab. If the right-hand side panel is hidden, it will pop up. Click the **(D) s2i-dotnet-ex** label at the top to go to the **Deployment** overview.

OpenShift is helpful in that it gives an opinionated view of the deployment, including bits that are directly relevant for the behavior of the application. On this front page, you can see a roundel that gives you a visual impression of the application's state. In this case, it should be blue. (Light blue means it is loading/initializing, and dark blue means it is terminating.) In the center of the roundel is a pod count, which indicates the current number of replicas running for this deployment.

If you've changed nothing, it should currently be set to **1**. Next to the roundel there are a set of arrows. One increments the pod count, and the other decrements. Click the up arrow until the number in the center is **5**. If you watch the roundel and the text, you will see the application scaling, and the color of the border (segmented by the proportion of pods running and starting) will change from light blue to blue.

This is an example where the interface offers direct functionality that changes the object definition, in this case, the specification of the deployment. Reset it to **1** copy using the down arrow and watch the roundel. As each replica pod is removed to meet the specification definition, a part of the roundel will switch to dark blue, indicating the pod is in a shutting-down state.

We will talk about replicas and the way traffic within the cluster is directed in Chapter 4, and we will look closely at the concept of replica sets in the next section. But for now, we will look at changing the specification of the deployment directly using the YAML.

You will notice that the **Deployments** overview page has a number of tabs. These include **ReplicaSets** (more in a moment), **Pods**, **Environment** (Chapter 3 goes into this), **Events**, and **Metrics** (Chapter 5). We will be using the built-in YAML editor, reachable using the YAML tab.

Click the **YAML** tab to get the editor pane up. The first thing you will notice is that the YAML renders and then collapses. This is because of the managed fields. If you remember when we talked about `create` and `apply` in terms of different behavior on the creation of objects, you

will recall that objects, when instantiated using `apply`, store the differences that have been applied within them for rollback. The managed fields component of the YAML contains those differences and can get very long and complex. Fortunately, for our sanity if not anything else, the OpenShift YAML editor collapses that part so the screen isn't full of "diff-gibberish" when you first open it. If you're brave, expand the managed fields and have a peek.

Scan down to the `spec:` tag of the deployment. Note that there are two `spec:` tags. The first is the specification for the deployment, and the second is the specification of the containers. Change the replicas to `5`.

Now scan down the embedded `spec:` tag. We are going to play with the actual specification of the containers. At the level of the `containers:` tag, replace the current definition, which looks like this:

```
containers:
  - name: s2i-dotnetcore-ex
    image: '.....'
    ports:
      - containerPort: 8080
        protocol: TCP
    resources: {}
    terminationMessagePath: /dev/termination-log
    terminationMessagePolicy: File
    imagePullPolicy: Always
```

with the following. Note that `'.....'` should be replaced with the image value you have. The easiest way to do it is to copy the entire `- name:` block of text and replicate it:

```
   spec:
      containers:
        - name: testcontainer1
          image: '.....'
          ports:
            - containerPort: 8080
              protocol: TCP
          resources: {}
          terminationMessagePath: /dev/termination-log
          terminationMessagePolicy: File
          imagePullPolicy: Always
        - name: testcontainer2
          image: 'quay.io/ilawson/testimage1:latest'
          resources: {}
          terminationMessagePath: /dev/termination-log
          terminationMessagePolicy: File
          imagePullPolicy: Always
```

Important: Do not replicate the `ports:` component. If you do, the deployment won't work because both containers will try to use port 8080. The specification for `testcontainer2` should contain no port information.

Now save it. If you pop back to the **Topology**, the deployment should reload. The `replicas` will switch to 5, and 5 pods will be deployed.

We changed the definition of the deployment to run two containers in each pod: the example Source-to-Image–built .NET application and the "world's worst container" we talked about earlier.

Choose a pod from the list of pods in the right-hand content pane (when you click the **(D) s2i-dotnet-ex** deployment). When the pod content pane appears, click **Logs**. (We will talk about logs in a later chapter.) Note that we can now look at the real-time logs of either of the containers running in that pod. Use the drop-down as shown in Figure 2-4 and you will see that the containers are named appropriately.

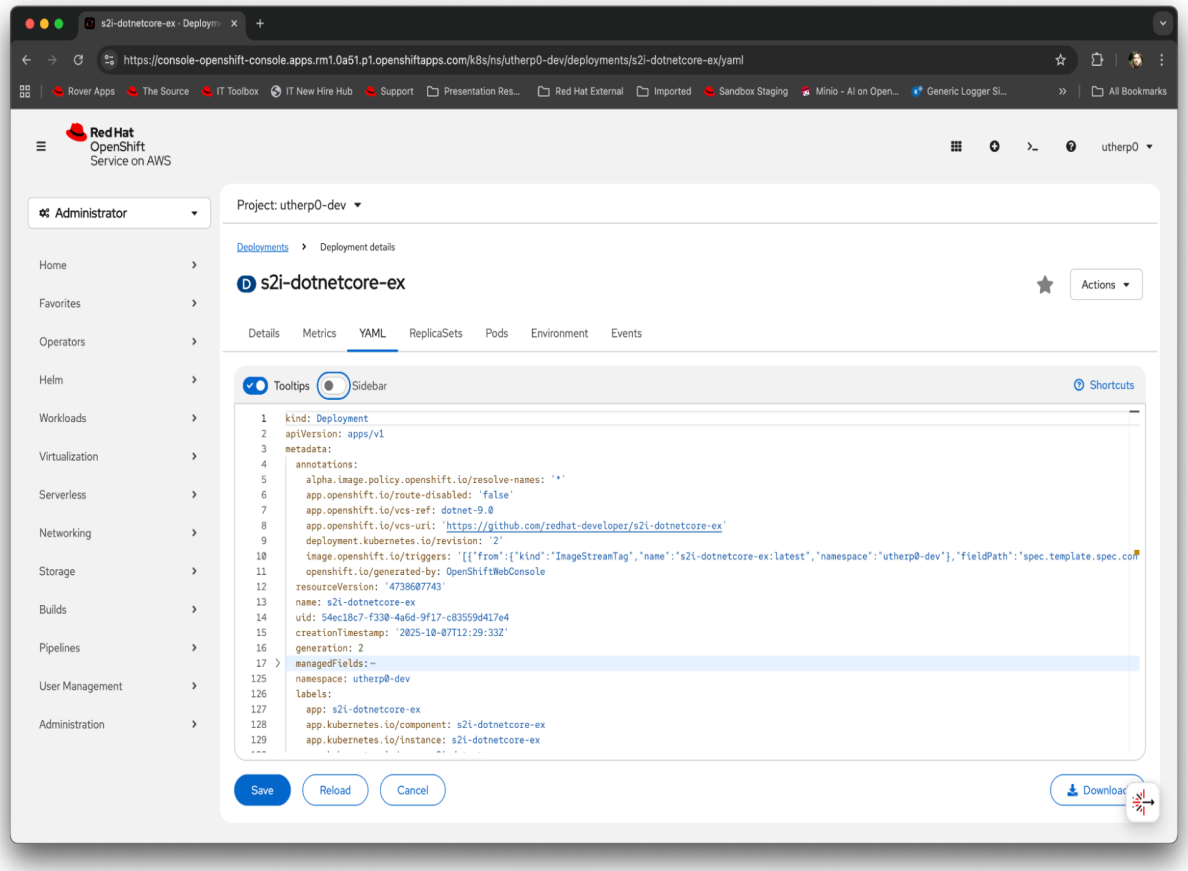


Figure 2-4: Log view for a deployment with multiple container drop-downs active.

Obviously, there are a huge number of things you can do from within the specification, but hopefully this shows how easy it is to change it.

Getting strategic

So, one last thing on the whole deployment stuff before we start to play with some fun things: when you looked at the YAML for the deployment earlier, you might have spotted this:

```
strategy:
   type: RollingUpdate
   rollingUpdate:
     maxUnavailable: 25%
     maxSurge: 25%
```

This is actually very cool; it allows you to specify the way in which the deployment is rolled out. It was designed to cater to two different real-world use cases:

- **Zero-downtime application update**: You want to roll out a new version of your application, but you must not interrupt its usage. This is the `RollingUpdate` strategy.
- **"Kill everything now!"**: When you need to roll out a new version of the application (say, to solve a security issue or deal with an application that cannot be left in a transactional state), this is the `Recreate` strategy.

How they work is as follows: The deployment is rolled out if any of its specifications change. In the case of a `RollingUpdate`, it works by shutting down one pod at a time, removing it, and then replacing it with the new version. This way, there is always an instance of the application running. In fact, if you look at the YAML extract, you will see that it has two parameters. One defines the maximum number of pods that can be starting at once (`maxSurge`). The other is the maximum number of pods that are allowed to be unavailable during the process (`maxUnavailable`), which allows fine-tuning of the rollout process to make sure the application is always running.

In the case of a `Recreate` strategy, it immediately terminates all replicas and then sequentially starts the new ones. That way, there is never a point at which pods of different versions are running.

CHAPTER 3

Changing stuff without changing the image

So far, we have talked about creating images, deploying containers as pods, and other fun basic stuff. As someone who spent too much time patching and rewriting applications (often while they were live in production—the fun side of working in the defense industry), you get to know that making it possible to change an application's behavior without having to recompile it is one of a developer's little pleasures.

Given that the process of recreating an image is reasonably intensive, and recreating an application can be overkill for, say, changing the `<title>` tag on a webpage, the idea of dependency injection came along. In English, this is the driving of an application's behavior by externally provided configuration. This makes complete sense. We used to call it "writing sausage machines," but given the number of vegans in the industry, it has sensibly been relabeled as "cloud-native" application design.

Given the native ephemeral behavior of pods, it makes absolute sense to write applications that are context-fluid. This was at the forefront of the Kubernetes creators' minds when they came up with the original object model, and the mechanisms of pods allow for changing a container's context and file systems without having to change the image.

Remember that images are immutable. When a container is served as part of a pod, the container's initial file state comes from the image. As we said, an image is a temporal map of file layers, or differences. These are applied in order to the container file space, and then an overlay file system is added to allow the container to change its contents. When the container is restarted, the file state is reset to the original image.

So how do you programmatically change the container's context or behavior without having to change the original image? OpenShift and Kubernetes provide two native objects, with a

number of ways to apply them, to allow for dependency injection. They also have the concepts of volumes, which we will cover in depth in Chapter 6.

Put simply, the two native objects, `ConfigMap` and `Secret`, allow for the injection of configuration information as either environment variables or files (including overwriting files in the image—this is incredibly powerful). You can also add entire file system mounts using persistent volumes, which allow containers and pods to interact directly with file systems that are retained when a pod is destroyed.

In this chapter, we will talk about (and play with) config maps and secrets for dependency injection.

Dependency injection with environment variables

Environment variables are an essential part of the Linux ecosystem, and many of us developers take them for granted. OpenShift and Kubernetes provide a mechanism, as part of container creation, to create and impose environment variables. Some are inherited from the container runtime host, some are inherited from the image, and the rest are injected at a container or pod level. They are all visible to the container itself, but don't assume that all are created. The environment variables at the container level are inherited from the user (more on that in Chapter 10).

Before we cover the native objects that allow you to group and import configuration information, it's worth being aware that you can code environment variables directly into the pods and deployments as part of the specification.

During this chapter, we will be playing with an application I wrote a long time ago. I still use it because (a) it's in Node.js so it builds super quickly, and (b) I added some interesting JavaScript scripts to render the environment variables exposed to the cluster. I like JavaScript, but that's a conversation for another book or my psychiatrist.

Recipe 3:0: Setting up the ocpnode application

Quick and easy. Log in to the Sandbox (or your own cluster if you are using it). Select **Home/Software Catalog**. Enter `node.js` in the **Filter by keyword** textbox. Now click **Builder Images** in the **Type** list on the left.

Click the remaining tile, **Node.js Builder Image**. Click **Create** on the information panel that appears.

In the **Git Repo URL**, enter:

```
https://github.com/utherp0/ocpnode
```

If you have no application groups in the project, you will be prompted for one and it will be prefilled with `ocpnode-app`. If, like me, you already have a lot of application groups, click the drop-down next to the application and click **Create Application**. Set the application name to `ocpnode-app`.

Leave the Build option as **BuildConfig**.

Set the **Deploy Resource** type to **Deployment** (click the drop-down—it will default to serverless, which we will talk about at length later in the book).

Click **Create**. You can watch the build working by clicking **View logs** next to the listed builds on the right-hand panel. Once it has finished and the image has been correctly added to the internal registry, the roundel will switch to blue (normal, not light), and you will have an application running.

If you click the route icon (top right of the roundel), the application will pop up in a separate tab. Apologies in advance, I haven't updated that stylesheet in 15 years. Secretly, I like it and don't want to.

Recipe 3:1: Injecting environment variables through the deployment

For this recipe we will be using the Developer Sandbox again; make sure you have set up an account and have logged in appropriately. Then follow Recipe 3:0 if you haven't already done it, to deploy the `ocpnode` test application.

To see what environment variables have been expressed into the deployed application, you can use two methods:

- In the **Topology** UX, click the icon at the top right of the roundel. The webpage for the application will appear in a new tab. In the web application, click the **Reveal** button. This will render a panel containing all the environment variables visible to the application. It's cool, but you can't really search it. If you like reading environment variables, use this approach.
- If you click the roundel itself, a panel will pop up on the right-hand side. In this panel will be details of the deployment. Click the center tab (**Resources**), and it will show the components that make up the application. Click the name of the pod listed in the **Pods**

section, and when the pod panel appears, click **Terminal**. Be aware, this terminal will auto-close if you leave it for too long. If it does, simply reconnect using the link in the panel. In the terminal itself, enter `env` and click return. You can search this easily using `env | grep (text)`, and we will use that in this recipe.

If you look at the environment variables, you will find that OpenShift creates and injects a number of useful ones to do with the connectivity of applications (not just this one) running in the project. This was designed to make service discovery easy, and we'll discuss more of that in Chapter 4.

Now do a quick search to see if the pod or container has any environment variables that start with `INJECTED_`. For example, if you are using method 2 from above, enter:

```
env | grep INJECTED_
```

You will find none (I know, not exciting).

Now go back to the **Topology** page in the OpenShift interface and click the roundel to bring up the information panel on the right-hand side. Now we will use the drill-down approach within the UX to look at the deployment itself. At the top of the panel, the deployment name will be shown (with a **(D)** next to it to indicate its type). Click the name.

> **Note:** You can also get to it as with any object in the system by searching the set of objects. In the interface, click **Home/Search** in the left-hand navigation panel. In the **Resources** drop-down, enter `deployment` and then click the **(D) Deployment** checkbox. The system will then list all deployments in the current project. (If you have access to more than one project, this page can be set to search individual projects or all of them at once.)

Clicking the **(D) ocpnode** name in the list acts exactly as clicking the **(D) ocpnode** title in the **Deployment** panel on the **Topology** page.

In both cases, you will get a content page that renders with the contents of the deployment object. This page has a number of tabs, including the **Overview** tab, which gives a visualization of the pod replicas (that is, you click the up and down arrows to add or remove exact replicas of the deployment components, creating new pods in OpenShift). For the sake of this example, click the **Environment** tab.

This chapter will go into the config maps and secrets, but you will see from this form (Figure 3-1) that this is where you can inject environment variables in different ways.

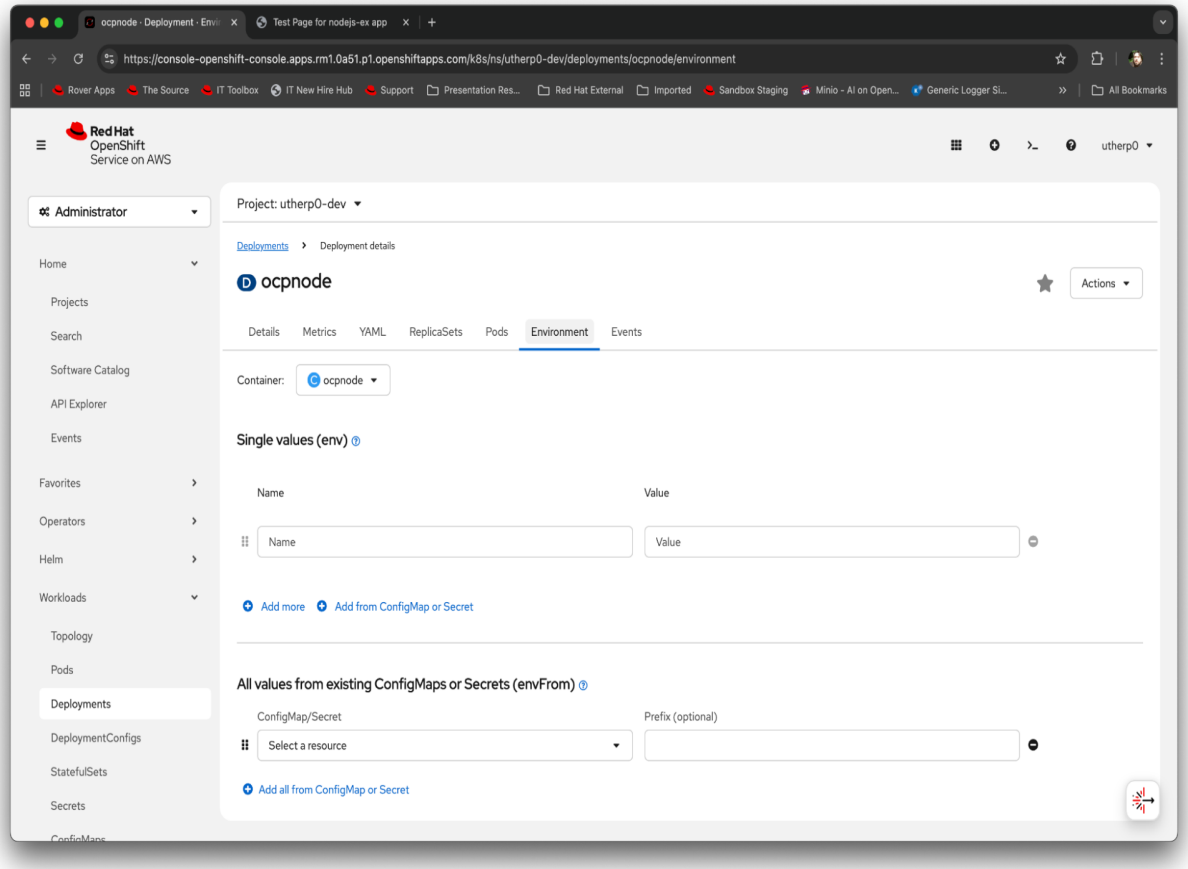


Figure 3-1: Content pane for environment settings on deployment.

Note the **Single values**. We will use these components to add a couple of environment variables to our deployment. These will be injected into the pods as part of the deployment itself.

Firstly, in the **Name** section of the Single values form, enter `INJECTED_variable1`. Then put anything you want in the **Value**.

Now click **Add more**. This time, in the **Name** section, enter `INJECTED_variable2` and put anything you want in the **Value**.

Click **Save** at the bottom of the form. This will change the object definition for the deployment, and OpenShift will work to reconcile it. What this means is that the specification for the deployment has changed. A new version of the deployment will overwrite the existing one in the shared data store. The existing components in the system created from this deployment (that is, `replicasets`, pods, and the like) that are impacted by the change are

removed and replaced. If you are quick, you will see the pod being removed and recreated in the **Topology**.

Once the pod roundel is blue again, indicating it is ready and healthy, follow the preceding steps to check the environment variables. When you search for variables that start with `INJECTED_`, you will now see the two variables you added.

Now that's fine but a bit fiddly. Imagine if you have, say, 100 variables you wanted to add. Or you have the same 10 variables, but you have five applications of different definitions that use the same variables. Doing it "by hand" at this point becomes intensive and prone to error. And that's where config maps and secrets come in.

One last thing before we go on to config maps: changes to the environment variables of the deployment change the specification of the deployment, which means the cluster immediately moves to reconcile the changes. You would have noticed that saving the variables as part of the deployment caused the pod to reload. Be aware that config maps do not work the same way. Because config maps are a standalone object, if you change one, the deployment that uses it won't redeploy—we'll discuss why next.

Dependency injection with config maps

The `ConfigMap` object addresses the issue of injecting information into the container without changing the base image in a different way than simply providing environment variables, as we have just done with deployments.

The deployments model works fine, but say you have two completely separate deployments that use the same remote data store. If you were replicating the information across both deployments, there is a possibility of inaccuracy and a chance to forget to update one when the data store information inevitably changes.

The concept of a `ConfigMap` is a standalone object, project-bound, that contains fixed information that can be applied into a deployment. What's nice about this is the way in which you can apply it, although, as mentioned earlier, you need to be careful of a little "gotcha" (to do with state changes, which we will look at in a recipe).

The easiest way to get a feel for the mechanism of config maps is to try one in anger.

Recipe 3:2: Injected stuff using an abstracted object (ConfigMap)

For this recipe, we will use the `ocpnode` application. It's nice in that it has a neat render feature for the environment variables visible to the application via a webpage. The recipe will use both the webpage and the terminal to check the variables.

Follow Recipe 3:0 to install the `ocpnode` application if you don't have it in place.

It might not be evident from my writing (cough), but I have a tendency toward technical specificity. I used to get moaned at a lot because I love to (over)comment my code. (Javadoc was my secret passion.) As such, I like my UX to be tidy.

One of the latest web console features is the ability to add custom shortcuts. Look at the left-hand navigation bar, where you'll find a selectable component labeled **Favorites**. When viewing a content pane, you will see a star icon; selecting this star will instantly add that pane to your Favorites.

Once we have the application running, we will add a couple of environment variables, as we did earlier. If you are carrying on from the previous recipe, there will be a couple of variables in the deployment (see Recipe 3:1 for details). We are going to do something similar, but this time using a `ConfigMap`.

Now click **Workloads/ConfigMaps** in the left-hand navigation. Interestingly, you will see your project already has a couple of config maps. These are ones that are generated by the Sandbox and used for some levels of authentication. Click **Create ConfigMap** at the top right of the content panel.

The input form (not the YAML editor) might seem a little complex. It's one of the original ones that matches the exact structure of the `ConfigMap` definition and isn't overly friendly, but it makes sense once you start to use it. A config map is actually pretty simple in its definition. It has a `Name` that uniquely identifies it within the project and a set of key-value pairs, where the value can be singular, a chunk of text, or a chunk of binary. We'll talk about the chunks in the next section, where we look at some cool stuff around mounting the contents of a config map as a file.

For now, set the **Name** to `myfirstconfigmap`. In the **Key** field, enter:

```
variable3
```

Now click the **Value** field, and you'll see why I think this content page is a little "confuddled." It's unselectable, which is a little bit confusing. Instead, in the **Drag and drop** field below, type `stuff`.

Now click the **Add key/value** button below the entry fields, and the fields we have used will be replicated. Set the key in the new fields to `variable4` and add `more stuff` in the **Drag and drop** field. Then click **Create**. The system will create a new `ConfigMap` object for you and take you to the ConfigMap overview content page. If you scroll down the content pane, you will see two keys in the map, as we entered above.

Now click the YAML tab of this pane. This is one of the few times where the YAML makes more sense than the opinionated input form. Your `ConfigMap` YAML will look similar to this:

```
kind: ConfigMap
apiVersion: v1
metadata:
  name: myfirstconfigmap
  namespace: utherp0-dev
  uid: b847fcd9-6020-4c88-bf5d-2b2470de1947
  resourceVersion: '227226966'
  creationTimestamp: '2025-02-01T16:29:28Z'
  managedFields:
    - lots of annoying gibberish (cough)
immutable: false
data:
  variable3: stuff
  variable4: more stuff
```

I have bolded the interesting bits. Note that `ConfigMap` is (interestingly) not immutable. Also, note the two key/value pairs in the `data:` component.

Now pop back to the **Workloads/Topology** page. If the right-hand content pane is not showing the **(D) ocpnode** details, then click **(D) ocpnode** under the roundel for the application.

In the **Actions** drop-down at the top right of the pane, choose **Edit Deployment**.

You will now have the opinionated form for the deployment in the content pane. If you do not, select **Form view**. Scroll down to **Environment Variables**, like we did in Recipe 3:1. Under the **name/value** entry fields, you will see another option for **Add from ConfigMap or Secret**. Click this.

This facility allows you to pick a single defined value from your config map and add it to the Environment definition for the deployment. In the drop-down for **Select a Resource**, you can

choose your config map and then choose the value, adding a name for it to be injected with into the deployment.

We're not going to do this (it's fiddly), so click **Cancel**. Luckily, there's a more generic way of doing it, which is more fun. In the right-hand content pane, click the name of the deployment to get to the deployment content pane.

Now choose the **Environment** tab. Here you will see two options: adding a single value (which was the shortcut on the overview pane) and the more fun one, adding all values.

In the **All values** section, click **Select a resource** and choose the config map `myfirstconfigmap`. Now for the fun bit: in the **Prefix (optional)** input point, enter:

```
INJECTED_
```

Now click **Save** to change the deployment specification. If you are quick (and you have to be very quick), you can pop back to the **Workloads/Topology** pane and see the deployment redeploying. It's quick because you haven't changed anything to do with the image being used, so it is simply a case of re-spinning the pod to inject the new environment variables.

Now click the pod name in the **Pods** part of the right hand content pane. Once the pod overview appears, click **Terminal**. Enter the following:

```
env | grep INJECT
```

And you should see the environment variables you have just added; the cluster has taken all the keys from the config map and added them, with a prefix of `INJECTED_`, with the values specified in the config map.

Now for the little "gotcha." Click **ConfigMaps → myfirstconfigmap** to get the overview of your config map.

Click **YAML** to get the editor for the object. Change the following:

```
variable4: more stuff
```

to:

```
variable4: some more stuff
```

And click **Save**. Now click **Topology**, then click the pod name in the right-hand navigation pane. Again, click **Terminal** and enter:

```
env | grep INJECT
```

Hmm. `INJECTED_variable4` is *still* set to `more stuff`.

So, this highlights the way in which Kubernetes/OpenShift maintain object state. The deployment specification says it uses the config map, but *changes to the config map itself do not change the specification of the deployment*. Yeah, it's a little confusing, and I have fallen foul of it a number of times.

Pop back to the **Workloads/Topology**. If the right-hand panel isn't there, click the **(D) ocpnode** label under the roundel to get it up. Select **Action** and then choose **Restart Rollout**. Watch the pod cycle down and up. The specification hasn't changed, so we have had to force a rollout ourselves.

Click the pod name for the new pod, then choose the Terminal and enter:

```
env | grep INJECT
```

And there you are, the update is there. Just keep in mind that the reconciliation loop we talked about earlier with regard to objects *only applies to the object itself*. In the case of a config map, updating it is not seen as a specification change to the deployment.

Config maps as volumes

That's all well and good—being able to inject environment variables dynamically. From a dependency injection perspective, it's very useful, but there's another mode of injection supported by OpenShift that, as a developer, I really like.

Before I explain how this works, I want to give a real-world example that shows how powerful this is. A while ago, around the creation of OpenShift 3 (the first Kubernetes-based version), I had to host a pod that offered an online chat facility for a workshop I was doing. I was hosting the chat facility on OpenShift, and it needed a database. The problem was, the original application (and it was Rocket for all you oldies out there) shipped with connections to MySQL. At the time, there was a little bit of oddness going on with MySQL and Oracle, and as a result, the creator of MySQL spun an open source version called MariaDB. At the time, there were some implications of running MySQL on OpenShift, so I decided to use PostgreSQL instead.

The docs for Rocket said I could use it; I just needed to change the configuration file for Rocket. The problem was that the containerized version of Rocket had an effectively hardcoded file for MySQL. This was before I had gotten my head around the witchcraft of Containerfiles (and this was back when it was just Dockerfiles).

So, I was aware of this cool little feature where you could express a config map as a file. So, I wrote a new config file for talking to an external Postgres database, created it as an object on OpenShift, and then wrote the deployment *to overwrite the container file system where the MySQL conf file was with the contents of my file*.

In English, the container was deployed with the files as defined, immutably, in the image. I just overwrote the exact file at the exact file mount point with the contents of my file. *Et voilá*, Rocket was running against Postgres, and I hadn't had to change the original image structure.

This might seem a bit hacky, but it's a very useful tool for using images where you can't overwrite the original contents.

So, how does it work?

It works by taking advantage of the way that OpenShift and Kubernetes allow you to mount volumes into the container file space. Let's do it with a recipe to show how simple it is.

Recipe 3:3: Injecting a ConfigMap as a file into a container or pod

Again, for this, we are going to use the good old `ocpnode` application. If you haven't gotten it running in your project, follow Recipe 3:0.

First, a test: go to the **Workloads/Topology** page. If the right-hand navigation isn't present, click the **(D) ocpnode** beneath the application's roundel.

Click the pod name for the running pod for the deployment. In the pod content pane, click **Terminal**, and enter:

```
ls /etc/config
```

You should get a little whinge saying the file or directory doesn't exist.

Click **Workloads/ConfigMaps** on the left-hand navigation. We are going to create a `ConfigMap` to contain our "configuration file." Now click **Create ConfigMap**.

In the name, enter:

```
myconfigmapfile
```

Here's the cool bit: in the **Key** field, we are going to give a filename:

```
myconfig.conf
```

In the **Drag and drop** field, enter the following:

```
# My file contents
var1=wibble
var2=wobble
var3=wubble
```

Then click **Create**. Once the object has instantiated, click the **YAML** tab to see the YAML we have created. You will notice it has the developer's little hack of defining a chunk of "stuff not to process as YAML," using the `|:` symbol.

```
data:
  myconfig.conf: |-
    # My file contents
    var1=wibble
    var2=wobble
    var3=wubble
```

Now for the even more fun stuff. We are going to change the specification of the application to mount the contents of this `ConfigMap` as a physical volume or mount in the container file space. It sounds more complicated than it is. Switch to the **Workloads/Topology** view, click the roundel for the application to get the deployment content pane on the right-hand side, and then in the **Actions** drop-down, select **Edit Deployment**.

Switch to the **YAML** view. We are going to code some YAML directly to map the volume.

Scroll down to the `specification:` tag for the containers. It will look something like this:

```
 spec:
      containers:
        - name: ocpnode
```

We are going to add a volume mount to the specification for the container. At the same indent as `name:`, add the following (in bold, and note that the following extract is not the entire YAML—I have cut it down to avoid cluttering the example):

```
   spec:
      containers:
        - name: ocpnode
          image: >-
            (stuff)
          ports:
            - containerPort: 8080
              protocol: TCP
          envFrom:
            - prefix: INJECTED_
              configMapRef:
                name: myfirstconfigmap
          resources: {}
          terminationMessagePath: /dev/termination-log
```

```
          terminationMessagePolicy: File
          imagePullPolicy: Always
          volumeMounts:
          - name: config-volume
            mountPath: /etc/config
```

Don't save yet. We have told the deployment to mount a volume called `config-volume` at that mount point, but we haven't defined the volume itself yet. Volumes are defined at the same indent as containers, as part of the specification. Add the following (in bold):

```
  spec:
      containers:
        - name: ocpnode
          STUFF HERE
      volumes:
        - name: config-volume
          configMap:
            name: myconfigmapfile
      restartPolicy: Always
```

We have defined and satisfied the volume definition (for `config-volume`) using the contents of the config map. In actuality, what it does is create the mount point and write the config map in with the key as a file name and the field contents as the actual file.

Once you save, the deployment will reload itself, as the specification has changed. Pop back to the **Topology** page. When the pod is active and healthy, click the pod name in the right-hand content pane. (For the deployment, again click the roundel if it is not there.)

In the pod content pane, click **Terminal** and enter:

```
ls /etc/config
cat /etc/config/
```

And there you go, the key and value have been expressed as a file (named by the key) containing the exact contents of the config map.

Now, to put the mind of the developer at rest, we will try and mess with it. In the terminal, enter:

```
sh-4.4$ echo "wibble" >> myconfig.conf
sh-4.4$ ls -al myconfig.conf
```

What will happen is that it won't allow you to write over or change the contents of the file from within the container. It is expressed as a symbolic link (the `ls -al` will tell you that in detail) that links to a read-only file. So naughty developers can't terminal in and mess with your configuration file. Result.

Keeping it quiet: Secrets

As someone who has, like most developers, accidentally committed my AWS credentials to a Git repo (if you know, you know), the subject of security is quite important to me. With the advent of Kubernetes/OpenShift and the shift to cloud-native development and dependency injection, there was always worry around sensitive information being provided to the applications.

In the old days, developers mostly just stuffed connection strings, database passwords, and the like into config maps, but there was a simple problem with that.

Firstly, config maps are stored in flat text, `utf-8`, or binary. They aren't encrypted by default. Secondly, when a container is mapped to a worker node as part of a pod, the config map is written as a file into the worker node OS and then mounted into the container file space. The obvious worry here is that someone with access to the worker node at the OS level can read these files.

This was assuaged a little in OpenShift 4 by the advent of Red Hat Enterprise Linux CoreOS. The default OS that underpins the nodes, worker, control plane, and infrastructure is immutable and has a lot (that is, all) of the OS tooling removed. But if you were sneaky (and I am), you could always terminal into the nodes from the Administrator viewpoint on the UX (and via `oc`) if you had the rights.

Then along came secrets. Secrets work exactly the same as config maps in the way they can be defined and applied, but they have one major and important difference.

When you create a secret, the contents are encrypted. This is at storage, in transit, and when written to the consuming nodes' file system. They are decrypted on write into the container space. In English, this means that no one except the application can see the original values stored in the secret.

As with config maps, you can express the contents of a secret as environment variables or as a mounted volume. We won't do any recipes, as the mechanism is exactly as with config maps. But if you want to have a go, repeat the recipes for config maps but use the **Workloads/Secrets** pane instead.

One thing to add is that when you create a secret, you will see the option to create other types of media for the secret. The screen looks like Figure 3-2.

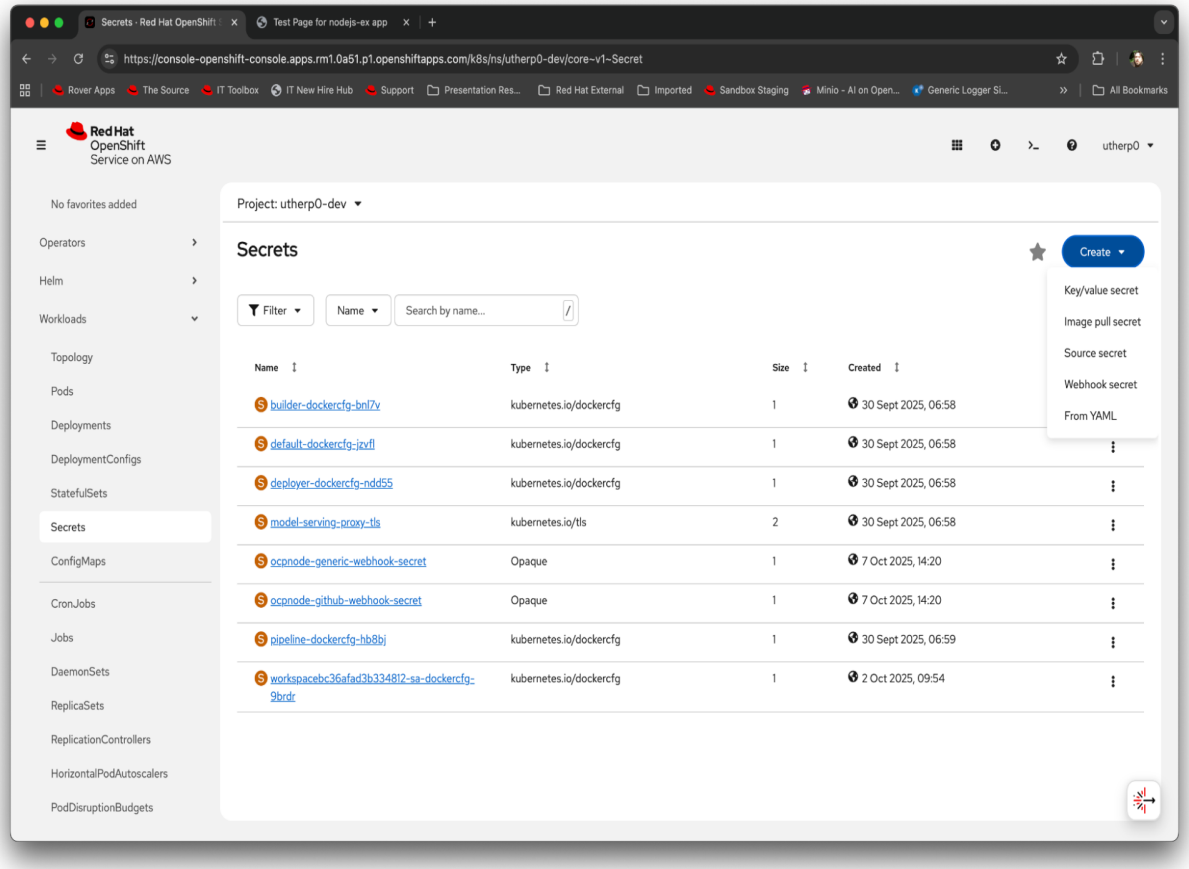


Figure 3-2: Selection of secret types from the Secrets content pane.

The `Secret` object also supports (and handles in the same way in terms of encryption) secrets for pulling images from registries (that is, `quay.io`), pulling and pushing source to code repos (like Git), and handshakes for webhooks. These are all things a developer uses.

For example, if I want to play with `devspaces` (which is a fantastic part of OpenShift that provides a hosted and in-browser editor and build/test environment with direct connectivity to GitHub), I have to set up a source secret in my project to allow `devspaces` to "be me" when interacting with Git.

PART 2

Infrastructure: The stuff around the application

CHAPTER 4

Wiring it up: The SDN

If we were face-to-face and I started a conversation with "Now, let's talk about networking!" I know the majority of you (myself included) would check out behind your eyes and think of something else. Developers don't care about networking. We never really have (unless you are a network engineer, of course), and really, we shouldn't—the whole concept of TCP/IP and packets should be way behind the covers.

But how many of us have spent sleepless nights trying to fix issues with our applications talking to other stuff? Granted, 99% of the time it's DNS (because it's always DNS), but things like service discovery, dependency injection for endpoints, and connection strings (even after 15 years away from programming against Oracle, I can still write a connection string in my sleep) are a headache.

So let's talk about Kubernetes and OpenShift networking for a little bit. This is actually quite fun because of the wonderfully abstract way they offer networking.

Doorbells

A little recap of the way OpenShift hangs together first: all of the OpenShift cluster's infrastructure lives on its own network, the SDN (software-defined network). This network is abstracted away from the real world and contains two sets of IP ranges: one for the application endpoints and one for the actual pod endpoints.

The control plane machines and the worker nodes, where workloads execute, live on the same SDN. This allows the control plane to talk to the `kubelet`. The `kubelet` is a little robot that exists on each of the nodes, responds to object state changes, and then watches the created components to make sure they remain at the required state.

The application endpoints provide a doorbell for traffic to get to an endpoint. The cleverness comes from the fact that an application endpoint, represented by a service, as shown in Figure 4-1.

So pods run on the worker nodes and have their own IP range (normally `10.*`). Application endpoints are cluster-wide and have their own IP range (normally `172.*`). All communications with the pods are via the application endpoint, which redirects traffic accordingly.

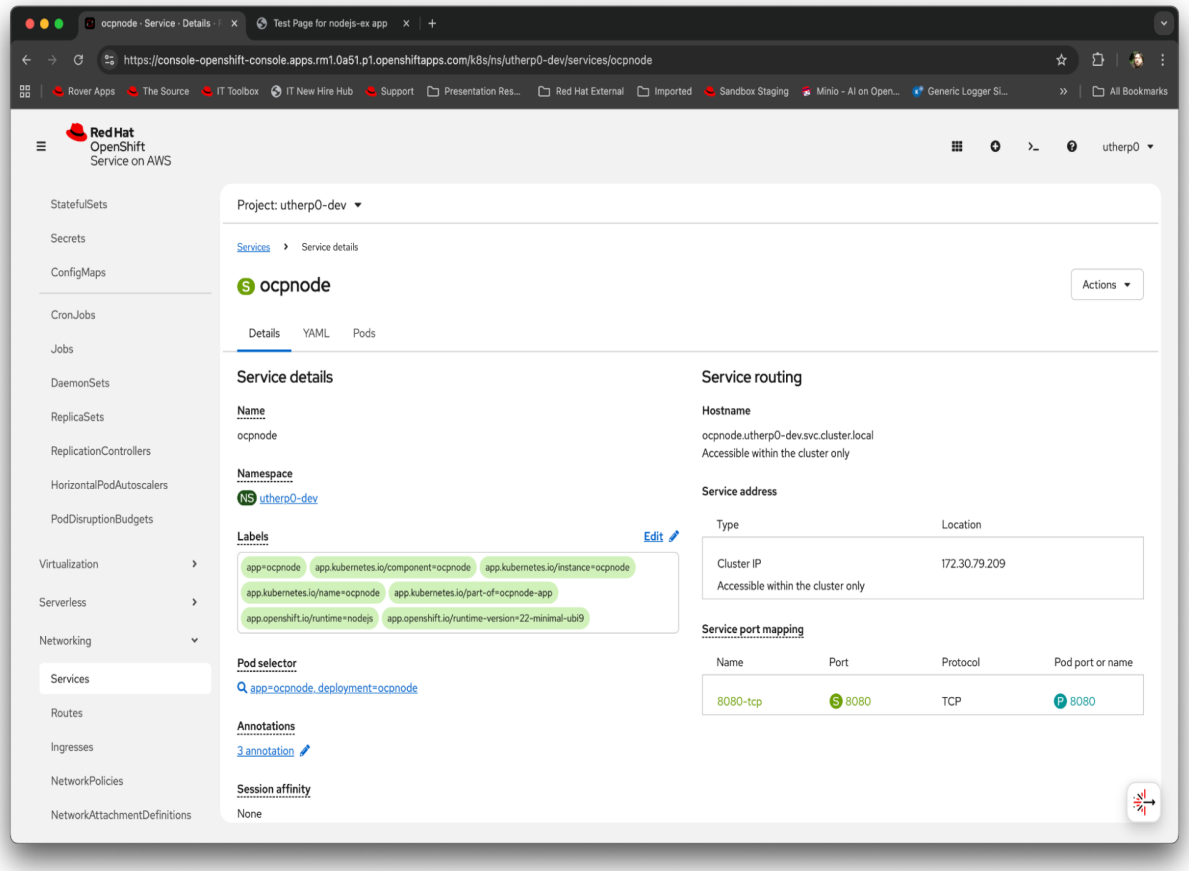


Figure 4-1: Service details content pane.

If your application has one replica, the service/application endpoint will push traffic into it. If you have more than one replica, the first time an external entity connects, the service will load balance the connection to one of the replicas and then "sticky-state" it. By default, if the source disconnects and reconnects, it will be connected to the same replica it got at the first connection.

To get traffic from the outside world into the cluster and into the application, OpenShift provides an object called a `Route`. This is different from Kubernetes. Routes were created to

make life easier for admins and developers, but Kubernetes has its own mechanism called ingress controllers. They are a bit more fiddly and require a degree of network understanding, whereas routes wrap all of the complexity internally, leaving you, as the developer, to just create a `Route` (which is done for you automatically if you create an application via `oc` or the OpenShift UX).

Essentially, OpenShift maintains a set of routes, which translate to resolvable fully qualified domain names (FQDN). The interesting thing is that the routes will always resolve to the same (or a similar) IP address regardless of the endpoint. This is because OpenShift maintains its own router system. The external FQDN delivers the connection to the cluster, and OpenShift uses the actual FQDN to translate the endpoint to an application's service. This service then translates to a pod's IP, depending on the number of replicas of the deployment.

So, this is all sounding very network-y, and I hope I haven't lost you because it gets so much easier for us developers now. Keep in mind you have routes (external FQDN to cluster edge), services (internal cluster doorbell), and pods themselves (you don't need to know about the networking of these—again, more in a second).

Any port in a storm

So, services are doorbells. Well, kind of—let's expand on that.

A service maps an application endpoint port to a surfaced port, but taking a step back, all "machines" have a set of ports that can be used for communication. In reality, these are not physical ports but rather an ID that is mapped into the packet for consumption on the machine. For instance, port 80 is the standard port a web server will listen on, and port 8080 is a standard port a dynamic web content provider will listen on.

But single machines only have one of each "port." For example, if an application grabs port 80, when another application starts that wants to listen on it, it is refused. With containers, this caused a problem. Let's say you want to run four or five web servers on your machine as containers. Only one can have ownership of port 80.

So when Docker was designed, it created the concept of "port forwarding." This allowed you to start a container that thought it was listening on port 80, but in actuality, the container runtime host was maintaining a redirection table. An external process would connect on (for example) 9001, and the runtime host would feed this into the container on its port 80. So you could have many copies of the web server running as containers, with each thinking it was

listening on port 80 whereas the container runtime host was actually listening on a different port for each of the containers.

This was fine when you were running a single host, but adding multiple hosts made it very difficult to configure. So Kubernetes introduced the abstraction of the process, allowing for singular services to listen on ports when behind the scenes, there are multiple instances on different worker nodes and even different ports within the replicas.

The way services are defined is elegant: you specify the port you want to offer for connectivity, the port within the pod to satisfy it, and the selector for where to push the traffic. Figure 4-2 shows a sample service.

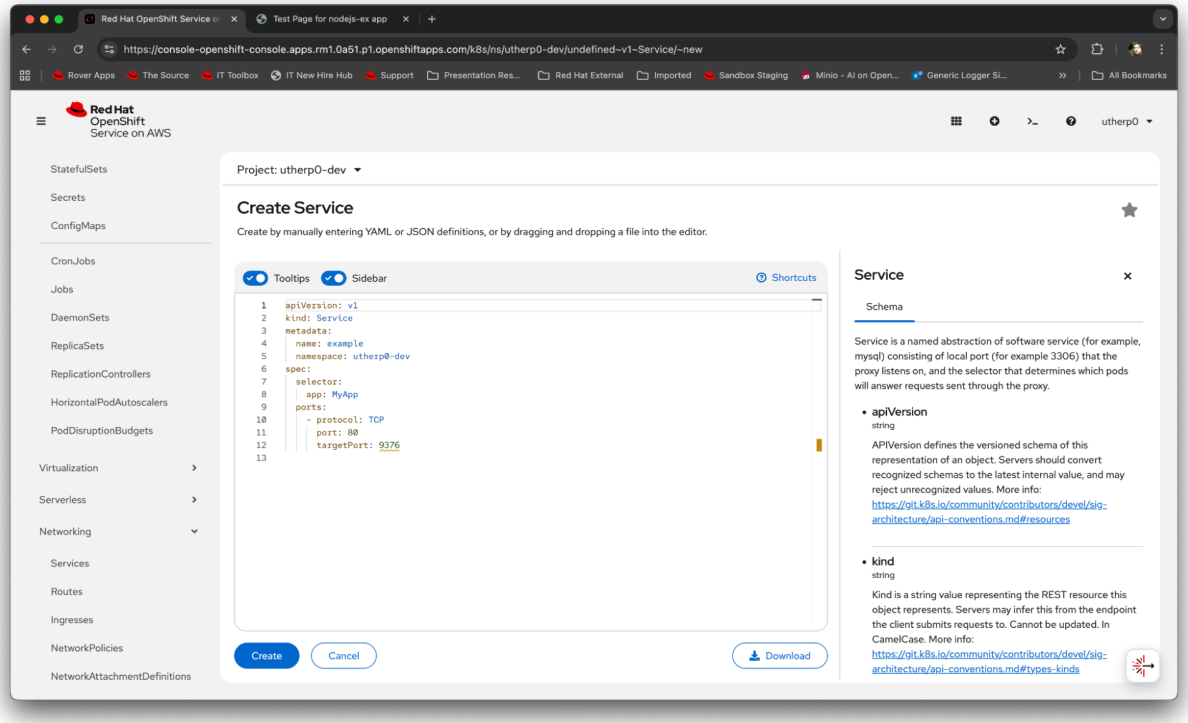


Figure 4-2: Service creation YAML pane.

The selector is an interesting one. You can add any label to the specification for your deployment and use that to select upon. In fact, with OpenShift Virtualization (where you host VMs directly on the worker node and control them/express them through a pod), you can also do this, which makes for very easy modernization (more in a later chapter).

OpenShift is very helpful when doing all this stuff for you, but for a quick exercise, we are going to remove the OpenShift-created service and craft our own, just to get a feel for it.

Recipe 4:1: Creating a service manually

What we are going to do here is use some of the OpenShift features to quickly create an application and then remove the service access. Then we will add a label to the OpenShift application and create a new service.

Make sure you are logged in to the Developer Sandbox (or your own instance of OpenShift) and in a project that you have admin rights to. (With the Sandbox, you only have access to one on your own instance; use an existing one or create it.)

Make sure you are in a project you have access rights to; if you are using the Developer Sandbox, select **Projects** and then your **-dev** project. Now, select **Home/Software Catalog**.

Switch to the Developer viewpoint and click **Add**. Select **Templates**, and then **Apache HTTP Server**. Click **Instantiate Template**. Leave everything as default and click **Create**.

Now let OpenShift do its stuff. Basically, it's creating all the objects you need: a `BuildConfig` to perform the build and produce the composite image, a `Deployment` to instantiate that application, and all the required networking bits. The output should look similar to Figure 4-3 (click the roundel to get the right-side content if it is not there already).

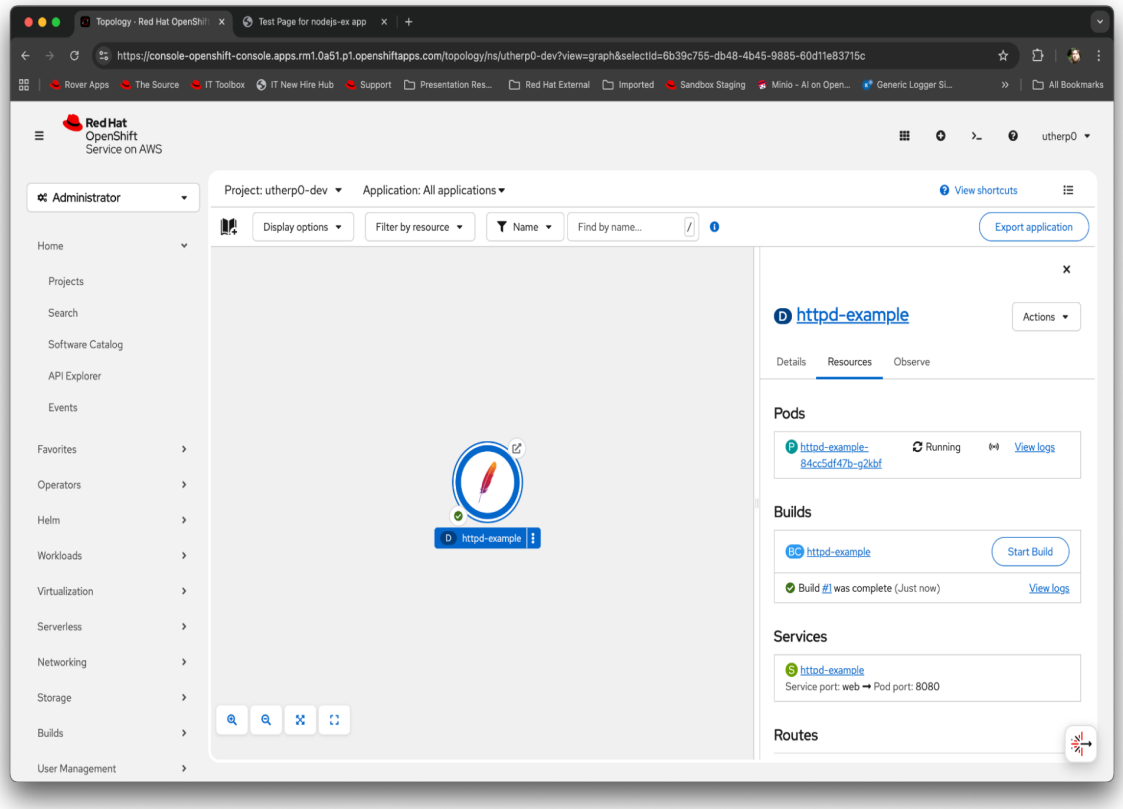


Figure 4-3: Instantiated template providing a predefined httpd application.

Now we are going to do some demolition. First, click the **Route** object in the bottom left, not the URL, but the **httpd-example** with the letters **(RT)** next to it.

When the **Route** overview appears, pull down the action menu on the right-hand side and select **Delete Route**. OpenShift will ask you if you want to be silly; acknowledge it.

Before we do any more mischief, we are going to check that everything is working, that is, before we make it not so. In the old days, you would have to create a test pod and use that to check network stuff. I actually produced an image that you could execute, and in the great way that is open source, the OpenShift folks thought it was a good idea and added a similar image (with all the appropriate CLI tools) as part of the core interface. You have to add the Terminal Operator, but luckily that is already installed in the Sandbox.

Click the terminal prompt icon at the top right of the user interface. This will bring up a little sub-panel at the bottom of the screen. Accept the defaults (and make sure it is running in your project), and after a little load time, you will get a Terminal window, giving you a Linux prompt. Try typing the following first, just to show you the context:

```
oc whoami
oc whoami --show-server=true
```

Interestingly, the `--show-server` will give you an IP endpoint. As we discussed before, by default, OpenShift Container Platform provides two IP subdomains: one for the actual pods (the actual SDN endpoints) and one for application endpoints (leading to a load balancer/route over the SDN endpoints).

You'll notice that the start of the IP address for the server is on the application endpoint subdomain. The API engine for OpenShift Container Platform exists as an endpoint on its own application endpoint subdomain, meaning applications can access the OpenShift/Kubernetes API directly without hopping out of the cluster and back in. You can also directly access the registry; we will talk about DNS shortcuts later in this chapter.

Now try this:

```
oc get svc | grep httpd
```

The terminal will list the service endpoint for our test `httpd` pod. You will notice it has a cluster IP, which is the application endpoint subdomain.

To check that the service is working before we murder it, enter:

```
curl http://httpd-example:8080 >> test.txt
ls -al test.txt
```

You will see the `httpd` server responds (using the service name—more on that later).

To show how services behave on the application endpoint subdomain, we are going to create another service that hooks traffic into the application, then delete the original. You can now minimize the Terminal window (using the horizontal line in the top right) to free up space in the web console.

Switch to the **Administrator** viewpoint. Select **Networking** in the left-hand navigation, then click **Services**. This will render the Services content pane. There might be more than one service in there, but you will see the `httpd-example` one. Note the **Pod Selector** field; this is used to deliver the traffic appropriately.

Now click **Create Service** at the top right. The content pane will change to a YAML editor with the basic definition for a vanilla service. Delete the entire contents, and then enter the following:

```
kind: Service
```

```
apiVersion: v1
metadata:
  name: testservice1
spec:
  ipFamilies:
    - IPv4
  ports:
    - name: web
      protocol: TCP
      port: 8080
      targetPort: 8080
  internalTrafficPolicy: Cluster
  type: ClusterIP
  ipFamilyPolicy: SingleStack
  sessionAffinity: None
  selector:
    name: httpd-example
```

Now click **Create**. The UX will render a content pane containing an overview of the new service. Either click **Services** at the breadcrumb (above the **(S) testservice1** title link), or use the left-hand navigation to go to the services list.

If you sort the list using the pod selector column (just click the text for **Pod selector**), you will see there are now two services using the `name=httpd-example` as a selector. The new service, when called by name, will direct traffic.

To test this, first remove the `httpd-service`. From the list, click the kebab (the three vertical dots) and select **Delete service**. Then click **Delete** to confirm.

Head back to the **Terminal** panel. If you have closed it, simply click the terminal prompt icon at the top of the UX.

First, enter this to see that the old service has gone:

```
curl http://httpd-example:8080 >> test.txt
```

You will get a response that shows 0 bytes downloaded and a status message of `Could not resolve host: httpd-example`. The `DNS` entry has been removed, and the service no longer exists or resolves.

Now try:

```
curl http://testservice1:8080 >> test2.txt
```

Note that when using the service name to request traffic internally, you have to add the port number, even though the port is exposed by the service. This command should execute

successfully: you have manually created a service, and your application is no longer orphaned and unreachable.

Shortcut notations

As a developer, I was always getting confused by endpoints. (I still do.) The whole realm of service discovery has been a pain since day one, and in the current environment of cloud-first and microservices, it's even more fiddly. So when I started playing with OpenShift a long while ago, I was pleasantly surprised (read: utterly) to find that the routing and resolution of endpoints is done in a very simple and usable way.

We had a hint of this in the previous recipes. By creating a `Service` object with a name, that name is directly resolvable within the project. This means that to communicate with an application hosted in the same project, you can directly use the service name as the DNS.

This is limited to the context of the project (that is, any pod running in the project can directly resolve the application endpoints of other pods using the short name of the service), but it can be directly controlled by another object called a `NetworkPolicy` (more in a moment).

By default, any services defined in the project are reachable by name as a DNS entry by any of the pods. (This has to do with the network namespace. Without going into stuff developers hate, each of the projects maintains a distinct namespace on which the pods within it run, unless you join them with another project.)

But wait, there's something else cool. The cluster maintains the lookup for all the IPs on both of the subdomains, the application endpoints and SDN endpoints. Each pod has its own addressable IP address on the SDN endpoints, which is load balanced/directed via the application endpoints. This allows for multiple replicas of the same application and scale-up/scale-down without changing the entry point for traffic.

It also provides these IP addresses as cluster-wide resolvable DNS entries, using a subdomain of `(name).(project).svc.cluster.local`. You'll see this notation a lot if you look at the cluster operators.

This is a DNS shorthand for communicating cluster-wide. With the Sandbox, you only have access to a single project, but in a full version of OpenShift Container Platform (or an organization one), a developer would be working across multiple projects. We'll try a recipe to show how simple the communication mechanism is.

Recipe 4:2: Using the short and full DNS endpoints

In the Sandbox, fire up the Terminal window (click the terminal prompt icon in the top right of the UX).

In the top section of the page (the OpenShift UX), select **Workloads/Topology**. Make sure the application we created earlier, with the service `testservice1`, is running. To save on resources, the Sandbox will "offline" applications/pods after a given time of inaction, so make sure the deployment for the `httpd-example` is running at **1** replica rather than 0.

Quick test to see if the application is active: in the **Topology**, click the roundel for the `httpd-example` deployment to open the content pane on the right with the deployment details. If not there already, switch to the **Resources** tab, and then click directly on the pod name (preceded by **(P)**).

Now click **Terminal**, and when the terminal window appears, enter the following. Note that this is a terminal directly into the container within the pod itself, rather than the project-wide terminal offered via the lower panel.

```
curl http://localhost:8080 >> test.txt
```

This will, if successful, indicate the `httpd` server is active (and write the response into `test.txt`).

Switch to the **Details** tab. Using the browser's copy/paste controls, copy the **Pod IP** field contents. This should be something like `10.128.xx.yy`. Switch back to the **Terminal** tab and enter:

```
curl http://(paste the IP address here):8080 >> test2.txt
```

This also works. From within the container, you can reference the IP address.

Now switch to the terminal panel at the bottom of the page (opened using the terminal prompt icon in the top right of the OpenShift UX). Enter the following:

```
curl http://(paste the IP address here):8080 >> test3.txt
```

This works, too. Within the context of the project, you can reach the direct IP address of the pods (rather than going in through the abstracted application endpoints). Now we will try the short notation for the application endpoints. Enter the following in the terminal panel:

```
curl http://testservice1:8080 >> test4.txt
```

This has connected to the application through the shorthand service name, which resolves to the application endpoint. An important note here: if you have more than one replica (let's say two pods running of the same deployment), each pod will have a different IP address on the SDN subdomain (for example, `10.128.xx.yy`) but the same application endpoint. In fact, it uses sticky sessions. Once you have connected via the service name, all requests from your IP endpoint will go to the same pod.

The usefulness of the service and the shorthand notation is that you can change the number of replicas (and even redirect the service point to other applications using the selector) without changing the way in which the end consumer calls the application. This abstraction is by design. Imagine an application that scales on load, handling burst compute. This mechanism lets you offer a single, non-changing endpoint for the consumers, meaning no configuration other than the original endpoint (and we will talk about routes in a second, which provide external linkage into the application endpoints) has to be pushed to the consumer. This is much better than the old days, where service discovery fell under the remit of the calling application.

Let's switch to testing the long form. Enter the following to see what the long form of your application endpoint looks like:

```
echo "http://testservice1.$(oc project --short=true).svc.cluster.local"
```

I love the fact you can embed commands in other Linux commands. Apologies for it looking so hacky. What we are doing here is using `oc project` to give us the short name of the project and output it as a composite URL.

```
curl http://testservice1.$(oc project --short=true).svc.cluster.local:8080 >>
test5.txt
```

This DNS translates to the IP address of the application endpoint. The subdomain `*.svc.cluster.local` is provided for all services in all projects. Because the Sandbox is a massive shared system, there are tight controls on what you can and can't access, which we will talk about in the network policies section later.

For now, just be aware that, as a developer, you can make some assumptions about the naming and connectivity of applications in your and shared projects. These remain the same regardless of which OpenShift cluster you are running on, which means code you write that uses this kind of functionality has to be written once, with next to no configuration changes when moving from cluster to cluster. It's one of the most important features to me when it

comes to writing massive distributed applications. We'll discuss more on that (and cooler ways to do it) when we get to serverless in Chapter 9.

Routes

So that's the way that internal applications can easily communicate with each other, but how on earth do we allow external entities to reach these applications?

This is where OpenShift and Kubernetes tend to drift apart a bit. In Kubernetes, you use a set of functionality around ingress controllers, which require a good deal of understanding how networking actually works. This is fine when you have network engineers to call upon, but if you're like me (as a developer) and have never cared (sorry, any network engineers reading) about how everything plugs together, it can be a bit, well, "faffy."

Routes are amazingly simple, but to understand them you need to visualize the way in which an OpenShift cluster talks to the outside world. Put simply, there are only two points of access into an OpenShift cluster: the API endpoint (where you send your object specifications and their state), which terminates at the control plane nodes, and the "router" endpoint, which terminates in a clever little engine that acts as the "signal box" for all the workloads running on the `*.svc.cluster.local` subdomain.

Of course, it is much more complicated than that because of "networking." But if you visualize this, the concepts of routes becomes easier to understand. A route is an externally resolvable DNS entry that translates at the router into an application endpoint. Basically, external consumers (such as browsers or applications) cannot resolve to the subdomains internally within OpenShift. The SDN is completely abstracted from the outside world, both the application endpoint subdomain and the SDN (pod) endpoint subdomain. Those IP ranges don't exist (or if they do, they point to other machines at an organization—`10.x.y.z` is a very popular internal subdomain).

So what OpenShift does, by default (and this can and normally is changed by organizations using load balancers), is require two DNS rules to be set. The first is a fixed DNS, such as `myopenshift.uths-company.com`, that resolves to the control plane. The second is a wildcard DNS resolution that resolves to something like `(*).apps.uths-company.com`.

Remember that URLs are just labels (like phone numbers) that are translated. What this rule gives you is the same IP or range of IPs endpoint for all permutations of the `(*)`. For instance, `ping myapp.myproject.apps.uths-company.com` and `ping`

`someoneelsesapp.theirproject.apps.uths-company.com` will resolve to the same IP address.

This delivers a connection to the router and its replicas in the system. At this point, the router uses definitions/rules based on the wildcard content. In the example I have given, the router would translate `myapp.myproject` to a given service, based on the existence of a `Route` object that has the defined URL. At the point the traffic enters the cluster, it resolves to the application endpoint, which then resolves to an SDN endpoint (based on the number of replicas, sticky sessions, and other rules that can be configured).

If you are anything like me (even though I wrote the previous paragraphs), you'll be going "Hmm" at this point. It's nice to know the mechanics, but as developers, we don't really need to know the ins and outs. However, the `Route` object gives some very cool features because it allows you, as the developer or the platform engineer, to dynamically change the endpoint of a static URL.

In a normal use case, you write your application, expose your endpoints (ports) as services, and then add routes. When doing this for the first time (and I fell into this trap/mindset), you have to remember that you have a route per endpoint.

A much simpler definition: say you have an application hosted on Tomcat (my favorite dynamic server, as I might have mentioned way too many times already). Tomcat exposes two ports, normally (three if you remember `https`, but security is fiddly, he says, knowing how many people will get cross with that statement): the web-serving endpoint (`8080`) and the administration endpoint (`8088`). To get to it, you would have a single URL, say `mytomcat.uth.com`, and get to the web-serving via `http://mytomcat.uth.com:8080` and the administration via `http://mytomcat.uth.com:8088`.

However, with a containerized version, it works differently. The `8080` and `8088` ports are exposed through different services. This is part of the abstracted design. So you would have two routes to get in, something like `http://mytomcat.myproject.apps.uth.com` and `http://mytomcatadmin.myproject.apps.uth.com`. Note that you don't give it a port. That caught me out at first.

Anyway, I digress. What makes a route so cool is that you can give it multiple endpoint services, based on a percentage. This means that if you have an application and you want to field-test a new version (a *canary deployment*), you can simply stand up a new deployment for the new version, expose a service, and then add that service to the route with a target percentage of traffic.

This is done "sticky-session" style. If you had, say, 80% going to the original service and 20% going to the new one, 8 out of 10 requests would be directed to the old one, and any further requests from that IP/header would always go to the old one. Two out of 10 would be directed to the new one.

Creating and modifying a route is simple. As with all things on OpenShift, they are Kubernetes objects (albeit ones that are supported by the OpenShift API, not the Kube one). What we will do now is create a route that ingresses traffic for our example service and then have some fun with it.

Recipe 4:3: Hooking a route into our service

In the Sandbox, click the **Networking** menu item in the left-hand panel. Now select **Routes**. If you have no other applications running other than the `httpd-example`, this content pane will show `No route found`, or it will list existing route objects in your project.

Click **Create Route**. As with all supported objects, you can either use a form (which prompts you for all the required, mandatory, and optional fields) or "slum it" in the YAML editor. I tend to do both. The forms are extremely useful for getting started, and the YAML editor is essential for updating. You can also craft the YAML externally and deliver it to the OCP cluster using the `oc create -f` and `oc apply -f` commands. Reread the chapter on `oc` for more details. I like using the command line, but it can be a little unfriendly.

In the route creation dialog page, set the name to `mytestservice`. Leave the hostname as is. In the old days, the route URLs were limited to the format `(application).(project).apps.whatever`, but later versions of the router allow for any URL—as long as the URL resolves externally to the OpenShift routers for the cluster.

Leave the path as is. You can use the path to define subdirectories in the route's URL. I always get confused about these because they resolve to the top level of the application. For example, you could define a route such as `a.b.c.d/e`, but in your application (say a web server), it would resolve to the root directory. If (and it's a silly example) you wanted to access the `/e` directory in your web server, you would need to use `a.b.c.d/e/e`. Down that road lies madness and hair pulling.

Now for the fun part: click the **Service** drop-down. This, by default, lists all the services in the project that you have access to. In this case, we are going to direct this route to the `testservice1` service, so select that. You will notice the page changes to add more selectables at this point.

The clever bit is the **Service weight** and **Add alternate service** components. We won't be using them in this recipe but will use them in the next one to show how to drive traffic to separate services from the same route. Leave the weight at 100. It works like percentages: the system adds the total of all the weights together and determines the chosen service point using probability. I always use numbers that total up to 100 because I like percentages.

Click the **Target port** drop-down. This dynamically provides all the ports that are exposed by the application via this service. Select **8080 → 8080(TCP)**.

Leave the **Secure Route** unchecked. This checkbox allows you to secure the route by providing certificates and the like. `https` and security is a complete topic on its own, but here's a quick 30-second overview.

OpenShift allows you to terminate the SSL at the cluster entry point, the application itself (passthrough), or re-encrypt. With re-encrypt, the SSL certificate is checked and terminated at the point the traffic enters the cluster, then re-encrypted against an internal cert and sent to the application/pod. This allows for single point control (edge termination), application-level control (passthrough), and "encrypted within the cluster" (re-encrypt). This means your packet traffic remains unreadable right up to the application but encrypted against an internal certificate. That previous sentence is why I, as a developer, never check that box.

Now, if you're still here after that brain dump, click the **Create** button. The cluster will create the `Route` object and provide you with an overview pane. In the overview pane, click the **Location** provided (this will be the fully qualified domain name produced as part of the route).

If all is well, you will see the home page for the `httpd` application. Now we are going to mess it up because there's a page you need to see (and will probably see a lot while developing with OpenShift). Switch to the **Developer** viewpoint, go to the **Topology**, and click the **(D) httpd-example** to get the right-hand content pane up for the deployment. Switch to the **Details** tab and scale the deployment down to **0** replicas using the down arrow.

When the deployment has scaled down to zero, click the small icon at the top right of the roundel. This is a shortcut to the route.

You will now see this gem of a screen (Figure 4-4).

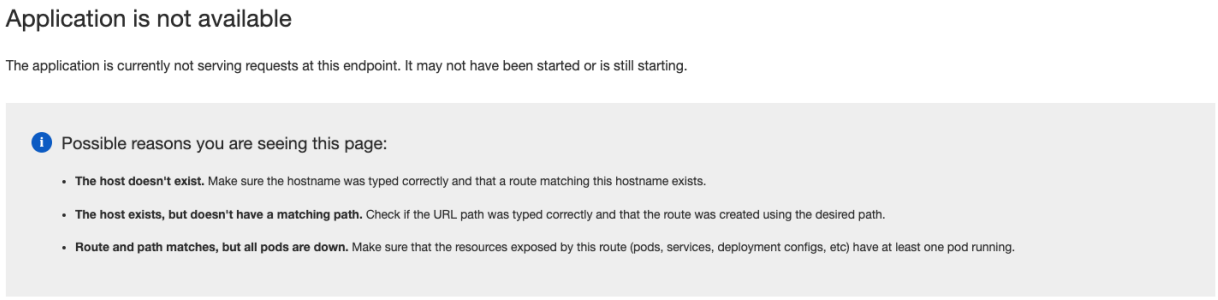


Figure 4-4: The application is not available.

I have learned to love and fear this page. In actuality, this is a formatted error page from the router. It is not an indication that your application is broken; it is an indication that the endpoint is not reachable. A quick rule of thumb: if you see this error page, it is coming from the router, which has received traffic for the route but the end `service` is not available. Knowing this will make debugging things much easier.

Scale the deployment back up to **1** for the next recipe.

Recipe 4:4: Distributing the traffic to multiple services

Just a quick one to show how easy this is. If you are not there already, switch to the Developer viewpoint. (Again, this is on Sandbox, but if you have your own cluster, it will behave the same way.)

Click **Add**. Select **All services**. Click **Builder Images**. Find the **Node.js Builder Image** tile and click it.

When the summary panel appears, click **Create**. Set the Git repo URL to:

```
https://github.com/utherp0/ocpnode
```

Set the application name (the cosmetic grouping) to `ocpnode-app` (this might be prefilled). Set the **Name** to `ocpnode`. Set the resource type to **Deployment**. Click **Create**.

While it is building, if you want a bit of cosmetic fun (which is actually very useful when you have massive deployments of multiple components and want to group them), click the **(D) http-example**. In the **Details** pane of the right-hand content, click the **Edit** link next to the Labels. When the dialog appears, add:

```
app.kubernetes.io/part-of=ocpnode-app
```

Now click **Save**. Watch the Topology. It will bring the deployment roundel for the `httpd-example` into a group with the `ocpnode-app`. Yeah, pointless, but I like keeping the Topology tidy.

By now, the build and deployment should have finished. Check that a service exists by either switching to the **Administrator** viewpoint, clicking **Networking** and then **Services**, or adding **Services** to your **Developer** viewpoint. (Use the search function, enter `Services` as the resource type, and click **Add to navigation**). You should see that `ocpnode` has a `service` now.

You can test the application itself by going back to the Topology and clicking the small icon at the top right of the `ocpnode` roundel. You should see a webpage. (Again, apologies; I have been using the same CSS for 20 years because I like it and am terrified of writing stylesheets.)

We have two distinct applications. Now, we are going to change the route of the `httpd-example` to point at both, with a 40/60 split in traffic.

To do this is simple. For ease of access, we are going to add the **Routes** content pane navigation into the **Developer** viewpoint. Use search, enter `route` in the **Resources** drop-down.

At this point, be careful. One of the amusing things about Kubernetes and OpenShift is a prevalence for reusing terms. So, when you search for `route`, you will see that in the Sandbox (and other clusters that have installed the Serverless Operator), there are two objects called `Route`. In actuality, they are distinct and different. The API listed below them indicates what owns the objects. Click the route object with the API `route.openshift.io/v1` below it. We will cover serverless routes later (and they are a lot of fun).

When the **Routes** content pane appears, click **Add to navigation** on the top right, which will add the link to the left-hand side navigation. Sometimes you might need to refresh the page. There is a timed update for the state of the navigation panel, and you might have to preempt it.

Once the **Routes** navigation appears, click it to list the route objects active in the project. You might get an error page that indicates you don't have access to see it. This is because the Search page defaults to a higher-level project (`ns`). Simply change the project in the drop-down at the top left of the content pane. You should see (at least) two routes: one for `mytestservice` (the original route for the `httpd-example`) and one for `ocpnode`. Click the `mytestservice` one to render the details of the route.

When the route content appears, click **Actions** and choose **Edit Route.** We are using the form. You could dive into YAML if that is more comfortable. After the form entries for the service (including service weight), there is a link labeled **Add alternate service**. Click it.

In the **Alternative Service target** drop-down, it will render all the services in your project except for the one currently selected for this route. Click **ocpnode**. In the form entries for the service, change the **Service weight** to `40`. Set the **Alternate Service weight** to `60`.

Click **Edit** to save. Now click **YAML** and look at the specification. It will be something like this:

```
to:
   kind: Service
   name: testservice1
   weight: 40
 alternateBackends:
   - kind: Service
     name: ocpnode
     weight: 60
```

The route is now defined to push 40% of initial connections to the original application and 60% to the new application. This is dynamic. By changing these values, the traffic routing is applied immediately.

However, this won't work. Amusingly, if you click the link to the route, you will get that wonderful router error page I talked about. This is because OpenShift Container Platform needs to secure the route to switch services. In the **Route** overview pane, click **Actions** and select **Edit Route** again.

Scroll down to **Security** and click the checkbox for **Secure Route**. In **TLS Termination**, select the **Edge** option. Leave everything else as is and click **Edit** to save the route.

The router will be happy now. If you go to the route, you will get either the original application or the new one. Remember that it applies sticky sessions, meaning that once you have visited the link in the browser, it will continue to go to the service it chose. Try firing up an incognito browser (or if you are a good old-fashioned Linux aficionado, have a play with `siege`).

Fine-grained access control to your doorbells

One last little bit on the whole services thing: OpenShift and Kubernetes allow you to directly control who can interact with services. Again, using an object (in this case, the `NetworkPolicy`), you can define rules as to the ingress and egress traffic for named services, which is incredibly powerful.

Put simply, you define network policies by stating rules and behavior. On the Sandbox, go to the **Administrator** viewpoint → **Networking** → **NetworkPolicies**. You will see a set of predefined policies that have been defined and instantiated for your project.

There should be one called `allow-same-namespace`. Click the link in the **NetworkPolicy** content pane to display this policy's details. Click the **YAML** tab and you will see the definition, which looks like this:

```
spec:
  podSelector: {}
  ingress:
    - from:
        - podSelector: {}
  policyTypes:
    - Ingress
```

It's a pretty vanilla definition. The specification applies to ingress and says that any pod (`podSelector: {}`) allows ingress from any pod (again, `podSelector: {}`). The `NetworkPolicy` is defined at the project level. Again, OpenShift projects derive from the Kubernetes namespace objects, so you will see references to namespace-level context.

There's a really nice interface for creating these objects, or YAML if you're that kind of way. The interface allows you to enter labels and values, and see exactly which pods in the namespace are going to be affected by the policy.

One little caveat: multiple network policies can get, well, a little confusing to debug and understand. The easiest way to get around this is to apply a blanket `deny-all` policy first, effectively isolating all traffic, and then craft access.

CHAPTER 5

What (and how) is my app doing?

I spent a lot of my development career writing `System.out.println( "[DEBUG]" );` statements. I'm not proud. So when developing nowadays, I tend to be a bit of a verbose developer, outputting anything and everything my application is doing for the inevitable point at which I have to fix or debug it.

With OpenShift and Kubernetes it started off being a little tricky to debug your stuff; when thinking of the two main ways you look at your application, functional and nonfunctional, in terms of the points of behavior, Kubernetes, and therefore OpenShift, could be a bit, well, oblique in the way they reported issues.

For example, because of the "desired state" nature of the system, you tend to get an abstracted view of objects. Objects that are not in their desired state report just that, not the core reason why.

A good example of this is using a persistent volume (which we will talk about in a later chapter). Let's say you have a deployment that requires a pod with a connected piece of storage. The deployment will not report itself as ready until the pods are healthy. The pods will not report themselves as ready until the storage is healthy. If the storage fails or gets stuck waiting, the other dependent components sit patiently in an "unhealthy" or "not ready" state without telling you why. By design, this is what they should do, but it's unhelpful for a developer.

With this in mind, OpenShift was designed with a three-tier approach to observability. The combination of all three makes for a very nice, fine-grained reporting mechanism. In addition, OpenShift allows you to change the state of objects based on their real-time telemetry, which is great for Ops (but not really of interest to developers).

We will look at the three distinct tiers from a pure developer viewpoint. There are a lot of nuances around the way these work, and most of them are "Ops-y" at best. But knowing what is what and where is a key plus for developers trying to figure out what the hell is going on.

The three tier observability approach

From a top-down perspective, OpenShift offers a distinct three-tier approach. These are:

- **Events:** Changes in the Kubernetes object state.
- **Metrics:** Real-time indication of system resource consumption.
- **Logs:** Individual and aggregated log output from the actual workloads.

Events, or what are the objects doing?

We have covered the fact that everything (and I mean everything) in OpenShift/Kubernetes is based around the object model. Put simply, OpenShift maintains a "desired-state" database or "brain" of object types and specifications and realizes the components that have a physical part on the worker nodes themselves.

To keep track of state changes, the system has the concept of events. These are fired every time a state changes. As you can imagine, there are a lot of these. To keep the system unloaded, these events are designed to be ephemeral; after an hour, they are deleted from the record. If you have a consistent problem, such as a pod that won't start, the event remains because it is refired every time the system tries to create it and fails. Hence, if you have something serious (as opposed to, say, a race condition), the event will remain firing until the issue is fixed.

As with all things, these events are reachable as objects. Using `oc`, you can get a list of the latest ones. If you are on the Developer Sandbox, you can fire up the OpenShift terminal (click the terminal prompt icon at the top right of the interface). Or, if you have `oc` installed locally, you can log in to your target cluster (get the login command from the web UX by clicking your logged-in name in the top right). Once you have an `oc` session, try:

```
oc get events
```

Most of the events you see will be uninteresting, but when you are developing and your workload won't start, events are your first point of call.

The only "gotcha" to be aware of is that OpenShift and Kubernetes objects only have context of themselves. In English, this means you always see the "effect" rather than the "cause" unless the object is the core point of the problem.

For example, you're doing a deployment and the pod won't start. If you get the events, it will say "Pulling image xxxx" and pretty much nothing else (normally). If you look at the **Events** for the project, you will find that some other object (normally an image stream or, if you are using persistent volumes, the PVC) is throwing an event indicating a failure. The deployment simply hangs, waiting for the component. This is both a strength and a weakness of the Kubernetes approach, where everything is effectively abstracted from everything else, and debugging can become an art.

Also, if you cast your mind back to the section where we talked about changing the contents of a config map and not seeing a change in the deployment, you need to remember that events are object-bound. This means the event you see might be a side effect of a side effect (of a side effect; you get my point). I always talk about root cause analysis with customers. Developers are like everyone; we will see a problem and immediately point at it. Take a breath, look at the objects the object depends on, and see if they are having issues.

In terms of real-life experience, the majority of events that a developer is interested in come down to "Why isn't my image pulling?" (your pod won't start and will sit there with a light blue roundel looking sad) and "Why isn't my persistent volume mounting?" Keep an eye out for those two, and you'll be able to debug the issue quickly.

Metrics, or how much of the system is my app eating?

Metrics in OpenShift are an interesting and very useful concept. The nature of the system, because of its tight integration with the underlying operating system, means that there is full visibility of the impact of components on the infrastructure.

In the second tier of observability, OpenShift provides and mandates a fixed set of real-time metric streams for all components in the system that consume them—not just the pods and containers, but the nodes themselves.

It does this using Prometheus, and the four standard streams of information are:

- CPU usage
- Memory usage
- Network I/O

- File system I/O

As with all things in OpenShift, the UX provides a drill-down visualization of these, along with a set of useful predefined dashboards. This information, and the dashboards, are access controlled (again, as with everything in OpenShift) using a fine-grained security model. We'll talk about this model (and it's a very clever setup) in depth in Chapter 10.

With the drill-down approach, you can get aggregated metrics for all your objects (the ones you have rights to interact with) that consume resources and drill down into individual streams for individual objects.

Think of this tier as "What is my workload doing?"

This is fine for workloads that are consuming resources, and you can actually use these resources to drive scaling behaviors. You can use autoscalers to change the number of replicas of a workload based on its aggregate resource consumption.

For instance, you can specify a workload to be between 2 and 20 replicas, with a target CPU consumption of 75% of its allocated resources. (You can hard-control the resources available to objects using an approach called `limit-ranges`. In fact, you have to have `limit-ranges` for the autoscaler to work as it uses the figures defined for upper range and request range in the aggregate calculation). With this autoscaler, the system will initially deploy two replicas, then monitor the aggregated `CPU`. If the aggregate is above the target consumption, it will create more replicas (up to 20) until the aggregate comes down. If the aggregate is below the target, it will remove replicas until the target is achieved.

Currently, these autoscalers support the use of the `CPU` and `Memory` streams. When we talk about serverless in Chapter 9, we will introduce the concept of autoscaling based on the network stream (it does autoscaling on concurrent connections and response times).

If you like messing about with the `Metrics` (and as an ex-data scientist, I do), you can explore the metrics for any resource-consuming object you have access to.

Recipe 5:1: Peeking at the stuff your application is consuming

First, log in to the Sandbox (or your own OpenShift cluster if you're lucky enough to have one). We will install our trusty `ocpnode` application using Recipe 3:0. Once it has completed and is healthy, walk away for five minutes (go doomscrolling to stop yourself from feeling happy, or if you are not a masochist, grab a good coffee/tea/whatever your poison is). Not to be glib, we need the cluster to process some metrics. The collection of metrics is done in real

time, but the aggregation can take a while to process. If we want to see an application in action, we need to let it run for a bit. Before you walk away, go to the **Developer** viewpoint, click the roundel for the `ocpnode` application, click **Details** for the `ocpnode` deployment in the right-hand content pane, and scale the application up to **2** pods. This is so we can see the metrics of the individual instances.

When you come back from your coffee/tea/doomscrolling sadness, look to right-hand content panel. If it is not up, click the roundel for the ocpnode application. There is a tab labeled **Observe**.

This will give you drop-downs for various observable metrics for the deployment itself. It's a drill-down. Make sure the metrics information is visible. The first graph shows CPU usage and will only have the metrics for the active pods for this deployment displayed. The same is true for the memory usage and receive bandwidth.

Now for the cool bit: hang the cursor directly over the graph, hold down the mouse button, drag a little across the graph, and let go. It will zoom into that time period for the metrics.

Logs, or what is System.out.println telling me?

Ah, the developer's best friend. I spent way too much of my development time hooking my own logging in (cough, `System.out.println( "[DEBUG] I'm here now" );`) and interestingly, never got on with `log4j`—if you know, you know.

Logs are essential, not only in development but when applications are running in anger. The way that Kubernetes/OpenShift works caused an initial interesting problem with the logging system. Let's say you have an application called `myApp` running with one replica. The cluster can pull whatever the application pushes to the standard outputs and store it against the application name. You can then trawl the stored logs for the cluster and find stuff using the name as the key.

Simple. Or it would be, until you get replicas. Or replace the application. In the case of replicas, the then-standard log system ELK (Elasticsearch, Logstash, Kibana) wouldn't be able to determine *which* of the instances the logs were attached to. Or, in the case of an application just plain crashing out and being restarted, which *instance* of the application actually threw the log message.

A quick overview on how these kinds of systems work: a lifetime of working with and writing search engines makes me a little over-interested in these mechanics, but it makes a lot of

sense when talking about what OpenShift does. When a log is grabbed (indexed), the logging system generates a set of searchable metadata fields to store along with a cleansed version of the log (and a raw version if necessary). These metadata fields make searching much more accurate and fast; they extend the identification information for the individual record.

In the case of Logstash, the key metadata field was the application name. In the case of a multi-instance system like OpenShift, this isn't enough to uniquely identify an application. To counter this, OpenShift started to use what was called EFK (Elasticsearch, Fluentd, Kibana). Fluentd was a configurable log scraper that could add specific Kubernetes information to the individual logs. In this case, it was the cluster the application was on, the node on which the pod was running, the namespace the application was running in, and the pod name for the instance generating the log.

This was brilliant. It retained the original information (for example, the application name) but could now search and filter on the Kubernetes components as well. Case in point: if you have done a search for, say, `FATAL`, and you get two records back, those records contain the direct information you need to identify which pod, which namespace, which node, and which cluster.

This was great. And now it has changed. Sorry, couldn't resist.

So this was fine. The only mild "fly in the ointment" was that although you could see the logs from an application in real time (we will show this in a second), the aggregated logs were refreshed on a timer. In terms of archiving and searching aggregated logs forensically, this was fine, but it wasn't perfect. If your application imploded at the point just after the timer had fired, you would have to wait until the logs were aggregated to find it in a search (as opposed to looking directly into the logs).

Also, log storage got big fast. The main cause of node issues on a production cluster was almost always a lack of disk space. Ops always want to cull logs, and developers want to keep them.

The log system was completely re-engineered in OpenShift to use a technology called Loki. Loki acts as an interim, real-time logging system. It indexes a minimum set of fields in real time and defers the aggregation of the full logs until later. This works perfectly. Loki does a real-time aggregation because the number of fields it needs to store are much smaller than the "full-fat" logs that are archived or aggregated in slow time. It's the best of both worlds.

Recipe 5:2: What are you doing, app?

So, logs are generated for the applications themselves. Think of the lowest atomic component of visibility. In Kubernetes, the smallest deployable is the pod, but pods can have multiple containers. OpenShift gives direct visibility of the logs from any active container.

Pop back to **Workloads/Topology**. We will use the pods we created in Recipe 5:1 for this example. Click the label **(D) ocpnode** to get the right-hand content pane up (if it is not there). Now click the **Resources** tab.

You will instantly see the list of the pods running in this deployment. On the far right of each entry is a helpfully named link point that says **View logs**. Click the first pod's **View logs** link.

This takes you to the pod content pane for the appropriate pod and opens the **Log** dialog. This is a very powerful tool, especially if, like me, your applications spew logs like it is going out of fashion.

The important points here are as follows: you have the ability to pause the log output (hugely important) using the pause button on the left-hand side. Next to the information label with the state of the displayed logs (streaming/paused) is a drop-down. This contains the container name that is generating the logs being displayed from the current pod.

In the case of the Developer Sandbox, logging is provided using Loki, but it doesn't have a long-term persistence logging system set up. In the case that your cluster is "offlining" the logs, the drop-down (which on Sandbox has `Current log`) would allow you to switch between the live logs and forensic logs. In the case of the Sandbox, you will only see `Current log`.

Finally, there is a free-text Search box. Try typing `808` in this to see how the search works on the displayed log. Note that, helpfully, the log switches to paused when you search.

These log features are reachable by drilling down through the pod content. You can go directly to **Workloads/Pods**.

CHAPTER 6

State of state, persisting in an ephemeral world

Containers are ephemeral beasts. By design, when you instantiate one, it is created from a layered set of file "differences" applied to a file system, with an overlay on top to allow the container to change its contents. That's fine and dandy: the container starts, you interact with it, files change, behavior changes, and then the container ends. When the container goes away, the overlay file system is lost. When the container is restarted, the file system components not defined in the image are reset.

The first time I ever played with containers, back in the Docker days, I made this mistake. I had written a quick Tomcat (best dynamic web server ever, and JSP was my favorite tech for years) application, created the image, and deployed it as a container. And it wouldn't work. I couldn't serve the page because I hadn't set the security context correctly. So, in my standard bull-in-a-china-shop way, I shelled into the running container, changed the security context files, restarted Tomcat, and *et voilà*, the application served correctly.

Now, because this was OpenShift (3 at the time), the application was running as a pod. As I celebrated the success of the application by going off for a quick toilet break, the application crashed for some reason. OpenShift noted the health probe was reporting unhealthy and, as it should, recreated the pod. I came back, having washed my hands of course, and started to show my colleagues how cool the app was, and boom, nothing worked. The changes I had made were to the ephemeral overlay file system and had been discarded when the container was recreated.

Whoops.

Introducing persistent volumes

This limitation would have made the use of containers very much a reduced set of use cases. So the Kubernetes project introduced the concept of persistent volumes, which always drives me nuts because I have a mental block on what they are called ("persisted"? "persistent"? argh). Anyway, the idea behind PVs is brilliant. The Kubernetes engine mounts a file system mount point into the container as part of container creation. This mount point can be one of many implementations, such as read-only file systems, files created from config maps and secrets, local storage, or shared storage exported to multiple worker nodes.

What this means in English is that the mounted file system is part of the container file system. It can be written to (when it is not read-only, of course) and persists past the destruction of the container. So the application can write transactional and persisting data and continue when it is recreated. But it is more than that. The abstraction of the storage means that one container can write to the file system, expire, and a different container can mount to the storage and consume it.

This leads to many really cool use cases for data persistence, transactions, and even hosting of the core data engines, like databases, data grids, etc.

The mechanics of PVs

When you start to dive into the way these PVs are surfaced and offered within OpenShift and Kubernetes, the elegant design starts to shine through. The persistent volumes themselves are chunks of storage that are defined cluster-wide. In fact, in the majority of recent storage implementations, they don't actually exist. This is a historical thing. When OpenShift 3 first came out, the practice was to define a number of pre-built PVs, often using `NFS`. So you would have to predefine the size of these individual chunks and make the storage available up front as PVs for pods to consume.

We'll talk about how these storage components are assigned to projects in a moment (using persistent volume claims), but for now, assume that PVs are still physical. There's another object defined as part of the storage components called the `StorageClass`. Every PV is served by a mechanism defined in the storage class. For example, if you are using storage on the Sandbox, which is hosted on AWS, the storage classes represent the mechanisms available to the cluster on AWS for provisioning and using storage (that is, `gp2`).

A very important part of these, which was one of the things that came from user experience, is what we call the "retention" policy for the PV, as defined by the storage class. Put simply, this tells the engine what to do with the physical storage once either the PV is destroyed or there are no longer any persistent volume claims that have locked the storage to a project. This policy is either `delete` or `retain`.

Why is this important? It's not so much important as it is essential. Let's say you have an application that has critical customer-sensitive information. If you define or consume a storage class with the retention policy of `retain`, then when the PV is free (that is, no longer linked to a project), the *file system contents remain in the storage*. This means another project could request access to the PV and then find (and consume) the files within. Setting the policy to `delete` (now!) performs a zero-set wipe of the storage, so nothing remains or can be forensically fetched.

How an application consumes a PV is by using a project-bound object called a persistent volume claim. Since these were created, they have become much more technically complex. In the original implementation, the PVC was just a lock to claim ownership of existing PVs. Now, with dynamic PV provision (not having to pre-create the PVs), PVCs of certain storage implementations (and driven by `StorageClass`) create the storage as part of the claim creation. Again, retention policies remain in place, so you can use one application to attach a PVC, write some data, remove the application, attach another different application through the PVC, and read the data.

An interesting "gotcha" on this is that the majority of times I've had a pod fail to start on OpenShift, it has been because of the inability to connect to a PV/PVC. But because of the way that events in the system are linked to objects, the cause of the failure is obfuscated. This means the pod says "trying to start!" and doesn't, while the PVC or PV has events such as “unable to obtain storage.” This just means you have to be aware of the root cause by looking at other objects. But, welcome to Kubernetes on that one.

At the end of this chapter, we will have a number of recipes for a quick play with the various mechanisms around PVs and PVCs. But first, we need to talk about the physical types of PV/PVCs, as they are critical to the behavior of applications.

The perils of RWX versus RWO

So, when you mount a PVC, you get an option that allows you (within the options provided by the back-end storage, `StorageClass`, or Kubernetes CSI driver) to define the behavior of the mounted volume. This gets a little confusing, so bear with me.

There are three modes of mounting a PV/PVC into a deployment or pod:

- **ReadOnly**: This does what it says on the tin. The storage is mounted as a given mount point (`/uth/myreferencedata`), and the application can read files from that mount but not write to them.
- **ReadWriteOnce** (RWO): A confusingly named mode, in my opinion. This describes a mechanism that maps the storage into one deployment or pod. Any replica of the pod will not start as the storage has a single usable mount.
- **ReadWriteMany** (RWX): The coolest storage mechanism. This allows any mounted deployment or pod to write and read into the same shared storage across the cluster. So, effectively, two replicas of an application, deployed to the same cluster, have a single shared point of storage.

It comes down to the underlying storage itself. Kubernetes/OpenShift cleverly abstracts infrastructure and machine interactions through the CXI APIs, including storage (CSI), networking (CNI), and pod runtime (CRI). Provision of `ReadOnly`, RWO, and RWX modes are done optionally through the CSI, meaning some back-end storage drivers will support all the modes, and some won't.

It takes a while to sink in (and believe me, I've had years of tripping myself up over PV modes), but it does make sense if you think of how the storage is physically connected to the worker nodes.

At the end of the day, a running container (in a pod) is a process-bound file system. This file system can contain mounts that are external to the image definition (which we talked about in Chapter 3). PVs and PVCs are the technology for hosting and persisting the storage away from the image and container.

So let's have a play and see how it works in reality.

To keep the examples simple, we will be using a basic application but playing with the storage via the terminal. This way, you will see exactly what happens at the PV/PVC without the confusion of the application. Remember that the persistent storage, in this case, is just a file

system. You can also interact with block storage in the same physical way (for example, S3 buckets via AWS).

Repeat Recipe 1:3 before you start to create the `openshiftbook-app`. Once the application has been built and installed, you will have a single running instance (of the world's worst hello-world style app, but at least it's simple).

Recipe 6:1: Adding some storage to a basic application

On the left-hand navigation, click **Storage** to expand the menu. You should see four sub-screens: **PersistentVolumeClaims**, **StorageClasses**, **VolumeSnapshots**, and **VolumeSnapshotClasses**.

You won't see PVs. This is because the level of access for a user on the Sandbox is restricted. You can still create PVCs (which are bound to your project space) as you are an administrator of your project. PVs are cluster-wide, which is why you cannot see them. (Interestingly, you can prove this by going to **Home → Search** and entering `PV` in the resources box. You will see the entry for **PersistentVolumes**, but when you select it, it will tell you that you do not have access to that resource. If you are using your own OpenShift Container Platform or an organization's OpenShift where you have access, you will be able to see and interact with the PVs).

Click **StorageClasses**. This will display all the mechanisms you can use to create PVCs. In the case of the Sandbox, these are the ones consumable on AWS. If you now click **gp3** and then, in the right-hand content panel, click **YAML**, you will see some basic definitions of the behavior of the storage provided by this class.

Notably, a reclaim policy of `delete` (see earlier), an ability to expand the volume post-provision, and something called `volumeBindingMode`, which means in the case of `gp3`, that when the PVC is created, the storage is not created until the PVC is physically bound into a deployment or pod. Also, note that the volumes are encrypted by default and provided, unsurprisingly, by the `gp3` type. These two aspects are defined in the parameters of the storage class.

So, we are going to create a PVC before we attach it to our application. The PVC is a local object to the project, meaning it is owned by the project in which you are working. Once created, it can be attached and shared to any application, deployment or pod which is also in the project. As mentioned before, a project provides a contextual and "ring-fenced" set of objects.

Click **Storage → PersistentVolumeClaims** in the navigation on the left. You will get the PVC content page appearing in the right-hand panel. If you haven't done anything other than create the example application, this will be empty. Otherwise, it might have some PVCs in there that are being used by other applications in the project.

Now click **Create PersistentVolumeClaim** in the top right of the content panel. Then select **With Form**. You can create a PVC directly from a file as well. This is used extensively with OpenShift Virtualization, where you can upload a full VM image directly into a PVC and create a machine from it, which is cool. We will talk about this in Chapter 11.

As with all objects in OpenShift, you can directly edit the YAML (if you are suitably masochistic; I'm joking). In this case, we will use the form shown in Figure 6-1, which abstracts the most important components for us.

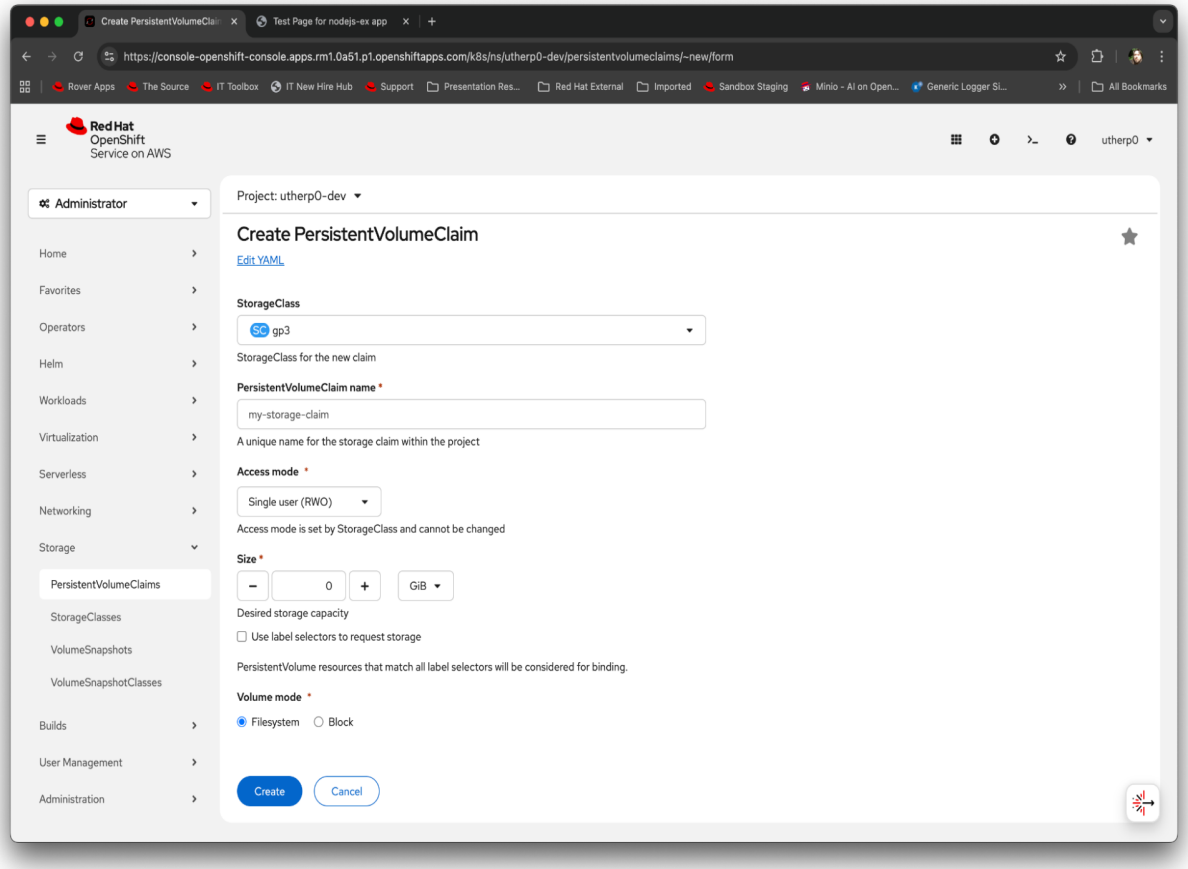


Figure 6-1: Form for creating a persistent volume claim.

Things to note: the **StorageClass** drop-down gives you all the classes you can consume. In this case, leave it on the default, which should be `gp3`.

Set the PVC name to `examplepvc1`.

Click the **Access mode**. Note that you can't change it, as the storage class implicitly defines what modes are supported (as we discussed earlier). Also, note that in this case, it only offers RWO (which it calls *single user*) but also lists ROX (read-only), RWX (shared access, also "read-write-many"), and a newer mode called RWOP (read-write-once-pod). This was added due to unpredictable behavior with RWO in the old days, as some drivers maintained single access for RWO on every worker node. This meant you could have multiple replicas running with different PVCs. What RWOP gives you is a fixed single instance of the storage, available to only one pod across all the worker nodes.

Set the size to **1 GB**.

Leave the **Use Labels to request storage** box unticked. This allows you to predefine the objects that can access this storage (for example, setting a label on your deployment such as `storagepvc=thebestone`, and then setting the same label information on the PVC, which is enabled if you tick this box).

The **Volume Mode** radio box allows you to switch between file and block. We will stick with file for simplicity's sake.

Click **Create** to instantiate the PVC. If successful, you will be taken to an overview content page that lists the configuration for the PVC. Note that the status, shown to the right of the PVC name, is Pending. The `gp3` storage class does not provision the actual storage (it creates a PV at the cluster level that you cannot see based on your requirements) until a request to consume the PVC is made.

To see a simpler synopsis of this, click the **PersistentVolumeClaims** navigation link again. The new PVC will be listed on the overview page with a status of Pending and a persistent volume of No Persistent Volume. The system hasn't received any requests, so it hasn't provisioned the volume yet.

So now we have a PVC that is owned by the project. Switch back to **Workloads/Topology** and click the **Deployment** (for `openshiftbook`). In the right-hand overview panel, click **(D) openshiftbook**. You will now have a content page for the active deployment. Click the **Actions** drop-down at the top right of the panel.

Click **Add Storage**. This will display the content page for adding storage to the deployment.

This form allows you to choose a PVC and map it into a mount point. Note that you get a drop-down with the PVCs available (to you) in the project, a mount path (the mount it will

appear within the application container), and also an optional subpath which locks the root of the storage (that is., subdirectories within the shared storage).

Click the drop-down and select `examplepvc1`.

Set the mount path to `/openshiftbook`.

When we click **Save**, it will change the state of the deployment. It will take you back to the content page for the deployment and show you the pod being updated. Once the pod has updated (the count will go back to 1), click **Storage** and then **PersistentVolumeClaims**, and the list of PVCs should now contain `examplepvc1` with a status of **Bound** and a PV name (confusingly starting with `pvc-`, the downsides of dynamic naming).

Now we will check to see if the storage is available to the application. Switch back to the **Developer** viewpoint, click the deployment (`openshiftbook`) to get the right-hand content panel up. In this panel, click the name of the active pod (listed in the **Pods** subsection).

When the pod content page appears, click the **Terminal** tab. Note that if you leave this tab without typing anything, the system will disconnect you after a brief timeout. You can re-establish the connection by refreshing the tab.

In the terminal, enter:

```
df -h
```

This lists all the mounts. You should see a mount specifically for `/openshiftbook`. Now enter:

```
cd /openshiftbook
echo "Hello World" >> test.txt
cat test.txt
ls -al
```

We have created a file in the PV itself. Now we are going to remove the pod. Click **Workloads/Topology**, click the deployment to get the right-hand content pane up, and click the deployment name to get the **Deployment** content page up. Select the **Details** tab. On this page, you have a graphical representation of the deployment (the roundel). Next to this, you have the scale-up and scale-down arrows. Scale the number of replicas down to **0**.

At this point, we have no container running. We will now scale up, which entails the system recreating the application from the image (which doesn't contain our `test.txt` file). Click the up arrow and set the replicas to **1**.

Once the roundel switches to blue and indicates one pod is active, click the **Resources** tab (on the content page; there are multiple ways to get to the content panes), click the pod, and then open the **Terminal** tab. Now enter `cat /openshiftbook/test.txt`.

```
cat /openshiftbook/test.txt
```

The test file has survived the pod being removed and recreated. This is the crux and power of persistent volumes.

As a little test, pop back to the **Workloads/Topology**, and on the **Deployment** content pane under **Details**, scale the deployment up to **2** replicas. You'll spot something interesting at this point: the deployment will stick at one replica, and the second replica will never start up. This is because the deployment needs to attach the mount, and the PVC is defined as RWO. This means the first pod "owns" the mount, and the second pod can't start without it.

Recipe 6:2: Moving storage from app to app

So, we have the world's worst application running now (I'm not proud), with attached storage. One of the cool things you can do with persistent storage is shift it between applications.

Create another instance of the application using the **(+)** button at the top right of the interface. Use the same Git repo as in Recipe 1:3, but set the name to `openshiftbook2`. (Remember to add the subdirectory `/chapter01` or the **Create** button will remain greyed out). For ease of visibility, make sure the application name (the grouping) is the same for both deployments.

Now delete the first application. Click the **(D) openshiftbook** label beneath the roundel on the left. In the content pane on the right, select **Actions** and **Delete Deployment**. Leave the checkboxes set (when it prompts you to delete child resources) and click **Delete**. The roundel will go dark blue (indicating the resource is being deleted) and then disappear, leaving just the roundel for the second application.

Select **Storage/PersistentVolumeClaims**. Firstly, you should see that the `examplepvc1` is still bound (it exists) but is no longer used.

Now that we have seen that the PVC still exists, we are going to attach it at a different mount point to the new application. Click **Workloads/Topology** and, if the right-hand content pane is not visible, click the **(D) openshiftbook2** label beneath the roundel. When the content pane shows the **Deployment**, click the **Actions** drop-down and select **Add storage**.

In the **Add storage** dialog, use the drop-down for **Use existing claim** to select **examplepvc1**.

Set the **Mount path** to:

```
/notthesamedirectory
```

Now click **Save**. Because the deployment specification has been changed, OpenShift will act immediately to reconcile it. You will see the transition on the **Topology** page.

Once the roundel has reset and is blue (and by itself) again, click the **Pods** tab in the content pane. Click the pod label to go to the pod content pane.

In the pod content pane, select the **Terminal** tab and enter (in the terminal):

```
df -h
cd /notthesamedirectory
cat test.txt
```

Et voilà, we have attached the storage to a (hopefully in the real world) different application.

So that's the basics of persistent storage. In real use cases, this is immensely powerful, especially with the serverless technology we will talk about later, where you can engineer ephemeral services that act as digital sausage machines, with transactional information and state stored in persistent volumes.

I think it's cool, but then again, I grew up with Fortran and Pascal, so everything is cool to me. As long as I don't have to use tab formatting (if you know, you know).

CHAPTER 7

Pass the tray, Jenkins

I love Tekton. Not just the tech; the logo is the coolest one I've ever seen.

Anyway, I have a long history with those two butlers, Hudson and Jenkins. For those who don't know, Hudson was the original pipeline tool (and it had a fantastic feature where it would "blamestorm" someone for breaking the build pipeline). Many a morning, I went into work and my email alias was blinking on the overview screen as public enemy number one. Jenkins came about when Hudson was acquired, but they both did the same thing: automating processes in a consistent and repeatable fashion.

A tiny history lesson before we get going on Tekton: OpenShift 3 was tightly integrated with Jenkins. We had extensions that allowed you to call back into the OpenShift API from within Jenkins build scripts, and OpenShift could call out to Jenkins to execute builds. The problem was that it was a handoff. Once OpenShift had called out to Jenkins, all it could do was wait for the pipeline to complete, successfully or not.

Introduction to Tekton

And then Tekton came along. Tekton is a different way of implementing internal automation in Kubernetes; it is incredibly simple in design but extensible enough to be a superb workhorse.

The design of Tekton is brilliant. You have definitions ("cookie cutters") for the atomic processes within a pipeline, called Tasks, and orchestration of these tasks using Pipelines. Tasks consist of one or more Steps, which are commands executed in containers using selectable images (basically anything you can containerize). If any of these sequential Steps "fail" (return anything other than a 0 return code), the Task stops and deems itself to have failed.

Pipelines are simply an organization of the execution of Tasks—in series, parallel, or both. If any Task fails, the pipeline is deemed to have failed (although you can have a "finalizer" Task

that is called regardless of whether the Tasks have succeeded or not, providing a transactional cleanup).

But the key thing that makes Tekton so cool is that while Tasks and Pipelines define the behavior, the actual execution is done through what are called `xxxRun` objects. These are individual persisting objects that contain the configuration for a Task or a Pipeline and the result of executing it. When a Task is run, for example, a `TaskRun` object is created which contains all the contextual parameters for the Task, and then stores the success, failure, or skip for each of the Steps executed. The `PipelineRun` object contains the pipeline level context and parameters for the individual execution of the Pipeline, along with the result of the pipeline and information on the `TaskRuns` used during it.

The `xxxRuns` form a historical audit of the temporal instances of the tasks and pipelines.

So, a quick overview before we dive deep: Pipelines are definitions of one or more Tasks that are run in a given order. If any of the Tasks fail, the Pipeline is deemed to have failed. A Task defines which sequential Steps are executed. Each Step defines the image to be used and the command or script to execute.

When a Pipeline is needed, a `PipelineRun` object is created that contains the parameters to use during the execution of the Pipeline and, after the execution, stores the results of each Task executed. For each Task executed, a `TaskRun` object is created that contains the configuration for the individual run of the Task and the result of each of the Steps executed during the run.

I like this approach. It feels like Lego. And most developers love Lego.

The cookie-cutters: Tasks and Pipelines

Again, put simply, a Pipeline object defines the Tasks to be executed, the order in which they will be executed, and definitions of shared storage spaces. The fact that the Pipeline is a "cookie cutter" means it says what internally named storage components it will offer, which are then sub-named by the tasks. You don't define actual storage or mount points until you execute the Pipeline.

It does this via a nice simple way of definition in the declarative YAML. Here's an example:

```
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: ubifilepipeline
```

```
spec:
  tasks:
    - name: ubifilecreate
      taskRef:
        kind: Task
        name: ubifilecreate
      workspaces:
        - name: working
          workspace: workingarea
    - name: ubifilecheck
      runAfter:
        - ubifilecreate
      taskRef:
        kind: Task
        name: ubifilecheck
      workspaces:
        - name: working
          workspace: workingarea
    - name: ubifileremove
      runAfter:
        - ubifilecheck
      taskRef:
        kind: Task
        name: ubifileremove
      workspaces:
        - name: working
          workspace: workingarea
  workspaces:
    - name: workingarea
```

Firstly, you get the standard "what object do I want" (`apiVersion` and `kind` fields), then some metadata relating to the object instance (in this case, just the identifying name), and then the good stuff: the `spec`(ification) of the Pipeline itself.

Each of the atomic tasks (more in a moment) can be defined in any order. The Pipeline uses an optional declaration of `runAfter` to indicate a Task that has to have completed successfully before the Task being defined is executed. A Task definition in the Pipeline can have zero or more `runAfter` definitions. This allows for some serious complexity to be created using a simple mechanism. Figure 7-1 shows an example.

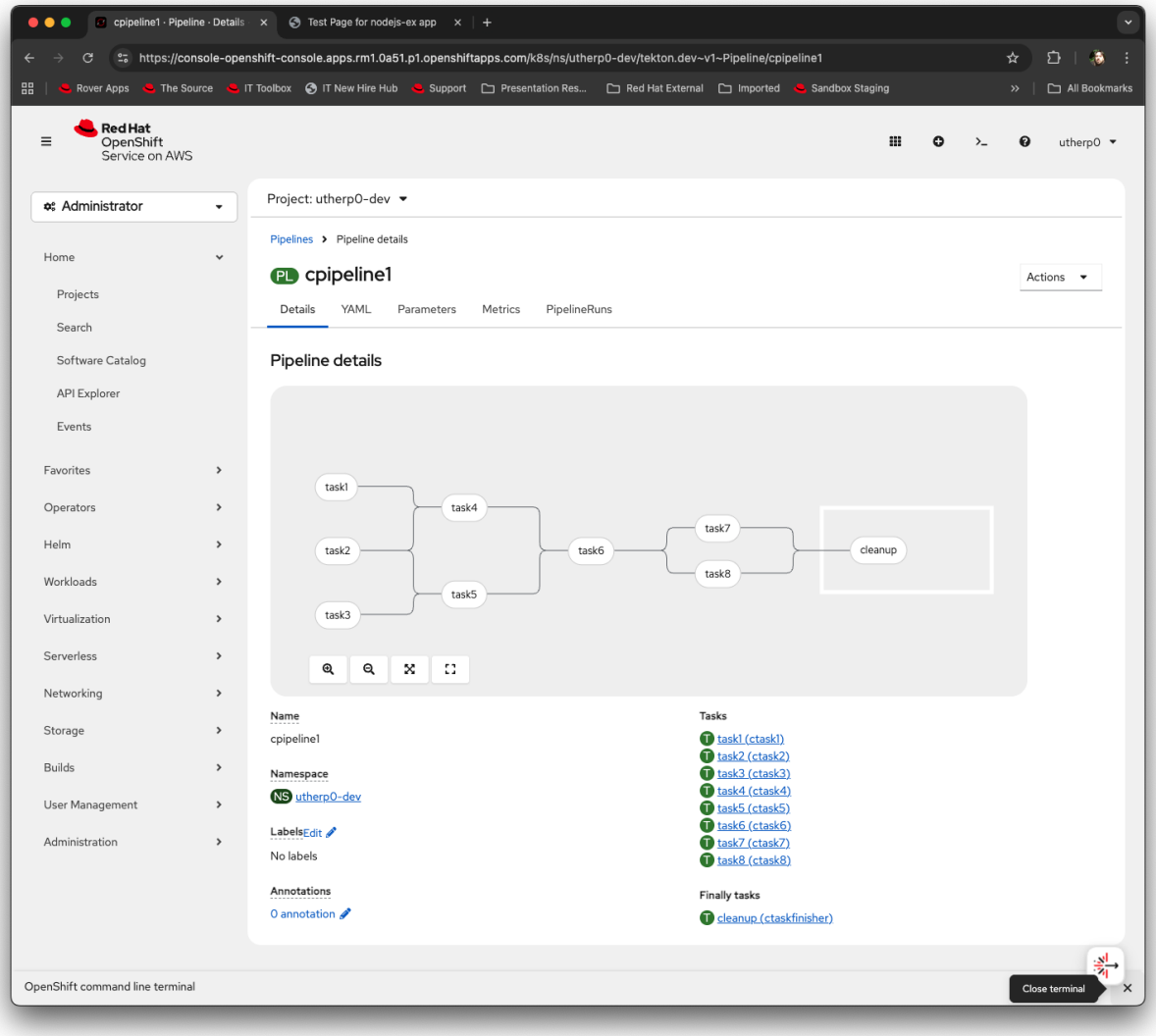


Figure 7-1: A complex Pipeline created using simple `runAfter` tags.

The nice thing is that using Tasks as atomic elements means you have full control over what runs when. Also (and we'll dive into them in a couple of sections), workspaces allow you to share file systems between the Tasks.

So, if Pipelines orchestrate the execution of the Tasks, then what do the Tasks do? Tasks define the Steps that must be successfully executed for the Task to complete successfully. Again, like Pipelines, tasks are "cookie cutters"—the definition of the Steps—and neither Pipelines nor Tasks are ever actually executed (we'll cover that next).

Tasks consist of one or more *Steps*, where a Step is the execution of a command or a script in a given image. And that's where the power of this approach is: anything you can stick in an image can be used in a Step of a Task.

Here's an example of a Task definition:

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: ubifilecheck
spec:
  steps:
    - name: id
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - cat
      args:
        - /etc/redhat-release
    - name: echo
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - echo
      args:
        - "Checking the file has been created in the workspace"
    - name: checkfile
      image: registry.access.redhat.com/ubi8/ubi:latest
      script: |
        #!/usr/bin/env bash
        ls -alZ /uth/test.txt
  workspaces:
    - name: working
      mountPath: /uth
```

Again, this, like the Pipeline definition, is a declarative definition of an object. In this case, `kind: Task`. The metadata relates to the instance of the object in the cluster. In this case, it will appear as a `Task` object with the identifying name `ubifilecheck`. It has three defined Steps. These are executed in sequential order depending on where they appear in the definition, so this Task will generate three Steps, named `id`, `echo`, and `checkfile`.

Each Step has a defined image that will be used, a command, and arguments for that command. Like most of these definitions, you can cheat a little and use `script: |` followed by a non-YAML set of script commands. Optional components for Tasks allow you to define and populate parameters across all the Steps (using the `params:` field in a specification, which exposes named variables to all the Steps).

So, I'm aware that there was a lot of tech-speak, so here's an easy definition of the behavior and relationships between Tasks and Pipelines.

Tasks define one or more Steps. The Steps are executed sequentially, and if any of the Steps fail, the Task is deemed to have failed.

Pipelines define the order in which Tasks are executed. Like Steps in Tasks, if any of the Tasks reports failure, the Pipeline has failed. Unlike Steps in Tasks, you can have a "finally" Task in a Pipeline that runs at the end regardless of whether the Pipeline has stopped execution of Tasks due to failure.

Think of Pipelines as hooking tasks together. Think of Tasks as atomic sets of successful operations.

Again, as with all things in OpenShift, Tasks and Pipelines are native Kubernetes objects, and as such, are defined declaratively.

Tasks and ClusterTasks

The definition for a Task is reasonably simple. Let's take a look at an example:

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: task1
spec:
  steps:
    - name: id
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - cat
      args:
        - /etc/redhat-release
    - name: echo
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - echo
      args:
        - "hello world
```

The thing to remember with Tasks and Pipelines is that they are "cookie cutters." They are the definition of what to do, not an actual instantiation—we will talk about those in the next section.

Firstly, be aware of the version. Even though Tekton (the open source project that OpenShift Pipelines derives from) is reasonably mature, the API is still in its early days, at least at the time of writing. The `apiVersion` is set to `v1beta1`. As with all objects, the `metadata` defines information for the object instance, in this case, just the Task name.

What we are interested in is the specification. In the case of a Task, this consists of the definitions for each of the Steps, executed in the order they are defined. In this (pithy) example, we have two Steps, and they use the same image (the latest Universal Base Image

for RHEL 8). The Step defines a name, the Image to use, and the command(s) to execute in this individual Step. In this simple example, step one outputs the contents of a file (it shows the release version of the OS), and step two echoes "Hello World." I'm not proud.

A quick note on the concept of a ClusterTask: when you use OpenShift, you will see references to Tasks and ClusterTasks. A Task is project-bound; you can instantiate and use it within your project, bound by the RBAC. A ClusterTask is a preinstalled task at the cluster scope, meaning if you can create a Pipeline, you can use those as well as the ones at your project level.

This is very important. You are not going to want to create every Task from scratch every time you build a Pipeline. There's a lot of interest in the community around Tasks and an online repo of useful ones built by others at the Tekton Hub. OpenShift installs a set of Red Hat–curated Tasks when you install the Pipeline Operator. These are exposed as a `ClusterTask` (and have examples like `git clone`, `buildah`, and `source-2-image`) and can be used in the construction of Pipelines.

In addition, the **Pipeline Builder** UX, which we will have a peek at in a recipe, allows you to add Tasks to a Pipeline like building with Lego, and you can add Tasks defined in your project, a ClusterTask from the cluster itself, and search the Tekton Hub.

I love writing code, but even I will use a prebuilt ClusterTask rather than craft one. Reluctantly.

Pipelines: Sticking the bricks together

Pipelines are surprisingly simple beasts to define. The abstraction of behavior into Tasks makes the Pipeline an orchestrator as opposed to a definer, and it does it using a very simple field: `runAfter`.

Here's a simple example of a Pipeline definition:

```
apiVersion: tekton.dev/v1beta1
kind: Pipeline
metadata:
  name: pipeline1
spec:
  tasks:
  - name: hello
    taskRef:
      kind: Task
      name: task1
  - name: uth-message
    taskRef:
      kind: Task
```

```
    name: task2
  runAfter:
    - hello
- name: goodbye
  taskRef:
    kind: Task
    name: task3
  runAfter:
    - uth-message
finally:
  - name: cleanup
    taskRef:
      kind: Task
      name: taskcleanup
```

A very quick explanation:

- The pipeline executes `hello` first.
- `uth-message` only executes once `hello` has successfully completed.
- `goodbye` only executes once `uth-message` has successfully completed.
- `cleanup` runs after all the tasks have completed *or* if any of the steps fail.

Note that the Pipeline does not define any of the Tasks because of the abstraction. This does mean that the Tasks `task1`, `task2`, `task3`, and `taskCleanup` must exist and be consumable by the user running the Pipeline. Also worth noting is that you can use PipelineRuns, discussed next, to completely define the Pipeline in a single definition (which is good for shipping a Pipeline around with your code and not requiring or assuming Tasks have been independently instantiated on the cluster). This involves putting the Tasks and Pipeline definition into the same file as the YAML for the PipelineRun.

The temporal instances: TaskRuns and PipelineRuns

You will be noticing that I have specifically pointed out that tasks and pipelines are "cookie cutters" on a number of occasions (I like the phrase). There's a very good reason for that, and it's Tekton's design that makes this such a cool implementation. Tasks and Pipelines only ever define what to do. When you want to execute a Task or a Pipeline, you create a new object that does the actual execution. These `Run` objects specify the parameters and configuration to be provided into the task or pipeline and then store the result of the execution.

With a `TaskRun`, the object instantiation executes the Task, providing parameters, then stores whether the Task was successful or failed. It stores the logs of all executed Steps within itself.

With a `PipelineRun`, the object instantiation marshals and executes all the TaskRuns needed to execute the Pipeline. It injects parameters and then stores the results of each TaskRun.

These objects act as instantiators and audit devices for the Pipelines. They are temporal instances (and immutable once created). One of the nicest things about these is that you can craft them, and they act as the creation point. If you want to test a Task you have written, you write a very simple TaskRun YAML definition and push it to the cluster.

Saving stuff: Workspaces

As we have talked about at length in the book so far, containers are ephemeral. You fire them up, you change files, you shut them down, and the file changes are lost, unless you are using persistent volumes. When Tekton was designed, they kept this in mind, and the whole structure of Pipelines and Tasks was built around an abstracted approach to allow persistence.

By design, they kept this very well abstracted. Steps within a Task, regardless of whether they share the same Image, are not guaranteed to share the same file space. Instead of individual steps/images defining the storage they need, the Tasks and Pipelines make reference to a concept called workspaces.

Put simply, workspaces are named components stated by Tasks that are provided, by name only, by the Pipeline definition. When you *execute* a pipeline using a `PipelineRun`, or if you execute a Task by itself using a `TaskRun`, you specify the implementation of the storage then. Until you instantiate the action, the storage is always abstract.

This is a very clever way of doing it. When executing a Pipeline or Task, you can specify how the workspace is provided, and the Tasks define how the abstract workspace is mounted.

Let's look at an example to see how this works in action. First, the Pipeline definition:

```
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: ubifilepipeline
spec:
  tasks:
    - name: ubifilecreate
      taskRef:
        kind: Task
        name: ubifilecreate
      workspaces:
        - name: working
          workspace: workingarea
```

```
    - name: ubifilecheck
      runAfter:
        - ubifilecreate
      taskRef:
        kind: Task
        name: ubifilecheck
      workspaces:
        - name: working
          workspace: workingarea
    - name: ubifileremove
      runAfter:
        - ubifilecheck
      taskRef:
        kind: Task
        name: ubifileremove
      workspaces:
        - name: working
          workspace: workingarea
  workspaces:
    - name: workingarea
```

I have bolded the important bits to show how the Pipeline wires it together. The Task definition specifies it needs a workspace. The name is the name that is provided to the Task itself, and the workspace is the name of the workspace defined by the Pipeline. In this case, `workingarea` is the Pipeline storage, and `working` is the workspace as seen by the Task.

Now look at the Task definition:

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: ubifilecreate
spec:
  steps:
    - name: id
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - cat
      args:
        - /etc/redhat-release
    - name: echo
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - echo
      args:
        - "Creating the file in the workspace"
    - name: createfile
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - touch
      args:
        - "/uth/test.txt"
  workspaces:
```

```
    - name: working
      mountPath: /uth
```

Again, I have bolded the important bit. The Task defines how the named component it is provided with (in this case, `working`) is physically mapped (to the directory `/uth`).

That combination of the Pipeline and Task wires the abstracts together, but how does that actually get mapped to storage?

This is the job of the temporal instance, in this case the PipelineRun. Have a quick look at this extract from an example PipelineRun that was used to execute this Pipeline:

```
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  annotations:
    pipeline.openshift.io/started-by: utherp0
  name: ubifilepipeline-s1tkv6
  labels:
    tekton.dev/pipeline: ubifilepipeline
spec:
  pipelineRef:
    name: ubifilepipeline
  workspaces:
    - name: workingarea
      persistentVolumeClaim:
        claimName: pipelinepvc1
```

Again, I have bolded the bit that is of interest. The workspaces definition in the PipelineRun maps a PVC to the name `workingarea`, which then permeates down to the Tasks.

You can have a number of workspaces in your Pipelines and, more importantly, by using PVCs you can then consume data that is produced by the Pipeline using other applications.

Triggers: Webhooking your Pipelines

Ahh, webhooks. As a developer who was there when the internet started (again, old), I remember all the hassles we had with trying to get something to trigger something. This was back in the good old days of servlets, for all you Java nerds out there (and yup, I'm still a Java nerd). What I would have given for a nice `http` endpoint I could drop stuff into that would do something magical was a lot.

So fast-forward 10 years or so (cough, my brain still thinks 2000 was 10 years ago). We now have OpenShift and Kubernetes, and we have Tekton. Tekton is great. The abstraction of Tasks and Pipelines makes sense. Triggering a Task or a Pipeline by crafting a `TaskRun` or a

`PipelineRun` is fine, but what about being able to trigger the start of a `PipelineRun` from outside of the system, and not through the declarative Kube API?

Tekton has this cool thing called an `EventListener`. When you instantiate one of these, it stands up an `http` endpoint that acts upon the arrival of traffic. You call the URL, bundle your `http` header and payload into bits, and throw it somewhere. The `EventListener` acts as a marshal point for the data, and Tekton provides some additional object types for the processing and direction of the event. These include `EventListener`, `TriggerBinding`, `TriggerTemplate`, and `Interceptors`.

EventListeners

These objects define the wiring for the process. An example is:

```
apiVersion: triggers.tekton.dev/v1alpha1
kind: EventListener
metadata:
  name: eventlistener1
spec:
  triggers:
  - bindings:
    - ref: triggerbinding1
    template:
      ref: triggertemplate1
```

Again, keep an eye on the version. As time progresses, this might change as the objects mature. What the `EventListener` specification is saying is that it defines a trigger (more in a second) that has bindings defined by a named object called `triggerbinding1` and a template defined by a named object called `triggertemplate1`. Yeah, not overly helpful at this point, but it will make sense as we go through the objects.

TriggerBindings

These objects define the binding of the parameters received in the request (via `http` header and parameters) into named parameters for the Pipeline process. Here's a simple example that demonstrates the way it works:

```
apiVersion: triggers.tekton.dev/v1alpha1
kind: TriggerBinding
metadata:
  name: triggerbinding1
spec:
  params:
  - name: param1
    value: $(body.param1)
```

```
  - name: param2
    value: $(body.param2)
```

This specification states that the binding will produce two consumable variables, `param1` and `param2` (I like to think I am imaginative, but evidence is starting to mount that I am not).

The first parameter, `param1`, is defined as the value from the body of the request called `param1`. This will be a parameter in the `POST` or `GET` of the delivering URL—for example, `http://myeventlistener?param1=wibble&param2=wobble`. This mapping takes the form of JSONPath, as we discussed with `oc`, so if you deliver a JSON payload, you can extract information using that clever little technique.

TriggerTemplate

This object is the clever bit. This defines the `PipelineRun` to execute, and maps the params provided by the binding into the `PipelineRun` specification itself. Have a look at this example:

```
apiVersion: triggers.tekton.dev/v1alpha1
kind: TriggerTemplate
metadata:
  name: triggertemplate1
spec:
  params:
  - name: localparam1
  - name: localparam2
  resourcetemplates:
    - apiVersion: tekton.dev/v1beta1
      kind: PipelineRun
      metadata:
        labels:
          tekton.dev/pipeline: triggeredpipeline
        name: pipeline1-triggered-$(uid)
      spec:
        params:
          - name: localparam1
            value: $(tt.params.param1)
          - name: localparam2
            value: $(tt.params.param2)
        pipelineRef:
          name: triggeredpipeline
```

I have bolded the important bits. The specification defines two local parameters to be used in the `PipelineRun` specification, `localparam1` and `localparam2` (again, not overly imaginative). The specification for the `PipelineRun` then defines how these local parameters map to parameters provided by the template.

The `tt` object in the JSONPath refers implicitly to the map of parameters created by the binding object (which, if you remember, was bolted with this template using the `EventListener`). So, for example, `tt.params.param1` refers to the binding's parameters, with the name `param1`.

You'll notice that the template defines the `PipelineRun`. So in English, what happens is the `EventListener` receives a request via `http`. It uses the binding to attach values from the request into local parameters, and the template to build a `PipelineRun` object. This object is then instantiated, which runs the Pipeline (if you look at the template code above, the `pipelineRef` field says which Pipeline to execute, in this case, one called `triggeredpipeline`) with parameters inserted as needed.

While this is a superb way to do it, there is also one more component provided by the triggers that gives a last piece of required functionality. This is a component because it is not a standalone object as such; it is an optional field or component of the `EventListener`.

Interceptors

These optional components allow you to change the data being provided before it is mapped into the local parameters. This is to allow for data cleansing, data enhancement, and the like. The interceptor functionality is applied before the binding is called, so the binding works off of adjusted data rather than the original data provided in the request.

These are complicated beasts and have specific use cases. It's worth reading about the different types and examples on the Tekton website. A good use case is checking and changing GitHub files depending on the trigger.

I have saved the fun stuff until the end of the chapter.

Recipe 7:1: Creating and executing a Task

Start by logging into the Developer Sandbox. If you are not using the Sandbox, make sure you (or Ops) have installed the OpenShift Pipeline Operator.

Select **Tasks**. The content pane will appear on the right, and it should be empty (unless you have been playing already). Now click **Create Task**.

You are taken straight into the YAML editor for tasks. Delete the content and replace it with:

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
```

```
  name: exampletask1
spec:
  steps:
    - name: whatversion
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - cat
      args:
        - /etc/redhat-release
    - name: sayhelloworld
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - echo
      args:
        - "hello world"
```

Then click **Create**. We now have a Task definition in our project.

Click **Pipelines** in the left-hand navigation, then **Tasks**. The Task content pane in the **Administration** viewpoint gives you a selection of tabs allowing you to drill down into the related objects. Click the **TaskRun** tab.

Here we have a **Create** option (top right). Click that, select `TaskRun` from the list, and a YAML editor will appear. As with the Task, delete the contents and enter this:

```
apiVersion: tekton.dev/v1
kind: TaskRun
metadata:
  name: exampletestrun1
spec:
  taskRef:
    name: exampletask1
```

The minute you click **Save**, the `TaskRun` will start to execute. Give it a minute or so to pull the Image and run the steps, and the status of the `TaskRun` should change to **Succeeded** (you will be taken back to the `TaskRun` tab when you save the object definition).

Now click the TaskRun name itself in the TaskRun content pane. The UX will render a content pane with an overview of the TaskRun. Switch to the **YAML** view. Scroll down to `status:` and you should see something like the following:

```
status:
  artifacts: {}
  completionTime: '2025-02-07T08:33:39Z'
  conditions:
    - lastTransitionTime: '2025-02-07T08:33:39Z'
      message: All Steps have completed executing
      reason: Succeeded
      status: 'True'
      type: Succeeded
```

I have bolded the bits of interest. All steps succeeded, therefore the result of the composite Task was **Succeeded**. It does make me laugh a bit, though. It is type `Succeeded` because it succeeded. Ah, Kubernetes.

If you now scroll down to the bottom, you will see something interesting. The `steps:` component lists the name of the steps and the result (both succeeded), and beneath that, you actually have the step specifications thus:

```
taskSpec:
    steps:
      - args:
          - /etc/redhat-release
        command:
          - cat
        computeResources: {}
        image: 'registry.access.redhat.com/ubi8/ubi:latest'
        name: whatversion
      - args:
          - hello world
        command:
          - echo
        computeResources: {}
        image: 'registry.access.redhat.com/ubi8/ubi:latest'
        name: sayhelloworld
```

This shows the exact commands and images used in the steps. Interestingly, rather than define a `TaskRun` that uses a Task, you can put the Task definition directly into the `TaskRun`, which means you can execute a `TaskRun` without having previously created and added the Task into your project. This sounds a little counterintuitive, but it makes a lot of sense if, say, you have a Pipeline that does a specific build and you store the `PipelineRun` and `TaskRun` objects with the original source material. This effectively provides code-as-config for the build process that requires no preinstalled components on OpenShift. As with the `TaskRun`, the `PipelineRun` can define the Pipeline itself as part of its own specification. Obviously, for reuse, it makes sense to define the Tasks and Pipelines separately, but this integrated "code-as-config" is very useful in certain use cases.

Recipe 7:2: Creating and executing a Pipeline

There are two easy ways to create a Pipeline in OpenShift. Pipelines are native objects (the OpenShift Pipeline Operator adds Pipelines, PipelineRuns, Task, TaskRuns, ClusterTasks, and all the Trigger objects to the cluster lexicon), so you can enter YAML. In this recipe, we will create a Pipeline from scratch using YAML, and in the next recipe, we will use the very cool Pipeline Builder UX in the OpenShift UX.

To make this example a little more fun, we are going to create a reasonably complex Pipeline (with very simplistic Tasks) to show how easy it is to build complexity using the simple `runAfter` command.

Select **Pipelines/Tasks**, hit **Create**, and choose **Task** again.

Wipe the YAML and enter this, then click **Save**:

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: ctask1
spec:
  steps:
    - name: id
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - cat
      args:
        - /etc/redhat-release
    - name: echo
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - echo
      args:
        - "In task 1"
```

Repeat the above step to create another task. This time, enter the following, and then click **Create**:

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: ctask2
spec:
  steps:
    - name: id
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - cat
      args:
        - /etc/redhat-release
    - name: echo
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - echo
      args:
        - "In task 2"
```

You might spot a pattern here. We are going to create eight of these highly uninteresting Tasks, plus a finisher. Bear with me, it's worth it.

Enter the Task 3 YAML, then click **Create**:

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: ctask3
spec:
  steps:
    - name: id
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - cat
      args:
        - /etc/redhat-release
    - name: echo
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - echo
      args:
        - "In task 3"
```

Enter the Task 4 YAML, then click **Create**:

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: ctask4
spec:
  steps:
    - name: id
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - cat
      args:
        - /etc/redhat-release
    - name: echo
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - echo
      args:
        - "In task 4"
```

Enter the Task 5 YAML, then click **Create**:

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: ctask5
spec:
  steps:
    - name: id
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - cat
```

```
      args:
        - /etc/redhat-release
    - name: echo
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - echo
      args:
        - "In task 5"
```

Enter the Task 6 YAML, then click **Create**:

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: ctask6
spec:
  steps:
    - name: id
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - cat
      args:
        - /etc/redhat-release
    - name: echo
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - echo
      args:
        - "In task 6"
```

Enter the Task 7 YAML, then click **Create**:

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: ctask7
spec:
  steps:
    - name: id
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - cat
      args:
        - /etc/redhat-release
    - name: echo
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - echo
      args:
        - "In task 7"
```

Enter the Task 8 YAML, then click **Create**. Stick with me, we're almost there:

```
apiVersion: tekton.dev/v1beta1
```

```
kind: Task
metadata:
  name: ctask8
spec:
  steps:
    - name: id
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - cat
      args:
        - /etc/redhat-release
    - name: echo
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - echo
      args:
        - "In task 8"
```

Enter the finisher YAML, then click **Create**:

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: ctaskfinisher
spec:
  steps:
    - name: id
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - cat
      args:
        - /etc/redhat-release
    - name: echo
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - echo
      args:
        - "Finished!"
```

Hopefully you stuck with it. The **Tasks** content pane should now show `ctask1-9` and `ctaskfinisher`, in addition to any tasks we might have added before.

Click **Pipelines** in the left-hand navigation. When the content pane appears (and there should be no pipelines), click the **Create** drop-down and select **Pipeline**.

You will see the cool **Pipeline Builder**. We will use that in the next recipe. For now, click the **YAML View** radio box, wipe the contents of the YAML editor, and replace with this:

```
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: cpipeline1
```

```
spec:
  tasks:
    - name: task1
      taskRef:
        kind: Task
        name: ctask1
    - name: task2
      taskRef:
        kind: Task
        name: ctask2
    - name: task3
      taskRef:
        kind: Task
        name: ctask3
    - name: task4
      taskRef:
        kind: Task
        name: ctask4
      runAfter:
        - task1
        - task2
    - name: task5
      taskRef:
        kind: Task
        name: ctask5
      runAfter:
        - task2
        - task3
    - name: task6
      taskRef:
        kind: Task
        name: ctask6
      runAfter:
        - task4
        - task5
    - name: task7
      taskRef:
        kind: Task
        name: ctask7
      runAfter:
        - task6
    - name: task8
      taskRef:
        kind: Task
        name: ctask8
      runAfter:
        - task6
  finally:
    - name: cleanup
      taskRef:
        kind: Task
        name: ctaskfinisher
```

Once you click **Save**, the Pipeline will render a map of the Tasks that should resemble Figure 7-2.

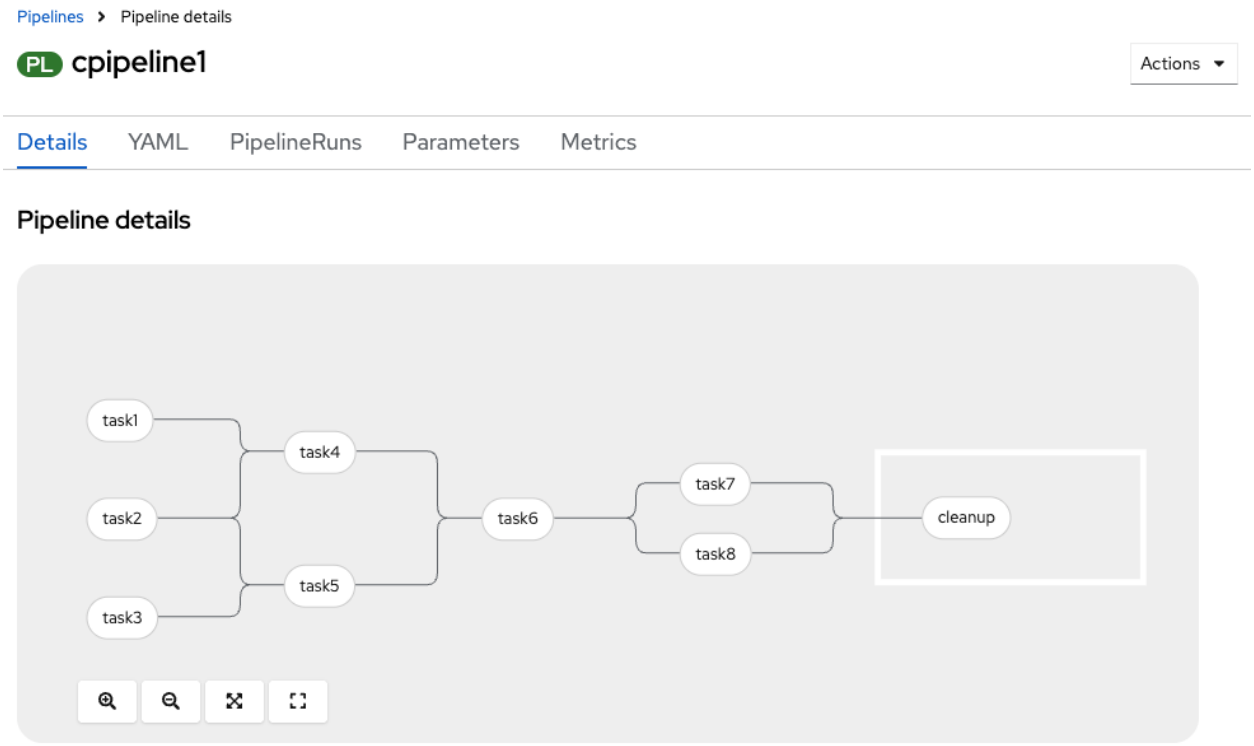


Figure 7-2: Rendering of a complex pipeline.

Just by using `runAfter`, which specifies which Tasks have to succeed first, we have created a nice little complex Pipeline with some parallelism as well.

Now click the **Actions** drop-down and select **Start**. Sit back and watch the Pipeline in action.

Each of the tasks will display the number of active and completed steps. They all have two steps, outputting the version of RHEL the image is based on and then outputting a simple message describing the Task they are in. Watch how the Tasks that run in parallel work: the subsequent Task that depends on the Tasks being executed will not start until all of the ones it defines as `runAfter` have successfully completed.

If you now click **Pipelines** in the left-hand navigation, the content pane will list the Pipeline but also show, via colors, the state of the last execution, in this case, green. Now click **PipelineRuns**; remember that the `PipelineRun` object will retain the audit of the Pipeline.

Click the PipelineRun name or link. You will now see a graphical representation of the executed run. If you hang the cursor over a task, it will list the steps and the time taken for the steps to complete. Click **task1**, and it takes you to the log/output pane. Here, you can select any of the tasks and see their output.

The `PipelineRun` acts as both a mechanism to execute a Pipeline, and an audit facility after the fact to tell you what happened.

Recipe 7:3: Fun with the Pipeline Builder

For this recipe, we will use the built-in Pipeline Builder UX in OpenShift to construct a Pipeline using some Tasks we will create, and a ClusterTask as well. We'll also look at adding a workspace for file persistence.

First, we will grab ourselves some storage. On the Developer Sandbox, we can create PVCs using the standard AWS storage class. If you are running on your own cluster, make sure you have a storage class installed with capabilities for RWO (read-write-once) volumes.

On the left-hand navigation, click **Storage** and then **PersistentVolumeClaims**.

Click the **Create PersistentVolumeClaim** drop-down and select **With Form**.

In the **Name** field, enter:

```
pipelinepvc
```

Set the size of the PVC to **1 Gb**, leave the **Volume Mode** as **Filesystem**, set the access mode to **RWO** and click **Create**. We now have a chunk of storage assigned to the project (but not yet bound).

Before we start playing with the builder, we are going to add three new tasks that interact with a shared file space. Switch to the **Developer** viewpoint and click the **Tasks** shortcut on the left-hand navigation.

Click **Create** in the top right, wipe the contents of the YAML editor, and replace with this:

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: ubifilecreate
spec:
  steps:
    - name: id
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - cat
      args:
        - /etc/redhat-release
    - name: echo
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - echo
```

```
      args:
        - "Creating the file in the workspace"
    - name: createfile
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - touch
      args:
        - "/uth/test.txt"
  workspaces:
    - name: working
      mountPath: /uth
```

Click **Create** to add this Task. Note that we are using a workspace in this Task called `working`. Click **Tasks** in the left-hand navigation and select **Create** again. Wipe the contents in the YAML editor and replace with:

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: ubifilecheck
spec:
  steps:
    - name: id
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - cat
      args:
        - /etc/redhat-release
    - name: echo
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - echo
      args:
        - "Checking the file has been created in the workspace"
    - name: checkfile
      image: registry.access.redhat.com/ubi8/ubi:latest
      script: |
        #!/usr/bin/env bash
        ls -alZ /uth/test.txt
  workspaces:
    - name: working
      mountPath: /uth
```

Click **Create**. We have one more Task to create, so click **Tasks** in the left-hand navigation, then **Create** again. Wipe the contents of the YAML editor and add this:

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: ubifileremove
spec:
  steps:
    - name: id
```

```
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - cat
      args:
        - /etc/redhat-release
    - name: echo
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - echo
      args:
        - "Removing the file from the workspace"
    - name: removefile
      image: registry.access.redhat.com/ubi8/ubi:latest
      script: |
        #!/usr/bin/env bash
        rm /uth/test.txt
  workspaces:
    - name: working
      mountPath: /uth
```

Click **Create** to add this Task. So we now have three tasks and a PVC ready. Select **Pipelines** in the left-hand navigation. Go to the **Create** drop-down and select **Pipeline**. If you are doing this recipe following on from the previous one, the content pane will default to the YAML view. Click the **Pipeline Builder** radio button.

The builder is an interactive visual tool for building Pipelines. Start by setting the **Name** field to:

```
cpipeline2
```

Now click the **Add Task** box. A search box will appear. In the search box, enter `git clone`. This dialog is the interface to the Tasks installed in the project, the ClusterTasks available preinstalled to the cluster, and connectivity to the Tekton Hub. You will see a set of Tasks that match the search terms. If you click the **git-clone** from **Community**, you will get an **Install and Add** button, which will download the Task from the Tekton Hub and add it to your Pipeline. At the time of writing, the ClusterTasks had been removed from the Sandbox. So for this use case, choose the **git-clone** task provided by Git, which is a community task.

Now look at the topology for the Pipeline. You will see a Task has been added called **git-clone**, and it has a red exclamation mark by it. This means the Task has mandatory parameters that have to be provided in order to use it in the Pipeline. Click directly on the Task icon, and a content pane will pop up on the right-hand side, listing all the parameters. The ones with a star indicate they are mandatory.

You will see that there are three mandatory parameters required: the name (which is preset to `git-clone`), a `url` from where the Git repo will be cloned, and a workspace called `output`, where the files will be written.

Set the URL field to:

```
https://github.com/utherp0/ocpnode
```

Scroll down to the **Workspaces** section. You will notice all the drop-downs are grayed out because the Pipeline has no defined workspaces as yet. In the content pane for the Pipeline, under the graphical representation of the Pipeline, are two sections: **Parameters** (for params at the Pipeline level) and **Workspaces**. Click the **Add Workspace** link in the **Workspaces** section.

Set the name of the **Workspace** to `myworkspace`. This is a Pipeline-level workspace name definition. Once you have done that, click the **git-clone** task (it will still have a red exclamation mark). Scroll down to **Workspaces** and you will see the drop-downs are now available. Click the **Output** workspace drop-down (the one with the asterisk) and select **myworkspace**.

You will notice the red exclamation mark disappears. The Task is now satisfied and can be executed. If you now hang the cursor over the Task icon, you will see a number of plus signs (**+**) appear. This allows you to add more tasks at those points in the Pipeline. Clicking on the left will add a Task before this one, clicking on the right will add a Task that executes after this one, and clicking the icon underneath allows you to add a Task in parallel.

For simplicity's sake, we will build a sequential Pipeline. Click the right-hand icon. A new component will appear that allows you to add another Task. Click this one and enter `ubi` in the search field.

You will see the three tasks we just created. Click `ubifilecreate` and then **Add**.

Again, you will see the task appears with a red exclamation mark, indicating it needs parameters in order to be correctly usable. Click the icon, and the content pane will appear on the right. You will see that the Task definition requires a workspace (go back and peek at the YAML we entered for the first task). Go to the drop-down with the asterisk, which represents the `working` workspace in the task, and select `myworkspace`, which is provided by the Pipeline.

The red exclamation mark will disappear. Now hang the cursor over the `ubifilecreate` task and click the plus-sign icon (**+**) to the right. We are going to add the second task to check the

file we created (which is written into the workspace). Click **Add Task** again and enter `ubi` in the search terms.

Click `ubifilecheck`, and it will appear, again with a red exclamation mark. Click the Task and set the `working` workspace to use the `myworkspace` from the Pipeline.

Once the red exclamation mark disappears, hang the cursor over the `ubifilecheck` task and click plus sign on the right. Click the **Add Task** box, enter `ubi` again, and select the `ubifileremove` Task. When the Task appears with a red exclamation mark, click it and set the workspace `working` to use the `myworkspace` from the Pipeline.

Now click **Create**, and *et voilà*, we have a four-step Pipeline. Now here's the cool stuff: on the Pipeline Details page, click **Actions** and **Start**.

Rather than just start the Pipeline, you will get a dialog. The workspace provided by the Pipeline in its specification needs to be physically provided. You will see a drop-down with the name `myworkspace`. Click it, and it gives you options as to how to realize it for the `PipelineRun`.

Select **PersistentVolumeClaim**, and a second drop-down will appear with the available PVCs in the project. We created one earlier, so clicking the PVC drop-down will allow us to associate `pipelinepvc` with the `myworkspace` workspace for this run of the Pipeline.

I really like this abstract approach to "satisfying" the Pipeline. Once you have selected the `pipelinepvc`, click **Start**.

Now watch the progress of the Pipeline in real time. You will see the `git-clone` Task copy files down from Git. You will see the `ubifilecreate` create a new file in the `PVC`. You will see a separate Task, `ubifilecheck`, check to see if the file exists, and finally, you will see another separate Task, `ubifileremove`, delete the file.

The PVC has retained file changes between the Tasks. If you click the completed Tasks in the Pipeline, you can look at the log output for each of the Steps.

Recipe 7:4: Setting up a trigger

For this recipe we are going to set up a very simple Pipeline and then wire connectivity from parameters delivered via JSON in the body of a URL request into the Pipeline; making a call to the URL with payload will make the Pipeline kick off, which is always fun to watch.

Log in to the Developer Sandbox (or your own cluster with OpenShift Pipelines Operator installed). In the left-hand navigation, select **Pipelines**, then **Triggers**.

The Triggers content pane will have four selectable tabs. We are going to create the objects we need in the order of the tabs.

First, click **EventListeners**. There should be none in the content pane. Click the **Create** drop-down and select **EventListener**.

Luckily for us, the way that Kubernetes/OpenShift works is that they don't check for required other objects until the object is used in anger. In this case, we will create the `EventListener` first with hooks to nonexistent components, and it won't complain. Result.

Wipe the contents of the YAML editor and replace them with this:

```
apiVersion: triggers.tekton.dev/v1beta1
kind: EventListener
metadata:
  name: eventlistener1
spec:
  triggers:
  - bindings:
    - ref: triggerbinding1
    template:
      ref: triggertemplate1
```

Click **Create → Workloads/Topology**.

We have a pod (Figure 7-3). This is the `http` endpoint and processor for incoming requests. You will notice it is missing a route (at the top right). By default, when an `EventListener` is created, it is internal. It is reachable by the internal SDN DNS. We will look at this later in the recipe.

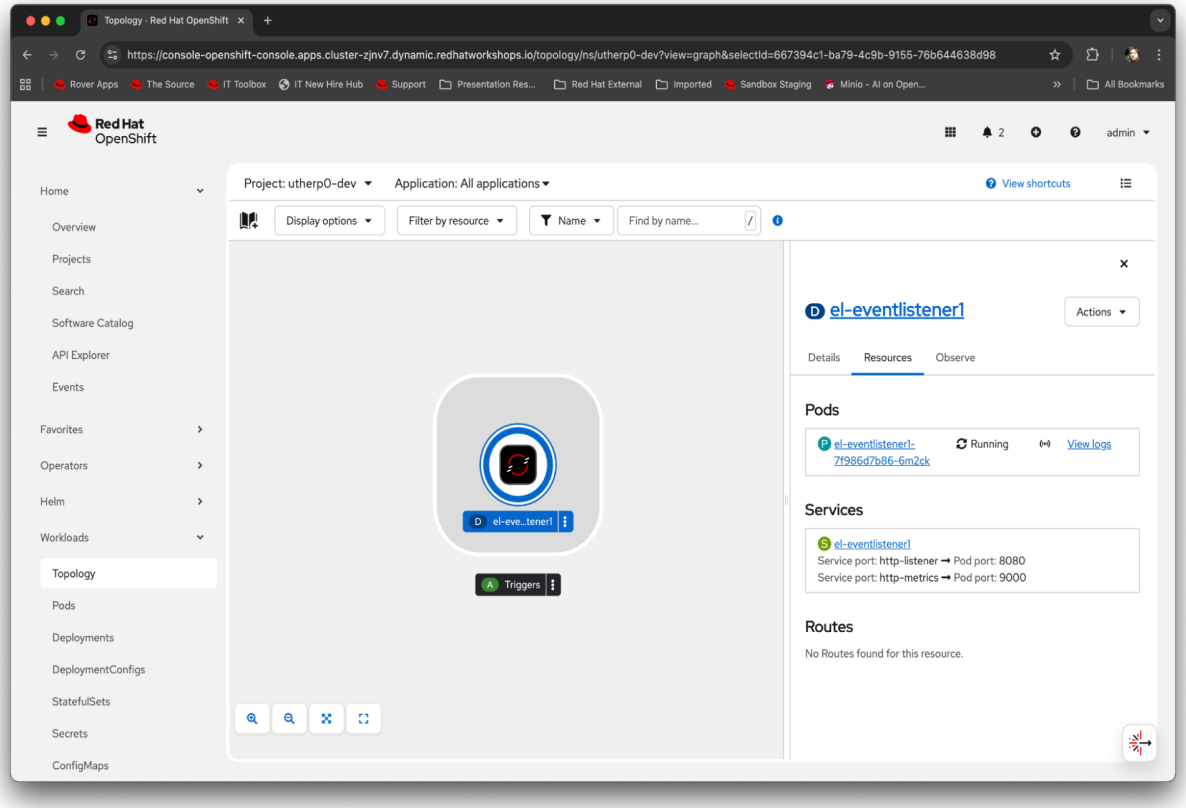


Figure 7-3: The `EventListener` pod.

Select **Pipelines** and then **Triggers** again. Now click the **TriggerTemplates** tab. Select **Create** and then **TriggerTemplate**.

Wipe the YAML from the editor and replace it with:

```
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerTemplate
metadata:
  name: triggertemplate1
spec:
  params:
  - name: param1
  - name: param2
  resourcetemplates:
    - apiVersion: tekton.dev/v1beta1
      kind: PipelineRun
      metadata:
        labels:
          tekton.dev/pipeline: triggeredpipeline
        name: pipeline1-triggered-$(uid)
      spec:
        params:
          - name: param1
```

```
          value: $(tt.params.param1)
        - name: param2
          value: $(tt.params.param2)
      pipelineRef:
        name: triggeredpipeline
```

Click **Create**. This is the template used by the `EventListener`. Notice that it makes a reference to a pipeline, called `triggeredPipeline`, in the template for the `PipelineRun`. We don't have that yet, but before we do that, we will create the binding to attach parameters being provided by the `EventListener` to the params to be consumed in the template (look at the `tt.params.xxx` components in the preceding YAML).

Click **Pipelines** and **Triggers** again. Click **Create** and choose **TriggerBinding**. Wipe the YAML and replace with:

```
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerBinding
metadata:
  name: triggerbinding1
spec:
  params:
  - name: param1
    value: $(body.param1)
  - name: param2
    value: $(body.param2)
```

So, that's all the wiring. Now we have to add the Pipeline (and Task) "cookie cutters" that will be called by the `EventListener` processing an `http` endpoint. Click **Pipelines** and then **Tasks**. Select **Create** and then **Task**. Wipe the YAML and replace it with this:

```
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: triggertask1
spec:
  params:
    - name: taskparam1
      type: string
    - name: taskparam2
      type: string
  steps:
    - name: id
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
        - cat
      args:
        - /etc/redhat-release
    - name: echoparam1
      image: registry.access.redhat.com/ubi8/ubi:latest
      command:
```

```
      - echo
    args:
      - "Param 1 - $(params.taskparam1)"
  - name: echoparam2
    image: registry.access.redhat.com/ubi8/ubi:latest
    command:
      - echo
    args:
      - "Param 2 - $(params.taskparam2)"
```

For the sake of showing how the parameters propagate, the Task has two params called `taskparam1` and `taskparam2`. These will be filled by the Pipeline, which is filled by the binding, which is filled by the template, which is provided by the `EventListener`. And breathe. Click **Create** to add the Task to the namespace.

Now click **Pipelines** and then **Pipelines**. If you have been doing the recipes in order, you should (hopefully) see a couple of Pipelines with green on them. Always a good sign. Click **Create** and then **Pipeline**. Switch to **YAML view** using the radio button at the top, wipe the YAML in the editor, and replace it with:

```
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: triggeredpipeline
spec:
  params:
    - name: param1
      type: string
      description: Example param 1
    - name: param2
      type: string
      description: Example param 2
  tasks:
    - name: task1
      taskRef:
        name: triggertask1
      params:
        - name: taskparam1
          value: "$(params.param1)"
        - name: taskparam2
          value: "$(params.param2)"
```

Note the mapping of the parameters at the bottom for the Task. Click **Create**.

So, we have all the pieces in place now. What we will do for ease of access is expose a port to the `EventListener`. This is so we can send an `http` request from outside the cluster. We could just use the internal SDN address, but it is worth knowing how you expose the endpoint.

Select **Networking → Services**. In the list of services will be an entry for `el-eventlistener1`. This is our "doorbell" for the `EventListener` on the SDN. Click the service link, labeled **(S) el-eventlistener1**.

Check out the service routing information. To expose access, we need to create a `Route` object. Click **Networking** and then **Routes** on the left-hand navigation pane.

Click **Create route**. In the data entry point for the route, change the **Name** to `eventlistenerroute`. Click the **Service** drop-down and select **el-eventlistener1**. Scroll down to **Target port** and in the drop-down, select **8080 → 8080 (TCP)**.

Click the **Secure Route**, then set the **TLS Termination** to **Edge** (read Chapter 4 for more on all the TLS stuff). Scroll down to the bottom and click **Create**.

As a test, click the route that is produced. This is the **Location** shown on the right of the panel.

Et voilà, we have a response. It's a broken one because we aren't providing any of the `JSON` parameters the `EventListener` needs, but at least it is listening.

Out of interest, select **Workloads/Topology**. Click the roundel for the **el-eventlistener**, and then click the **View logs** for the single pod that is running the event listener. The logs will show it received an event but was missing the data it needed:

```
{"severity":"error","timestamp":"2025-02-
11T12:49:11.400Z","logger":"eventlistener","caller":"sink/
validate_payload.go:42","message":"Invalid event body format : unexpected end of
JSON input","commit":"b27feb0"}
```

"I ain't got no data," in English. So, click **Pipelines/Pipelines**. You will see the `triggeredpipeline` pipeline, and it will not have had any runs. Now for the fun part.

Click the terminal prompt terminal icon at the top right of the interface. For those not on the Sandbox, either install the Terminal Operator on your cluster or do the following commands directly from your own terminal. You will need the `oc` command line and `curl`.

If you are using the terminal in OpenShift Container Platform, it should look something like Figure 7-4.

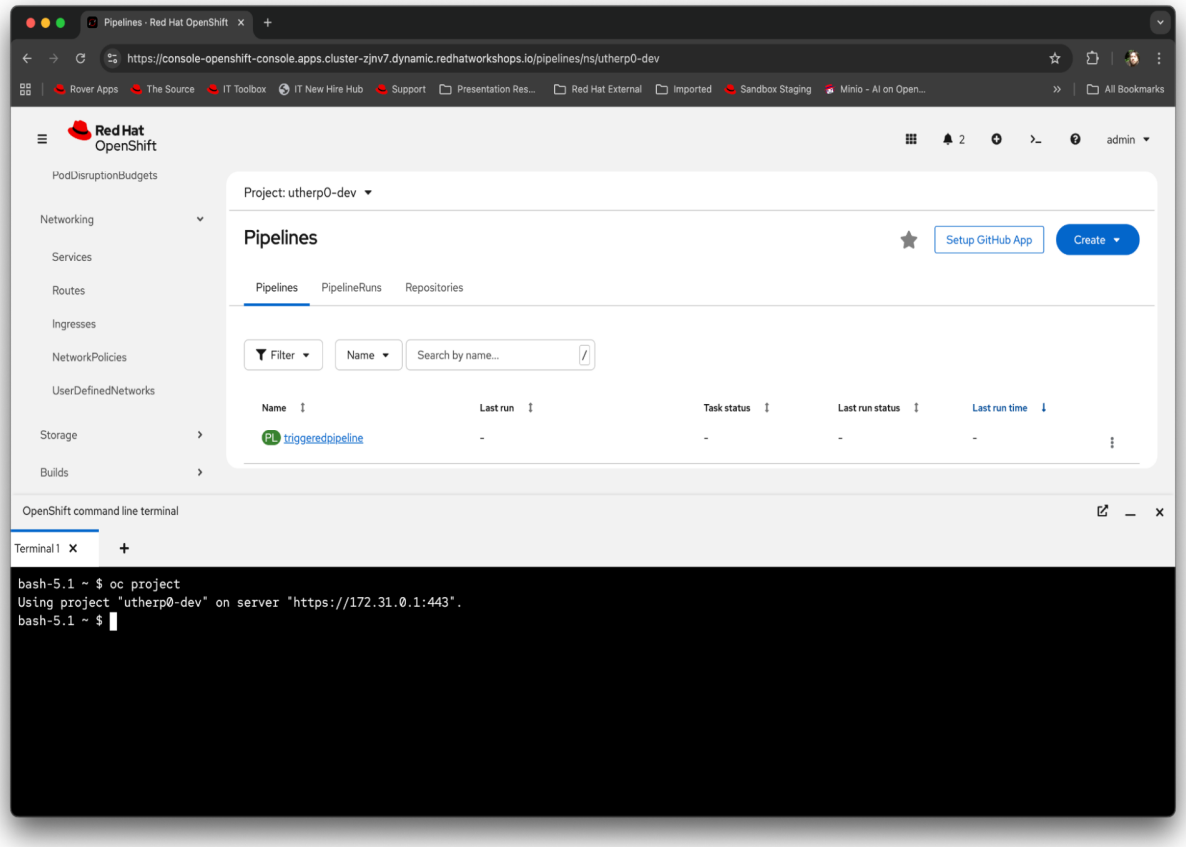


Figure 7-4: Pipelines overview pane with the built-in terminal active.

Enter the following commands:

```
oc get routes | grep route
```

You should get a single route back, showing the name and the URL.

Now enter this:

```
curl -X POST https://$(oc get route/eventlistenerroute -o
jsonpath="{.spec.host}") -H 'Content-Type: application/json' -d
'{"param1":"hello","param2":"reader"}'
```

Yeah, it looks a bit "scripty," but what we are doing is sending a `POST` request to the route with a data package in JSON, containing the parameters we want to provide. When you enter it, you should instantly see the `triggeredpipeline` start. Let it complete, and then click the green bar for the Pipeline. You can click each of the tasks to see its status. You will see:

```
step-id

Red Hat Enterprise Linux release 8.10 (Ootpa)
```

```
step-echoparam1

Param 1 - hello
step-echoparam2

Param 2 - reader
```

The three steps of the Task output the ID of the OS in the container, the value for `param1`, and the value for `param2`. I know, not exciting, but we have hooked an external URL into the execution, with parameters, of a Pipeline inside OpenShift. That's cool.

As I might have mentioned, Pipelines are immensely useful, and once it clicks that Tasks can use anything in a container, the sky is the limit.

CHAPTER 8

GitOps and the state watchman, Argo CD

Before someone came up with the term DevOps, developers and Ops were not... friends.

Developers were tasked with writing stuff. Ops had to run it in production. The responsibility for their (squeaky clean) machines being up and available fell squarely on the poor sods in operations, and developers (myself included) had a reputation of writing our software in our own environments and then "chucking it over the wall." We'd often include a hastily, last-minute scribbled set of instructions (`make sure this ENV is set`, `give the JVM a lot of heap and stack space`) about extra-application configuration to make the application work.

Why Ops hated me

I (and many developers like me) had a reputation of "tainting" the clean machines we were given. It wasn't wholly our fault. Back in the day, you would build an application against a specific JVM, and the machine that hosted it had to have that version of the VM. Prior to containerization, that meant that the whole machine was tainted to the application configuration, which wasn't part of the delivered application.

Long story short, Ops were always sighing when I showed up with my latest creation. Fast-forward some years and we had the concept of DevOps, which, hilariously, had Ops and developers sitting together, virtually or physically, with much better communication between the creator and the hoster.

But this still led to the classic problem. A developer delivers their end product and instructions on how to host it best. Ops deploy and maintain it, and things would break. Ops would have to do quick fixes (looking at you, `-Xmx`) to the production system to get the application to

perform correctly, and over time, the application plus its configuration would drift from the original intended deployment. It's a natural side effect of dividing responsibility. When the developer then delivered a new version of the application, it was often completely incompatible with the configuration of the production application because of this drift.

Then along came Kubernetes and containers. Finally, the application taint could be packaged, via code, with the actual application itself. In the old case of having a distinct version of, say, a JVM, the deliverable would be a container with this version installed. No longer did Ops have to change the configuration of the hosting machine. The container runtime hosts have none of the application baggage installed directly on them—it travels with the container image itself.

This was brilliant. Finally, all the container runtime hosts (OpenShift worker nodes) could be clean and untouched, and there was a definite "config-as-code" definition for the application and its associated objects.

And everyone was happy ever after. Not.

The missing piece

The great thing about OpenShift is the object model. Everything is defined as "code-as-config," and OpenShift maintains this "eventually consistent/reconciliation" approach. So, a developer builds their software, and it is deployed as an image, container, pod, and deployment with additional configuration. Let's say an Ops person deploys this as is, then walks away. Anyone with rights to those objects on the cluster can change them, and that includes automated processes (more on that when we talk about operators).

Here's a scenario: Ops deploy an application with two replicas, carefully sized so as not to interfere with other workloads in the cluster. A developer comes in with access rights, looks at the deployment, and says "nah, needs more oomph," and ups the replica count in the deployment to four replicas, which OpenShift then creates.

The application has drifted away from the optimal definition that the Ops person put in place. In technical terms, we were missing a link in the chain: deployment and post-deployment automated state compliance.

And along came Argo CD to fill the gap.

The GitOps workflow

First thing, Argo CD, like a lot of stuff in Kubernetes and OpenShift, suffers from overloaded term syndrome (OTS). There's a lot of reuse of terms for different things. Argo CD works with Applications and Projects. Forget for now what we mean by these terms in OpenShift.

Components that make up an active Application (Argo CD term) are chunks of declarative YAML that define the required state for the objects needed to instantiate this application.

GitOps requires you to store these definitions in a Git repository. This is the source of the requested state.

Argo CD acts by reading and monitoring this repository for changes and applies all of the required state to one or more target OpenShift/Kubernetes clusters. Argo CD acts as the creator and maintainer for the objects created at the cluster.

The objects active on the target cluster(s) are the source of current state. Argo CD works to ensure that the source of current state matches at all times the source of requested state.

Basically, if a change happens to the object state at the cluster that contradicts the state in the repo, Argo CD changes it back to match the source of requested state. It acts as a real-time watchman of the state of deployed objects on a cluster, which is extremely useful, and not just for developers like me.

Given that the whole of OpenShift's currency is objects and Argo CD can control and maintain the state of these objects, Argo CD becomes an amazingly powerful tool for cluster state control. Keeping your network state defined, your network policies, and users/groups (everything that is defined as an object), makes Argo CD a perfect 24/7 watchman.

Talking the talk, Argo CD style

One of the lovely things about the way operators work (and we will talk about them in depth in the fun section) is that they add distinct new objects to the cluster when installed. In the case of Argo CD, it adds three simple new objects that can be created and manipulated.

Application is an annoyingly overused term. In the case of Argo CD, an Application is a group of declarative YAML components that define a required state (sourced from a Git repo). The definition of an Application involves a name, a source, and a target namespace, but more on that in a second. The Application is the smallest atomic piece of currency used by Argo CD.

Project (dammit, Argo CD) is another multiply-defined concept in OpenShift. OpenShift uses projects as a grouping of objects and configuration. Argo CD uses its object `Project` to group together applications. It's about tidiness and cosmetics.

`ApplicationSet` is a very interesting newish component. This is a mapping of *multiple* `Application` objects to *multiple* target clusters. It is a "super-controller" of applications.

And from the object side, that is all it is. I just wish they had different names...

Behave yourself, GitOps

So, with the objects defined, let's look at the simple behavioral model of Argo CD. From a developer's point of view, it doesn't really influence us, but it is a nice bit of software logic.

So, given a Git repo with defined state and a cluster onto which that state has been deployed:

- If the requested state at the Git repo changes, these changes are applied to the target cluster.
- If the current state at the cluster changes and does not reflect the requested state at the Git repo, the state at the cluster is set back to the requested state.

It's that simple. The only slight caveat offered by Argo CD is the way in which this synchronization takes place.

Say you have an Application live, defined by and enforced by Argo CD. You want to change something about the state, but you want to do it in the middle of the night to avoid downtime and interruption. Argo CD supports two different synchronization models: automatic and manual. In the case of automatic, the system enforces changes whenever a difference is spotted. With manual, Argo CD holds back on applying the change but indicates that the Application is out of sync.

Whenever I demo GitOps, I always put it in manual because it's quick to change when you alter the state, but you can see the need for both of these strategies. The mode is applied at the Application level.

The canny of you will have noticed no recipes in this chapter. GitOps is brilliant, and I have lots of examples, but the Developer Sandbox doesn't have it currently installed. It's a concept that is worth knowing about as a developer, but it is more "Ops-y."

Kustomize

Argo CD takes advantage of a technology called Kustomize, which is worth having a quick chat about. Kustomize works as a preprocessor for Kubernetes YAML, which is a poor one-line description given how cool and flexible it is.

With Kustomize, you can define and group sets of object definitions. When you execute the Kustomize process, it takes all of the composite YAML files provided to it and generates a single YAML file with all the components in it and allows for parameter injection and programmatic changes.

Why is this important for Argo CD? Well, firstly, it uses a "shopping list" approach. Have a peek at this (very simple) `customization.yaml` file:

```
resources:
- deployment.yaml
- service.yaml
- route.yaml
```

Yeah, it's a simple example, but that one `customization.yaml` file will generate a composite single piece of YAML that can be pushed to a cluster. In fact, the `oc` command line supports it. If you want a quick peek (and I know I said no recipes in this chapter, but I am notoriously changeable), do the following.

In the Developer Sandbox, fire up the terminal (click the terminal prompt in the top right of the UX). At the prompt, enter:

```
mkdir git
cd git
git clone https://github.com/utherp0/gitopsdemo1
cd gitopsdemo1
cd app
ls -al
oc kustomize
```

So what have we just done? For a start, we have cloned my favorite repository full of fun GitOps stuff. (If you look in the `pipeline1` and `pipelinetriggers` directory, you might recognize the YAML). But more importantly, we have run Kustomize over a set of predefined YAML. When you did the `ls -al`, you would have seen the files. All of them, except `customization.yaml`, are object definitions. The `customization.yaml` is the shopping list. When we ran `oc kustomize`, it processed the shopping list and generated the composite, singular YAML to provide to a cluster to create *all* the objects needed (as defined in the `kustomization.yaml` file).

That was a simple example. You can do some fun and complex stuff using Kustomize, which can automate the provisioning of objects, especially ones that require, say, slight changes for names or clusters. Kustomize supports the concept of *overlays*. These allow you to define base versions of the YAML, then specify only the changes to be applied—inheritance, in other words.

It's a matter of timing

Kubernetes and OpenShift are both effectively "eventual state systems." You ask them to realize an object at a given state, and they go and do it. Like most modern systems, it's an asynchronous promise rather than a "done it" response when you throw an object definition into the cluster.

With Argo CD, this caused a little problem. An example case: you have a Kustomization file that says you need a deployment, a project, and a service. Stick them in a Git repo, set up an Argo CD application, and away you go.

Except, there's no concept of order there. Let's say that the deployment specifically states the project or namespace it needs to be in. If the cluster gets the definition for the deployment first, it will create the object, but the object will go into a non-reconciled state. It can't create the deployment because the project *doesn't exist yet*. If the cluster then gets the project to create, it creates it, but the cluster goes through a bit of "wobbling" in terms of the objects. It's not a critical failure, but it would be throwing events saying it can't create the deployment until the project has been created and is stable. It's a little messy.

So Argo CD came up with the idea of *sync phases and waves*, which sounds like an album from the Red Hot Chili Peppers, but in fact is an ordering system for the execution of the syncs.

Think of the operation of a sync as an atomic component. Argo CD acts to perform a sync when the state is out of sync, either because the requested state is different from the current state or vice versa. A *sync* involves pushing the definition to the cluster and waiting for definitive confirmation. It's not a "fire-and-forget" action; it's a receipted action.

By the addition of a label into the actual definition YAML for the objects, you can specify what is called a *wave*. This is a simple numerical value but indicates the order of this object. Put very simply, Argo CD will collate all the objects it needs to sync and then do them in order such that no object with a higher number is synced until all objects with a lower number have successfully synced. It's a very nice solution.

Phases are a different concept. With the atomic concept of a sync, you can now execute components as part of the sync at different points within the atomic process: `pre-sync`, `sync`, and `post-sync`. These operations consist of Jobs, an object in Kubernetes/OpenShift that executes a container as a single operation (that is, instantiates the container, executes the action, and shuts it down). It's quite an advanced feature, but it allows you to do setup and teardown around the sync operation.

Normally, you would do setup in what is called an `init-container` (like a Job, an image that is instantiated and executes, then shuts down) which is tied to a deployment. But the addition of a "before" and "after" operation is useful for actions specific to a "sync," like waiting for an external component to be ready or cleaning database tables out that were used for the pod's initialization.

PART 3

The fun stuff

The book is going to take a turn now. So far, we've looked at the basics, the bits that make OpenShift so cool to use from a developer's perspective. This section is where we have some fun.

When I get asked to do demos and deep dives on OpenShift, I'm often told what the attendees want to see. When serverless, virtualization, security and, yeah, even AI is mentioned, I get a buzz because there are so many good things to show and fun to be had from the developer's perspective around these concepts. Well, not security—that was my bread and butter for a long time, and I still have a soft spot when it comes to exploiting systems. I mean, "securing" systems.

Anyway, to avoid getting into trouble, I'll deftly change the subject. The following chapters dive deep into some of the newer and very important features that a developer should be aware of. Serverless is bliss—a brilliant way not only to use a cluster more efficiently, but to think about developing distributed and atomic software, making it much easier to maintain and, I dare to say, to write. We'll dive deep into both sides of serverless (Serving and Eventing) with some cool recipes for trying it on the Developer Sandbox.

Virtualization shouldn't be cool. It's a little old school, and the nature of virtual machines is counter to the idea of containerization—none of the density, ease of control, etc. But the way OpenShift does virtualization is brilliant and is a real help for developers with legacy applications on "die-on-the-vine" VMs. We will dive into what makes the tech so good, how to use it, and some recipes on playing with it in the Developer Sandbox.

Security will give you an oversight of the way OpenShift controls the workloads. Knowing this kind of thing will make debugging big applications in a cluster easier. The majority of problems you will see will be down to permissions. The model OpenShift uses for this level of control is very nice and, again, native Kubernetes objects. Oh, and little robots.

Artificial intelligence is a little more of a tricky subject. It's very much "the in thing" at the time of writing this, but there is a misunderstanding in the industry between predictive AI (the good stuff) and generative AI ("waffle parrots"—bear with me, it will make sense soon). This section will give a brief overview of the difference but also show how you can quickly have some fun on OpenShift with both (using the Developer Sandbox and the Podman Desktop tool).

And finally, we have a section on operators. This is interesting because, as you will learn, operators extend the object model of Kubernetes/OpenShift, but a developer can easily write them. This section will explain the basics behind what an operator is and provide a little overview of writing your own.

CHAPTER 9

Serverless with Knative

I'll let you in on a little secret: I wrote this book just to talk about serverless. Well, not really, but it is my favorite bit of OpenShift technology, and I love to go on a lot about it.

Let's start with the elephant in the room. “Serverless” is a daft name. There, I said it. It should be "compute-when-you-need-it," but that doesn't exactly roll off the tongue. Just like cloud means "someone else's computer," serverless means "someone else's compute."

Enough complaining; let's dive into it.

Why Kubernetes makes serverless essential

Let's start with a little story. Say I have a legacy system that has 10 core pieces of functionality that I have decided to split out into microservices. Of these 10 pieces of functionality, the composite system or outside world calls nine of them once a day (think of a service that returns a postcode). The tenth one is called every 10 milliseconds and does transactional activities.

I write 10 separate microservices as containers. I then deploy all 10 to my OpenShift cluster and sit back and watch the system run. 10 pods sat there patiently waiting for requests. One of them is being thrashed, and the others are just sitting there reading books, or whatever it is that frameworks do when they are not being called (garbage collection, normally).

The point is this: Kubernetes is designed as a "desired-state" engine. When you deploy something, you ask for a pod. That pod is deployed and stays active because the specification says it has to be there to satisfy the object definition. So, for 23 hours and 59 minutes a day, those pods are sitting there, consuming system resources and doing nothing.

They take up space. In fact, the node placement algorithm used by OpenShift to place new deployments on worker nodes takes into account the resource consumption of these mostly inactive components when placing new stuff. It's inefficient.

What serverless, or more accurately, the open source project Knative, does is introduce the concept of "on-demand" execution and resource consumption. In English, the applications are only resident and active when they are being used. When they are not being used, they are offline and consume no resources on the cluster.

Serverless basics

It's even simpler than it sounds to implement, although under the covers it is a neat technology. Put simply, serverless provides the ability to deploy applications that dynamically scale down to 0 replicas when not in use.

But, I hear you say, OpenShift allows you to downscale a deployment (via `replicaset`) to zero already, doesn't it? Good question. It does, but we need to talk about entry points.

If you downscale an application to zero using the standard way, the services remain active on the SDN (and the routes to the outside world), both of which throw unhappy errors when you connect to them. If you remember the chapter on networking and the discussion of that wonderful formatted error page from the router, you know what I mean. OpenShift and Kubernetes do not have the concept of automatic scale up or scale down.

The serverless components maintain a constant entry point that acts depending on the state of the application. If it receives traffic and the application is scaled down to zero, it scales it up and forwards the traffic. If the application is active, it just forwards the traffic. Here's the really cool bit: when it receives traffic and forwards it on, it starts a timer for the entry point. If that timer expires before any further traffic arrives, it scales the application down to zero (or another value, this is configurable, and we'll talk about this shortly). If it receives more traffic, it resets the timer.

It's that simple, and very clever. Yeah, there's a lot more to it behind the scenes, and we'll dive into how it works in the next section. But first, let's have some fun.

Recipe 9:1: The simplicity of creating a serverless component

Enough enthusiasm. Let's see just how easy it is to create a serverless component. It is ridiculously easy because of the way Knative is designed.

Firstly, log in to your Developer Sandbox account. If you are using your own cluster, make sure you or Ops have installed the Serverless Operator and set up an instance of Knative Serving (and Knative Eventing for later in this chapter).

We're going to use our faithful `ocpnode` application. But rather than follow the usual instructions, we are going to walk through the creation process, as it has one different but important step.

Select **Home/Software Catalog**. Click **Add → All services**.

In the **Filter by keyword** box, enter `node`. Click **Builder images** on the left. Select the one tile provided (**Node.js Builder Image**).

When the **Node.js** overview panel appears, click **Create**.

In the **Git Repo URL** field, enter:

```
https://github.com/utherp0/ocpnode
```

Here's the good bit. If you scroll down to **Resource type** under **Deploy**, it would have already chosen **Serverless Deployment**. And this is the point: serverless requires *zero* code changes in the application itself. Anything you can containerize can be deployed as serverless. We'll talk a little bit about PVCs and data persistence. That used to be an issue, but later versions of Knative handle the connection of PVCs, and now anything can be deployed serverless.

Quick note here: if you've been diligently following the recipes, you might have already gotten an `ocpnode` application running in the project. If so, either delete the `ocpnode` application or change the name of this one to `ocpnode2`, whichever is easier.

Leave everything else as is and click **Create**.

The **Topology** will instantly look different. Instead of a roundel, you have a **Revision** box, which will start out as empty. This is because the application is building. Once it has built, a roundel will appear; this is the application running for the first time. Sit and watch the roundel. There will be a minute or so for the timer to expire, but after that, it will do something cool. The roundel will go dark blue (indicating application shutdown) and then white (indicating the application has downscaled).

At this point, we have a deployed application, scaled down to zero. Now click the URL icon for the route (at the top right of the revision box). You'll spot an arrow going from it to the empty roundel with the label 100%. More about that in a moment.

The application webpage will appear in a separate tab. Now pop back to the **Topology** and watch the roundel again. After the timer has expired, it will scale down to zero.

Now click **Observe** and set the dashboard to **Knative Serving - Revision CPU, Memory, and Network Usage**. The graphs will tell a very cool story: when the application (via the Revision) is active, it will have consumption metrics; when not, nothing is being consumed.

Serving versus Eventing

We are going to dive into Eventing later in the chapter with a neat little recipe, but it's worth describing what the differences are here.

Firstly, Serving is the serverless side. It provides mechanisms for the control of entry points into an application that has been served as serverless, scaling up and down the replicas of that application and controlling the direction of traffic. By default, it works on traffic to the SDN service.

Eventing is a set of APIs that allow you to build an event-driven mesh over a set of serverless components. In English, Eventing allows you to drive the serverless application using events within the cluster rather than direct traffic to the service. Think of Eventing as a mechanism to build a microservice mesh driven by a standardized eventing structure—more on that later.

The basics of Knative Serving: More overused terms

Knative Serving is actually realized using four simple objects, but again, sigh, they have shared names with other Kubernetes objects. A quick note on this, as you will see it a lot: it's absolutely fine for objects to have the same name in an OpenShift/Kubernetes system. The name is actually a combination of the API that owns it and the object name. One of the objects provided by Knative Serving is called a `Service`, which clashes with—ta da—a Kubernetes Service. However, the actual name for the `Service` object in Knative is `Service from serving.knative.dev/v1`, whereas a `Service` object when referring to the SDN doorbells is a `Service from core/v1`. Yeah, I know, still confusing.

Anyway, to help with the linkage of the objects, I have borrowed an image from the Knative documentation (Figure 9-1).

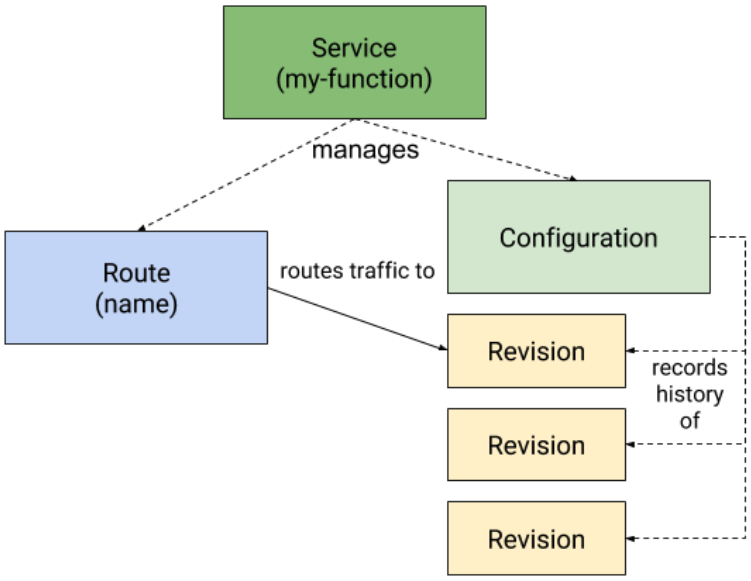


Figure 9-1: How Knative Serving resources coordinate with each other. Source: https://knative.dev/docs/concepts/#knative-serving.

There are four key objects here, but interestingly, a number of them are created by the others.

- **Service:** This is the key object. It acts as a controller and orchestrates the combination of the other components.
- **Route:** This is an object that provides the network endpoint to one or more revisions. More on those in a moment.
- **Configuration:** This contains the exact current deployable definition for the application. The key word here is current; think of this as the template for deployment.
- **Revision:** This reflects an immutable point-in-time version of the configuration.

So how do these hang together? When you create a Knative deployment, the service controls where traffic goes, using the definition in the route. The configuration contains the deployment reference; it uses that definition for the actual application. If you noticed when we created one, it was just a case of choosing **Serverless Deployment** instead of **Deployment**; in actuality, it creates but manages a deployment, tied to a revision.

Revisions are where the "gubbins" (technical term) happen. If you remember on the Topology in the last recipe, we talked about a revision appearing when we created the application, and the roundel then appearing in it. We also mentioned an arrow from the route, marked 100%. So, the Revision, and this is important, is an immutable object with the exact deployment

definition (for example, `ENV` variables, the image being used, etc.). When a serverless application is deployed, the contents of the configuration are used to create a revision. If you change anything in the configuration, a new revision will be created with an application as defined by the change. The revision is the application that is scaled up and down.

And the 100%? Well, you can configure the route to pass traffic to multiple revisions based on a weighting. Let's say you have a serverless application and you want to test a new version. You can change the configuration for the application, create a new revision, and then set the traffic for the route to, say, push 90% of calls to the first revision and 10% to the second.

The thing to remember about the revisions (and I might have mentioned this a number of times) is that they are immutable. They also remain until you cull them. When I first started playing with these for a demo at Red Hat Summit (anyone remember the Battleships game? I loved that), I was unaware they remained, and when I shipped the last version, I had something like 210 revision objects in my project. Whoops.

The mechanics of Knative Serving

Now, knowing the objects, we can describe exactly how the Serving mechanism works:

1. A Knative Serving application initially deploys without traffic based completely on the definition in the configuration and produces an initial revision.
2. Upon an initial healthy deployment, the application is now scaled down to 0 replicas. This is configurable within the configuration; there are use cases we will talk about in Scaling where you want an application always active.
3. When traffic arrives at the route (which is internally reachable through a single service point into the Knative Serving instance), it is forwarded to the appropriate Revision of the application. In the case of an initial single revision, this is 100% of the traffic.
4. If at this point the application described by the revision is scaled to 0, the application is upscaled depending on the configuration. A timeout, configured in the configuration, is then started. If no further traffic arrives at the route before the timeout expires, the application is scaled down depending on the configuration; if traffic arrives, then the timeout is reset.

And breathe.

Networking on a sliding (auto)scale

One of the best features and side effects of using Knative Serving is the ability to drive the scaling of the application by the network traffic itself.

We talked a little about autoscaling earlier. Basically, standard deployments in OpenShift, using the horizontal pod autoscaler objects, can change the number of replicas in the deployment based on a target percentage of CPU or memory consumption. This requires the deployment to have `limit-ranges` (so the autoscaler can compare the current aggregated usage to the maximum usage permitted). This works fine with CPU and memory, but there's nothing there for scaling based on traffic.

Along comes Knative, and suddenly you have an observable entry point for the application. So part of the specification for the Knative service allows you to directly specify the scale-up behavior based on combinations of the networking metrics.

In English, you can directly specify the scale-up behavior, and whether to disable scale to zero for the applications if you need them to be constantly resident, based on the concurrency and requests per second for the application. *Concurrency* is the amount of parallel connections the application has at once. *Requests* per second is the network throughput for each of the replicas.

This can be set for each revision by setting annotations within the `Service` object (this is imposed on the configuration, and therefore changes to it result in new revisions). You can also set default scaling behaviors for all serverless components within an OpenShift project and, if you are feeling dictatorial, impose default scaling behaviors cluster-wide for all Knative Serving components.

Here's a quick YAML snippet to show how easy it is:

```
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: myserverlessstuff
  namespace: myproject
spec:
  template:
    metadata:
      annotations:
        autoscaling.knative.dev/target: "200"
```

It does use a simplistic approach. By just stating a `target`, it indicates the number of concurrent connections. If you wanted to set a value for requests per second, you would use:

```
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: myserverlessstuff
  namespace: myproject
spec:
  template:
    metadata:
      annotations:
        autoscaling.knative.dev/target: "10"
        autoscaling.knative.dev/metric: "rps"
```

Simple, but very effective.

What is Knative Eventing?

As mentioned before, Knative actually provides two distinct components, Serving (described before) and Eventing. Put simply, Eventing provides the ability to hook the serverless components and concepts into event-driven architectures.

It differs from Serving in that Eventing actually provides a set of APIs and objects for simply routing unified events (CloudEvents) from producers to consumers (also known as *sinks*).

Sounds simple? Well, it is, and that's the joy of it. The key element from my own perspective is that eventing provides and conforms to the CloudEvent specification, which is a superbly simple and code-agnostic way of bundling events.

I spent years writing event-based systems (looking at you, Qpid) where I had to craft listeners, message protocols, and generators. I wrote so much code for mashing stuff into an event and ripping it out (without losing information). I have scars. So, to come across a technology that completely simplifies the whole create/send/process and hooks the consumers into serverless components is pure bliss.

We will get into the mechanics in a second, but think of Eventing as the way to build meshes of behavior for event-driven applications. The sinks, which are the consumers, are Knative Serving endpoints. Events drive the traffic into the components rather than direct traffic, and eventing allows you to control the routing, replication, replay, and dynamic delivery of events.

CloudEvents

Back in the day, I spent a lot of my life formatting messages. It wasn't fun. In fact, it is the key reason (other than the complexity of configuring a JMS system) I stopped working on event-

driven systems. That and trying to get my head around Kafka. Anyway, enough whining. CloudEvents solves all the problems I had in the past in a very simple way.

There's a great definition of the standard on the CloudEvents webpage, but put simply, a CloudEvent consists of a header and a payload. Nice.

The header has three mandatory pieces of metadata: `ID`, which is a unique identifier for this event; `Source`, which is a textual representation of the producer of this event; and `type`, which is a key for distributing the event. And that's it.

The combination of `ID` and `Source` gives a `UID`, which is used to identify an event when it is processed. We will talk about brokers in a second, but for now, just think of the `UID` as an indicator of having been seen. Once a broker broadcasts an event, the `UID` is used as a reference in order not to broadcast it again.

Eventing basic mechanics: Brokers and triggers

As with all the OpenShift stuff, Eventing provides objects that can be instantiated (as well as the APIs). The base atomic ones of interest here are the brokers and the triggers.

Brokers are a namespace/project bound endpoint for sending events to. They are reachable using the SDN (more on that in the recipe) and consume events using an `http/header` model. For us old developers, they receive `POST`.

They act as a marshaling point for the events. When an event arrives at the broker, it consults any trigger objects that exist in the project. A trigger defines a link between a consumer (sink, which can be a serverless component) and a CloudEvent type.

Here's a quick example of a trigger that should clarify the behavior model:

```
apiVersion: eventing.knative.dev/v1
kind: Trigger
metadata:
  name: trigger-quarkus
spec:
  broker: default
  filter:
    attributes:
      type: quarkusevent
  subscriber:
    ref:
     apiVersion: serving.knative.dev/v1
     kind: Service
     name: svctest1
```

In the specification, the broker indicates which named broker this trigger is attached to. In this case, I was lazy and set the name to `default`. The filter links the CloudEvent type of `quarkusevent` to the Knative serverless endpoint (the `Service` object we talked about earlier) called `svctest1`. Notice that this is all configurable; you can wire triggers to target any deployment that conforms to the API for sinks.

Now the broker, and it's, well, simple:

```
apiVersion: eventing.knative.dev/v1
kind: Broker
metadata:
 name: default
```

Yeah, I did say simple.

Building event meshes

Triggers and brokers are fine for the basics. We'll talk in a second about the persistence of events as well using brokers, but for more complex event mesh behavior, the Eventing model provides a couple of additional useful objects.

Channels provide an object for defining a delivery mechanism for fanning out events. This is done using subscription objects, which tie channels to sinks. This sounds complex, but it isn't. Imagine having a channel that takes an event type and delivers it to three consumers and another broker. You don't need individual triggers; you can craft the whole mesh itself through a single channel object.

Ephemeral or persistent?

The last thing on Eventing before we have a little bash with a recipe: by default, when you create a broker in a namespace/project, it is ephemeral. What this means is that once the event is received and delivered to any/all triggers that subscribe to it, the event is discarded (although the ID/source combination is retained to avoid resending events).

This is fine for most standard streaming architectures, but persisting brokers can be written as long as they conform to the Broker API. In fact, when you install the Eventing components of Knative into OpenShift, it provides two persisting brokers: one based on Kafka (via a topic) and the other on, shudder, RabbitMQ.

The advantages of these is that you can replay events through the system (on Kafka by changing the temporal point in the topic). This is great for repeating sets of events through

the system. Here is an example of a defined broker backed with the Kafka persistence approach for reference:

```
apiVersion: eventing.knative.dev/v1
kind: Broker
metadata:
  annotations:
    # case-sensitive
    eventing.knative.dev/broker.class: Kafka
    # Optional annotation to point to an externally managed kafka topic:
    # kafka.eventing.knative.dev/external.topic: <topic-name>
  name: default
  namespace: default
spec:
  # Configuration specific to this broker.
  config:
    apiVersion: v1
    kind: ConfigMap
    name: kafka-broker-config
    namespace: knative-eventing
```

Note the `broker.class` and the optional annotation for using an `external.topic`. You can then pass in additional configuration options, specific to the implementation (that is, Kafka or RabbitMQ), using a config map. If you're really brave, you can write your own data-backed implementations, as the Broker API provides all the endpoints you need.

To finish this chapter, let's set up and play with a simple Eventing example.

Recipe 9:2: Playing with the Eventing model

This one is a bit fun. Log into the Developer Sandbox (or your own cluster—we will be using the Terminal in the UX for ease of consistency of the command line). Click the terminal prompt icon at the top right to get the terminal window up.

When the terminal window appears, enter the following commands:

```
mkdir git
cd git
git clone https://github.com/utherp0/gitopsdemo1
cd gitopsdemo1/serverless1
oc kustomize > test1.yaml
oc create -f test1.yaml
```

So, what you have done is to clone the repository down to the terminal pod, execute `kustomize` to build a composite piece of YAML (if you look in that directory, you will see YAML definitions for the broker, the Knative service, and the trigger). We have dumped the composite YAML into a file and then pushed that file into the cluster.

In the terminal, enter:

```
kn service list
```

It might ask you whether you want to run the `kn` installed locally or download the one from the cluster. Enter `N` to stick with the local version.

So, `kn` is a brilliant little command line that uses your current Kubernetes config (in the case of the Terminal, you are pre-logged into the cluster) to allow you to interact directly with the Knative objects. It's an incredibly useful tool, if a little low level. With the preceding command, you would have seen your new service (and possibly the `ocpnode` one if you did the previous recipe). Now enter:

```
kn broker list
```

This is interesting. Look at the URL for the broker listed. When you create a broker in a namespace, they all use the same base route, which will be:

```
broker-ingress.(the namespace where knative eventing is
installed).svc.cluster.local
```

You should recognize the `.svc.x` bit; it is the DNS for the internal SDN service endpoints. What is interesting is the rest of the URL—it will be `/(your_project)/(broker_name)`. Every installed broker will have an endpoint like this, allowing you to easily throw events at them.

Now enter:

```
kn trigger list
```

Note that the sink is listed as `ksvc:svctest1`. This trigger will take events arriving at the broker with a type of `quarkusevent` and push them to that service.

So, how do you test it? I had this issue a long while ago and wrote a fun little application that allows me to throw events at brokers. Select **Home/Software Catalog**.

Choose **All services**, enter `node` in the **Filter by keyword** text box, then click **Builder Images** on the right-hand side. Click the **Node.js builder image** tile → **Create**.

Set the **Git Repo URL** to:

```
https://github.com/utherp0/cloudeventemitter
```

For tidiness, set the **Application** text field to `cloudeventemitter-app`. To do this, you will need to click the field and choose **Create Application**. For fun, leave the deployment type as **Serverless**. That way, this application will spin up when needed to throw events. Efficient.

Click **Create**, and the cluster will start to build and deploy the application.

So, we'll also have a bit of fun here to demonstrate how updating the configuration for a serverless component causes a new revision. Once the application has completed and spun up, then down, open the terminal again and enter:

```
kn services list
```

You should see a `cloudeventemitter` service. Because I am occasionally lazy (cough) and like to minimize the amount of typing I do in demos, I have defined an `ENV` variable for the deployment of the `cloudeventemitter` that allows you to pre-fill the broker address. In the terminal, enter:

```
kn service update cloudeventemitter --env CEBROKERURI=$(oc get broker/default -o
jsonpath="{.status.address.url}")
```

The configuration for the serverless service `cloudeventemitter` will update and a new revision will appear. Now click the route icon for the revision (the roundel *inside* the **(ksvc) cloudeventemitter** display box. The old-school webpage I have written for emitting CloudEvents will appear and, hopefully, the target broker textbox will contain your broker address. Also notice how the Pod for the `cloudeventemitter` itself spins up and down.

So, the application provides a text box for the target broker (which should be prefilled), a **CloudEvent Type** field, and a payload. Set the **Type** to `quarkusevent` (matching the trigger type). In the payload, enter:

```
{"payload":"stuff"}
```

If you can, have the OpenShift UX open in a window next to the tab with your emitter in it. Click **Emit Cloud Event**.

Pop back to the **Topology**. You should see the `cloudeventemitter` serverless component spin up, and then the `svctest` component (the one listening for the event) spin up. Click the `svctest` roundel and then **View logs**, and you will see the payload has arrived. Cool.

CHAPTER 10

A developer's guide to OpenShift security (and little robots)

Developers and security go together like... well, nothing, really. As developers, we accept (and normally try to work around) security as a necessary evil. Having come from a secure background where literally everything had to be "triple-stamped" by a security accreditor (most of whom wouldn't know a computer if they were hit with one), I have had security irrevocably bolted into my mindset.

Before we look at (a) how clever the security model in OpenShift is and (b) talk about the tiny little robots hiding out in projects, a little serious chat first.

No computer system is secure. Ever. If you want to secure a system, unplug it from the network and smash the keyboard. As long as data is coming in and there's a "squidgy chunk of meat" typing stuff, there will be ways to exploit a system. All these tools, scanners, and the like give you (in my opinion) a false sense of security. Any scan is only relevant at the time it is done with the knowledge it has. If you have a system that hosts components, you are always at the risk of bringing something in.

Developers have the option of doing things *right*. And by that, I mean writing your code to be secure. The key rule is always to assume the platform is compromised. If you want a maximally secure system, you need to first identify what your critical risk is: it's a normal application or data. And risk is all about reputational or physical damage.

There are actually only two genres of risk when it comes to applications: is your risk "application-centric," or is it "data-centric"?

"Application-centric" is all about availability. If your application goes offline or is slow, will that cause damage somewhere?

"Data-centric" is all about the data in the system. The application can be offline, slow, or whatever, but external visibility of data or data loss will cause critical damage, reputational or physical.

Always code defensively. Never offload anything you think is critical to the hosting platform or infrastructure. Yeah, it's risk offloading and responsibility pass-off, but that's not the point. If you have data you think is sensitive, encrypt it everywhere. Nullify any object where it might persist. Throttle connections yourself where you think there's an option for a DDoS.

I'll take off the "security hat" now. OpenShift has some core features designed to protect the framework and infrastructure. It was designed from the bottom up to be "default-secure" (as opposed to Kubernetes, which is "default-insecure"). The reason we are talking about this in this chapter is that as an OpenShift developer, you will fall foul of the security constraints at times, and it's absolutely worth understanding how it works under the covers.

Oh, and if you're like me, this is the holy grail of commands:

```
oc adm policy add-cluster-role-to-user cluster-admin (your username here)
```

But I didn't tell you that.

Let's talk robots.

Deny by default

OpenShift has a brilliant approach to security in that it doesn't let you do anything. By default, yeah, this is a standard approach for good secured software. We'll talk about roles in a moment, but for now, keep in mind that whatever a user wants to do in terms of objects is, by default, prohibited unless they have a specific role allowing them to do it.

What is a user?

When you log in to OpenShift (be it the Developer Sandbox or your own cluster), you go through an authentication method, which depends on how the cluster has been configured, and boom, you are logged in.

What is actually happening is the authentication method is making sure you are who you say you are and you have access to the cluster. Yeah, I know, obvious. This produces two object instances on the system: a user and an identity. The user is the base object of currency. The identity links that user to a defined authentication method. What is interesting about this is

that you can have two users with the same name on the system. We had this a lot when we were transitioning an authentication method from `passwd` to `LDAP`. I ended up with two users called `uther`. One was effectively `uther:passwd`, and the other was `uther:ldap`. As far as the cluster was concerned, they were completely different entities.

Now for the fun part: let's look at the process of creating, modifying, or deleting objects. You send a request to the control plane to change the state of an object. This is done as you. The changes to the object state are executed using a combination of the user and a service account. Or, as I like to think, a little robot. It's kind of like Wall-E, only not as lonely.

The object state change is performed from the context of a service account. This sounds a little obtuse, but when we look at *what* can be done, it starts to make sense.

Actions and objects

We'll talk about operators in Chapter 12, but this is relevant in our discussion here. Every object in OpenShift, owned by a controller or a "custom-controller" (operator), advertises a set of actions that can be performed on it. In reality, these map to functional endpoints in the controller. For instance, the controller that "owns" pods will have functions for create, modify, delete, and export these actions as the supported ones for the object.

What OpenShift/Kubernetes does under the covers is to maintain an RBAC model based on a mapping between users and roles, service accounts and roles, and the actions specified for the objects.

Sounds complex, but it isn't. The role defines exactly what actions can be applied to which objects. These are tied to users and service accounts (note both, more in a second) using another object called a role binding.

The role binding maps the link between the entity (user or service account) and the role definition.

What's nice about this is that you can see it on the Developer Sandbox (and on your own cluster, depending, snort, on your role).

Recipe 10:1: Looking at my roles

Log in to the Developer Sandbox if you are not already there. Select **User Management**.

Click **Roles**. This page might be a little, err, intimidating, but it does make sense. It shows you, as the User, your currently bound roles. If you scroll down, it shows *all* of the actions you can execute against objects and the controller/API that provides them.

Go to **Filter → Namespace roles**. This shows you the roles that are bound to you within the project. Click one of them (for example, `rbac-edit`) and it will list the exact actions you can perform on the resources (objects).

Recipe 10:2: Binding the roles

Now click **RoleBindings**. Click **Filter** and select **Namespace RoleBindings**. If you look at the **Subject kind** column, you will see the bindings are bound to three subjects: group, user, and service account.

Click **<*your name*>-edit**. When the (minimal) content page appears for the role binding, click the **YAML** tab.

This is where the magic happens. If you look at the `subjects:` component, it will list you as a User, along with the `rbac` Controller as the `API` it belongs to. Now look at the `roleRef;` components. This maps the cluster role `edit` to you.

It's a very flexible but very secure way of controlling who can do what at the finest level of functionality within the system.

Users and groups

You might have noticed the reference to a "group" in the previous recipe. To minimize the overhead of maintaining multiple users of the same type, the system provides an object called `Group`. This is a grouping of users by name. What the system does is impose a set of roles/role bindings to every member of a group, meaning you can define a set of roles for a set of users and not have to maintain them independently.

In addition, the authentication model can inherit and process additional information from the authenticator. For example, if you are using Active Directory (and sorry if you are, cough), you can synchronize the OpenLDAP groups from that into groups on OpenShift, making the administration of users and the control of behaviors simple.

You promised robots!

So, service accounts. When a user requests an action be applied on an object, the roles in the user are examined to see if they are allowed to do it. If there are no roles in the user and the action is being applied using a service account, the roles of the service account are then examined to see if it has access.

I like to think of these as little robots with privileges to change the OpenShift brain. And there's a good reason, internally, to separate these actions. To show it in action, let's have a quick play with the Developer Sandbox.

Recipe 10:3: Peeking at the 'bots

On the left-hand navigation panel, click **User Management** and then **Service Accounts**.

For now, we will ignore the `pipeline` one. It will make sense when we have discussed the three basic ones. You might also have one for `workspace`. Again, this little robot controls aspects of the terminal application for reasons we will talk about in a moment.

Every project has a default service account, imaginatively named... `default`. This is the core robot for the project. If you have two users with access to the project, they both use the same robot to make object state changes. But you'll also note there are two others: `deployer` and `builder`.

The service accounts have their own role bindings. `default` can do most things as an administrator in the project space, but when you build an image or deploy an application, the action of doing so changes some aspects of the cluster object model. To make it more secure, there are specific service accounts for these behaviors. The `builder` service account has rights to call actions in objects that can change the file systems on the worker node. Building an image means constructing something outside of the controlled file space of a pod. `deployer` has rights to interact with the registry in ways that `default` doesn't.

This separation is essential when, for instance, you have a project being used in production that must not do any builds. In that case, you remove the `builder` service account.

And here we need to talk about the way in which the user interacts with a service account. When a user makes a request to change an object, it is normally (unless you have `cluster-admin`, see hack earlier) the service account is used and inherits the user's roles *unless* it has bindings of its own that contradict, in which case it doesn't do it.

So why are we talking about this in a book about development? Well, when developing on OpenShift, you will occasionally hit a brick wall and not really understand why. Because OpenShift and Kubernetes are desired-state engines that focus at the object level, you can get odd information from an object as to why it can't do something.

The best example of this is something we talked about earlier: if you deploy an application and it uses a PVC, and the PV can't be bound or created, the deployment has no view as to why it can't start.

The OpenShift UX is usually very good at telling you you can't do something. Try opening the Terminal in the Developer Sandbox and enter:

```
oc get users
```

The cluster will respond with:

```
User cannot list resources "users" in API group "user.openshift.io" at the
cluster level
```

You've been nailed by the RBAC model.

CHAPTER 11

Those pesky legacy VMs

Ah, virtual machines. I still think they are witchcraft, but that's because I come from the dim and distant past when computers filled rooms, and you programmed by writing your code on a piece of gridded paper and giving it to the magicians who talked to the beasts.

Oddly enough, we are now at the point where (and I hate to break it) the age of the virtual machine (VM) is coming to an end. Not technically, of course. Every single organization has "that" computer: a growling piece of hardware sitting in the corner, supported by illicit purchases of hardware from questionable eBay sites, running a long-defunct hypervisor (Xen and hyper-v, even though Microsoft doesn't think so) on which some critical software sits. This software was written by a programmer who retired 10 years ago to raise birds on a farm. That software is going nowhere. No one knows what it does under the covers, but most of the core systems make an occasional embarrassed call to it.

Being serious for a second, VMs are not going anywhere. Containerization is the way of the future, period. There are no real use cases that cannot benefit in some way from containerization, but organizations are stuck with vast legacy estates. And trees.

I heard your brain put the brakes on then. What I call *trees* are those VMs that have been provisioned and stood up for *one* application. You know, the developers write something cool, like a `post-code` lookup service, and want to host it somewhere, so the overworked Ops fire up a VM that ends up in production, running a single application. I call them trees because it's a huge chunk of roots that have to travel with the tree.

In the age of containerization, these applications are prime for modernizing. The nature of file layers and sharing and clever image construction radically shrinks the footprints of these applications, and in most cases, migration to a container is simple.

The main problem I see when talking to end users is the concept of business as usual (BAU). Most Ops have huge estates of VMs, and the idea of doing a "big bang" modernization is

insane. Many organizations have put off legitimately moving to containers because of the technical debt of VM estates.

To be honest, there are a lot of "die-on-the-vine" VMs that are literally waiting to stop being used. It would cost too much in engineering and replatforming, and they are often used by one person or one team. So they sit patiently on a hypervisor waiting for the day when no one wants them anymore.

This has been a core problem with adopting Kubernetes and OpenShift for a number of people. Luckily, there's a new kid on the block, who is actually an old kid, come to help.

Virtualization in containers is absurd

Yes, it is, which is why it isn't the solution. OpenShift Virtualization is a very clever solution using the KubeVirt project, which combines both technologies in such a way as to retain the strength of virtualization but bring it into a modern context and orchestration system.

Take a step back. Containers work by building file systems *over* a shared kernel. VMs work by being executed directly on the kernel (based on the hypervisor type). They are two completely different approaches. Here's the cool bit: VMs running in OpenShift are just VMs running directly on the kernel, in this case on the worker node. The difference is in the way they are controlled.

With a normal VM system, you have a control plane (like VCenter), and hypervisor endpoints that respond to API calls to execute, stop, move, and configure the VMs on them. In OpenShift, *every* VM has its own control plane, which is, ta-da, a pod.

This is vitally important to understand. The VM is just a VM. It uses tech from 2008 (KVM, for *Kernel-based Virtual Machine*), running on a Linux box, which happens to be an OpenShift worker node (running Red Hat Enterprise Linux CoreOS). However, it is directly controlled and configured using native Kubernetes objects and a pod, called a `virt-launcher`. The pod acts as a conduit between OpenShift and the VM. The VM is reachable via service endpoints (and this is huge), and is controllable by changing the state of an object in OpenShift.

So what does this give you? It gives you a single next-generation orchestration system running and maintaining VMs. You can "lift and shift" your existing VMs directly into OpenShift and then have them monitored, controlled, and live-migrated exactly as you did with your original VM control plane, but instead, it is done by "code-as-config" in the same way as every other object.

That's probably a lot of stuff a dev isn't interested in, but there are some key features that make it hugely important to us.

Firstly, like OpenStack, OpenShift provides the ability to self-serve VMs. I can fire up the Developer Sandbox and create an instance of RHEL, Fedora, or CentOS. I can import my good old Windows VMs (I love demoing my Windows 10 box via a web page. Especially as it is constantly asking me to activate it).

Secondly, developers can modernize applications in their own time. "Lift and shift" straight into OpenShift, expose the applications as a Service ("doorbells"), then write or migrate the application to a container or pod and simply change the definition of the Service to direct traffic into the pod rather than to a port on the VM.

Enough chat, let's have a play.

Recipe 11:1: Fun with Fedora

You'll need an account on the Developer Sandbox, as usual. If you are running your own cluster, there are a couple of caveats.

OpenShift Virtualization is currently only *supported* on bare-metal worker nodes. It does work on VMs itself, but without `pass-through`, it will run like an absolute dog. On your own cluster, you'll need to install the OpenShift Virtualization Operator.

The other thing is you might have to delete some stuff off the Sandbox. It's a shared system with constrained resources per User. There are enough to run a VM, but if you have multiple other applications/pods, it might not start the VM.

Click **Virtualization** in the left-hand navigation pane, then **Catalog**. This is where you can create a VM. If you have a prebuilt image (and you can convert from, say, `VMDK` to the format for OpenShift, `QEMU`), you can upload it as a `PV` and then start it as a VM, but for this recipe, we will create from scratch.

Switch the content pane to **Template catalog**. Look for the tile for **Fedora**. Note that some tiles have “Source available,” and some don't. The ones that do have a prebuilt VM image that you push config into (using `cloud-init`), the others, such as Windows, you need a prebuilt ISO with a license. Click the **Fedora** tile.

Give it a name (the generated ones are unintentionally funny at times—I got `fedora-apricot-bedbug-48`), using the **VirtualMachine name**, and click **Optional parameters**.

This is the bit I always forget to do. If you don't give it a cloud-init password, it is hard to log in. Cough. Set the **cloud-user-password** to something secure. I use `password`. Again, cough.

Quick note: Make sure you are in your `*-dev` project. Sometimes the Sandbox will default your working project to the shared project it needs for the OS images, and you will not be able to create a VM in there. Have a look at the **Project:** indicator at the top of the content pane if you get a "no permissions" error.

Now click **Quick create VirtualMachine** and let the Sandbox do its magic.

While it's building, I need to talk about the VNC console. It's my favorite little bit of pointless tech. Any VM running on OpenShift can be VDI/VNC'd using a URL. You get a direct portal into the VM that appears in the browser window. In the case of Windows, it's the good old user interface, but with Fedora/RHEL, etc., you will get a terminal. You can easily install an `x-windows` component. The fun thing about the VM overview content pane is that it renders the VNC in a tiny window you can watch and use. You can't read it, but it makes a comical demo.

The VM should start quickly. Once it has, click the **Console** tab, and it will take you to the VNC UX. The user name will be `fedora`, and the password is whatever you chose. Hint: `password`.

And you now have a full Fedora VM running in OpenShift. It's that easy.

CHAPTER 12

Hello, operator? I'd like a new object

You made it to the last chapter! And surprisingly, so did I. I thought I'd finish with a quick overview and dive into the mysterious and brilliant technology of operators.

To get the concept of operators, you need to have absorbed the whole principle around OpenShift/Kubernetes—the object model.

A little fun history story for you: back in the days of OpenShift 3, which was the first Kubernetes-based version (and yes, I still remember fondly OSE2 with its cartridges and gears). OpenShift had a number of additional objects above and beyond the standard set in Kubernetes, which were things like routes and DeploymentConfigs.

Extending the lexicon

We've scratched the surface a bit, but Kubernetes, at its core, is just a set of `while-true` loops, called *controllers*, an API for submitting declarative definitions of an object, and a "brain" called *etcd* that stores the state of all objects in the system. Every object has a controller that owns it. This `while-true` loop waits for submissions that ask for state changes of the objects it owns. These arrive as events on the internal event bus. The controller takes the event off the bus, sets the state accordingly in the brain, and does "stuff" to make sure the physical object (if needed) matches the required state and then watches the object to make sure it remains reconciled with the state.

When OpenShift added new objects, it had to create and integrate new controllers. This was a code change to the core control plane. This was also back in the day when Kubernetes had an insane release schedule, and every time Kubernetes was updated, the OpenShift control plane had to be recompiled and retested with all the new objects. It was, pardon the phrase, a pain in the arse.

The Operator Framework

If you recall, Kubernetes actually calls the objects `resources`. Along came a great company called CoreOS, who wrote something called the Operator Framework. This was then used to build the whole of OpenShift 4, and here's how it works.

The control plane now has an additional object in it called a `custom-resource`. You can submit declarative definitions of a custom-resource, and the event bus will match the type of custom-resource as an object to registered operators that behave exactly like a controller. In fact, they were/are called custom-controllers. They behave exactly the same way as the `while-true` loops in the control plane but are hosted off the control plane. They can even be hosted off-cluster if you're mad; I like doing that with Java ones.

Anyway, the mode of operation is identical. The only difference now is that the control plane will look for `custom-resource` definitions if it can't find the object in the standard lexicon.

So I can create an operator that, say, provides an "orange" type of object. Going back to what we talked about around the RBAC, the Operators individually publish the objects (`custom-resources`) they support and the actions they provide. In fact, when you code an operator, there is a functional endpoint for every action you publish (and again, with a "security hat" on, you can control who can access each action).

Reconciliation: 24/7 automated maintenance

Also, the fact that it is a `while-true` loop means that operators can monitor the state of the objects they control in exactly the same "reconciliation" loop way as with Controllers, meaning you can do some seriously cool things. Imagine writing an operator that deploys all of the bits needed for a complex application, monitors it in real-time, and can change configuration based on metrics and application behavior.

This concept makes OpenShift infinitely extensible as a platform, as well as a lot of fun from the developer side. The initial downside was (and don't hate me) Go. The whole of Kubernetes and OpenShift is written in Go, a language that makes my teeth hurt, if only because it flagrantly does things that Java doesn't do on purpose (looking at you, multiple return values). Anyway, you can also write operators in Java now, which makes me very happy.

POSTSCRIPT

Developers are artists

So, that was a whirlwind ride around the cool things in OpenShift. There are a lot more; this book could be hundreds of pages longer, but I don't have time because OpenShift is so much fun to actually use as opposed to talking about it.

But if you've stuck with me all the way through, thanks. This is a work of passion; I love OpenShift as a development tool because it does everything I need it to do as a developer.

I moan a lot about the life of a developer. Hence the "grumpy" moniker. But the whole Kubernetes design and approach is so elegant and useful for a developer that I'm having to try hard to find things to moan about. And as a developer who wants to create, that's a good thing.

Always remember that developing is an art. We are digital wizards in a world that sees writing software as "just another job." We know differently. If you've ever felt that rush of seeing a build succeed after hours of trying, seen a webpage appear in the browser without spelling mistakes, punched the air when there's no stack trace (I'm not ashamed to admit I've done that—at 5:00 in the bloody morning), you know what it feels like to develop.

Why do I love OpenShift? Because it lets me *develop*.

Now go have fun.

About the author

Ian "Uther" Lawson has been an enthusiastic developer since 1981. He discovered Java in 1996 when it was still named after a tree. He spent way too long as a developer "gun for hire" before ending up at Red Hat, his spiritual home, in 2012.

Since then, Uth has been an advocate for making developers' lives easier, notably through the use of OpenShift, and making sure everyone knows just how cool being a developer really is.